

# 파이썬 프로그래밍

(데이터 분석)

## · 아나콘다 환경 설정 및 주피터 노트북 사용법

아나콘다란 수학, 과학관련 패키지들, 머신러닝이나 데이터 분석 등에 사용하는 여러 가지 패키지가 기본적으로 포함된 파이썬 배포판이다. 또한 아나콘다는 파이썬 가상 환경을 구축하는데도 유용하게 사용할 수 있다. 내부적으로 conda라는 환경/패키지 관리자가 존재하며 이 conda를 통해 패키지를 설치하거나 가상 환경을 관리할 수 있다.

### · 아나콘다 설치

<https://www.anaconda.com/>

### · 주피터 노트북 정의

웹브라우저(크롬) 환경에서 파이썬 코드를 작성하고 단계적으로 실행 가능하도록 하는 개발자 도구이다.

### · 주피터 노트북 실행 및 폴더 생성

[Anaconda]-[Jupyter Notebook] 프로그램 실행한 후 [PythonDataWorkspace]-[Pandas] 폴더를 생성한다.

### · 주피터 노트북 사용법

- 1) 초록색 : 편집모드
- 2) 파랑색 : 명령모드 (편집모드 상태에서 ESC)
- 3) 각 Cell의 Line Numbers : [View]-[Toggle Line Numbers]

#### ▪ '01.주피터 노트북 사용법.ipynb' 파일을 생성한다.

```
print('Hello World')
print('환영합니다.')
name = '연탄이'
print(name)
print('Ctrl + Enter : 현재 Cell 실행')
print('Shift + Enter : 현재 Cell 실행 후 다음 Cell 선택')
print('Alt + Enter : 현재 Cell 실행 후 다음 위치에 Cell 삽입')
```

```
Hello World
환영합니다.
연탄이
Ctrl + Enter : 현재 Cell 실행
Shift + Enter : 현재 Cell 실행 후 다음 Cell 선택
Alt + Enter : 현재 Cell 실행 후 다음 위치에 Cell 삽입
```

#### ▪ 클래스 + 함수 + '.' 입력한 후 'Tab' 키를 누르면 사용 가능한 명령어 목록이 나온다.

```
import random
print(random.randint(1, 45))
```

#### ▪ 함수 + '?' 입력한 후 'Shit' + 'Enter' 키를 누르면 설명이 출력된다.

random.randint?

Signature: random.randint(a, b)  
Docstring:  
Return random integer in range [a, b], including both end points.  
File: /opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/random.py  
Type: method

- 함수 + '??' 입력한 후 'Shift' + 'Enter' 키를 누르면 source code와 설명이 출력된다.

```
random.randint??
```

Signature: random.randint(a, b)

Docstring:

Return random integer in range [a, b], including both end points.

Source:

```
def randint(self, a, b):
    """Return random integer in range [a, b], including both end points.
    """
    return self.randrange(a, b+1)
```

File: /opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/random.py

Type: method

- 문장 앞에 '#'을 입력하거나 'Ctrl' + '/' 키를 누르면 주석처리를 할 수 있다.

```
# random.randint??
```

- 실행중인 경우 Number 대신 '\*' 표시되고 [중지] 아이콘으로 실행중지가 가능하다.

```
import time
for i in range(10):
    print(i)
    time.sleep(1)
```

## • 마크다운 사용법

마크다운이란 간단한 서식을 이용하여 일반 텍스트 문서의 양식을 편집하는 문법으로 HTML의 형태로 변환이 가능하다.

- '1라인 이동' + '명령모드' + 'A' 키를 입력하여 한 행을 추가한 후 'M' 키를 입력하여 Mark Down을 변경한다.

### # Jupyter Notebook

주피터 노트북은 웹 브라우저 상에서 개발할 수 있는 도구이며, 코드를 Cell 단위로 묶어서 실행하고 그래프나 표, 그리고 이미지나 영상 등을 쉽게 볼 수 있어서 특히 데이터 관련 작업을 할 때 많이 활용됩니다.

> 교육을 위한 강의 노트로도 아주 훌륭해요!

### Jupyter Notebook

주피터 노트북은 웹 브라우저 상에서 개발할 수 있는 도구이며, 코드를 Cell 단위로 묶어서 실행하고 그래프나 표, 그리고 이미지나 영상 등을 쉽게 볼 수 있어서 특히 데이터 관련 작업을 할 때 많이 활용됩니다.

교육을 위한 강의 노트로도 아주 훌륭해요!

- 마크다운으로 이미지 입력하기 1 : [구글]-[Jupyter Notebook] 홈페이지를 화면 캡처하여 붙여넣기 한다.

```
![image.png](attachment:image.png)
```

- 마크다운으로 이미지 입력하기2 : [Edit]-[Insert Image] 메뉴에서 입력할 이미지를 선택한다.

```
![a.png](attachment:a.png)
```

- '#'의 개수에 따라 글자 크기가 결정되고 목록 작성을 작성하려면 숫자 '1' 키를 입력한 후 문장을 작성한다.

## ## 특징

1. 코드를 Cell 단위로 작성 및 실행
1. 마크다운을 통한 문서화
1. 그래프나 표 등을 실시간으로 확인
1. html, pdf 등 파일로 저장

### 특징

1. 코드를 Cell 단위로 작성 및 실행
2. 마크다운을 통한 문서화
3. 그래프나 표 등을 실시간으로 확인
4. html, pdf 등 파일로 저장

- '---' 키를 누르면 수평선이 출력되며 '-' 키를 입력하면 목록을 작성 한다.

---

## ## 시각화 예제 1: 그래프

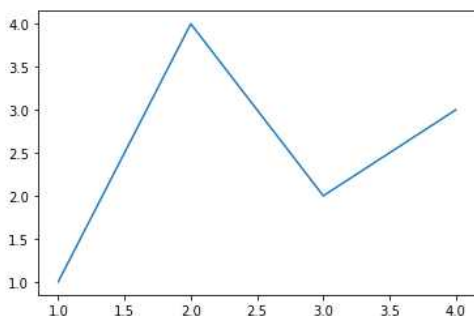
- matplotlib를 이용하면 다양한 그래프를 그릴 수 있으며 수행 결과를 바로 확인 할 수 있다.

### 시각화 예제 1: 그래프

- matplotlib를 이용하면 다양한 그래프를 그릴 수 있으며 수행 결과를 바로 확인 할 수 있다.

- [구글]-[matplotlib 검색]-[Tutorials]-[Basic Usage] 기본 그래프 코드를 복사하여 그래프를 출력해 본다.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
fig, ax = plt.subplots() # Create a figure containing a single axes.
ax.plot([1, 2, 3, 4], [1, 4, 2, 3]); # Plot some data on the axes.
```



- [구글]-[pandas 검색]-[User Guide]-[Table Visualization] 테이블 작성 코드를 복사하여 테이블을 출력해 본다.

```
import pandas as pd
import numpy as np
import matplotlib as mpl

df = pd.DataFrame([[38.0, 2.0, 18.0, 22.0, 21, np.nan],[19, 439, 6, 452, 226,232]],
                  index=pd.Index(['Tumour (Positive)', 'Non-Tumour (Negative)'], name='Actual Label:'),
                  columns=pd.MultiIndex.from_product(
                      [['Decision Tree', 'Regression', 'Random'], ['Tumour', 'Non-Tumour']], names=['Model:', 'Predicted:']))
df.style
```

| Actual Label:         | Model:     |  | Decision Tree |            | Regression |            | Random |            |
|-----------------------|------------|--|---------------|------------|------------|------------|--------|------------|
|                       | Predicted: |  | Tumour        | Non-Tumour | Tumour     | Non-Tumour | Tumour | Non-Tumour |
|                       |            |  |               |            |            |            |        |            |
| Tumour (Positive)     |            |  | 38.000000     | 2.000000   | 18.000000  | 22.000000  | 21     | nan        |
| Non-Tumour (Negative) |            |  | 19.000000     | 439.000000 | 6.000000   | 452.000000 | 226    | 232.000000 |

- 하이퍼 링크 텍스트를 이용하려면 '[링크단어]' + '(링크주소)'를 입력한다.

## 사전 학습 자료  
 다음 링크에서 파이썬 기본편에 대한 학습을 하실 수 있습니다. space 두 칸 입력  
 - [나도 코딩 블로그](https://nadocoding.tistory.com)

### 사전 학습 자료

다음 링크에서 파이썬 기본편에 대한 학습을 하실 수 있습니다.  
[나도 코딩 블로그](https://nadocoding.tistory.com)

- [유튜브]-[나도코딩 python 기본편 검색]-[오른쪽 마우스]-[소스코드복사] 메뉴를 선택하여 복사하여 'Code' 상태로 변경(Markdown인 경우 명령모드에서 후 'Y')한 후 상단에 '%%HTML' 입력 후 실행해 본다.

```
%%HTML
<iframe width="640" height="360"
src="https://www.youtube.com/embed/kWiCuklohdY"
title="YouTube video player" frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen>
</iframe>
```

- Line Number 재 정렬 : [Kernel]-[Restart & Run All]-[Restart and Run All Cells] 버튼 선택
- HTML 저장 : [File]-[Download as] - [HTML(\*.htm)] 다운로드 폴더에 저장된다.
- Jupyter 파일 저장 : [c:]-[사용자]-[사용자 계정]에 확장자 IPYNB(Interactive PYthon NoteBook)로 저장된다.
- Jupyter 파일 열기 : [File]-[Open] 메뉴에서 파일을 선택한다.
- Jupyter 종료 : [Ctrl + S]로 저장-[File]-[Close and Halt]-[명령창]-[Ctrl + C, C]

## • 판다스(pandas)

판다스(pandas)란 데이터 조작과 분석을 위한 파이썬(Python) 소프트웨어 라이브러리이다. [Extensions]-[jupyter]

### 01. 시리즈(Series)

Series는 Indexing된 데이터의 1차원 배열이다. Numpy는 명시적 인덱스 없이 묵시적으로 '0'부터 차례로 정수형 인덱스를 사용하여 접근하고 Series는 명시적으로 정의된 인덱스가 존재하기 때문에 이 명시적 인덱스를 사용할 수 있다.

- 새로운 파일 '01.시리즈.ipynb'를 생성한다.
- 1월부터 4월까지 평균 온도 데이터(-20, -10, 10, 20)를 입력 후 출력한다. print문은 생략할 수 있다.

```
import pandas as pd
temp = pd.Series([-20, -10, 10, 20])
print(temp)
```

```
0    -20
1    -10
2     10
3     20
dtype: int64
```

- index 값 '0'에 저장된 데이터(1월 온도)를 출력한다.

```
print(temp[0])
```

```
-20
```

- Series 객체를 생성하면서 특정 index를 지정할 수 있다.

```
temp = pd.Series([-20, -10, 10, 20], index=['Jan', 'Feb', 'Mar', 'Apr'])
temp
```

```
Jan    -20
Feb    -10
Mar     10
Apr     20
dtype: int64
```

- index Apr(4월)에 해당하는 데이터 값을 출력한다.

```
print(temp['Apr'])
```

```
20
```

- 존재하지 않는 index에 접근을 시도하면 에러가 발생한다.

```
temp['Jun']
```

```
-----
KeyError                                Traceback (most recent call last)
...
KeyError: 'Jun'
The above exception was the direct cause of the following exception:
...
```

## 02. 데이터프레임(DataFrame)

데이터프레임(DataFrame)은 2차원 배열 데이터 즉 시리즈(Series)들의 모임을 의미한다.

- 새로운 파일 '02.데이터프레임.ipynb'를 생성한다.
- 슬램덩크 주요 인물 8명에 대한 데이터 (dictionary 타입으로 저장한다.)

```
data = {  
    '이름' : ['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'],  
    '학교' : ['북산고', '북산고', '북산고', '북산고', '북산고', '능남고', '능남고', '능남고'],  
    '키' : [197, 184, 168, 187, 188, 202, 188, 190],  
    '국어' : [90, 40, 80, 40, 15, 80, 55, 100],  
    '영어' : [85, 35, 75, 60, 20, 100, 65, 85],  
    '수학' : [100, 50, 70, 70, 10, 95, 45, 90],  
    '과학' : [95, 55, 80, 75, 35, 85, 40, 95],  
    '사회' : [85, 25, 75, 80, 10, 80, 35, 95],  
    'SW특기' : ['Python', 'Java', 'Javascript', '', '', 'C', 'PYTHON', 'C#']  
}
```

data, type(data)

```
({'이름': ['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'],  
 '학교': ['북산고', '북산고', '북산고', '북산고', '북산고', '능남고', '능남고', '능남고'],  
 '키': [197, 184, 168, 187, 188, 202, 188, 190],  
 ...  
 '사회': [85, 25, 75, 80, 10, 80, 35, 95],  
 'SW특기': ['Python', 'Java', 'Javascript', '', '', 'C', 'PYTHON', 'C#']}, dict)
```

data['키'], type(data)

```
([197, 184, 168, 187, 188, 202, 188, 190], dict)
```

data['이름'], type(data['이름'])

```
(['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'], list)
```

- pandas 라이브러리를 import 한 후 dictionary(data) 객체를 데이터프레임(df)로 생성한다.

```
import pandas as pd  
df = pd.DataFrame(data)  
df
```

|   | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|-----|-----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 3 | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |            |
| 4 | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |            |
| 5 | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 6 | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 7 | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 '이름' 칼럼 데이터는 시리즈(Series) 타입으로 출력된다.

```
df['이름']

0    채치수
1    정대만
2    송태섭
3    서태웅
4    강백호
5    변덕규
6    황태산
7    윤대협
Name: 이름, dtype: object
```

```
type(df['이름'])

pandas.core.series.Series
```

```
df['이름'].values

array(['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'],
      dtype=object)
```

- 데이터프레임(df)에서 '학교' 칼럼 데이터는 시리즈(Series) 타입으로 출력된다.

```
df['학교']

0    부산고
1    부산고
2    부산고
3    부산고
4    부산고
5    능남고
6    능남고
7    능남고
Name: 학교, dtype: object
```

- 데이터프레임(df)에서 '학교', '이름', '키'열을 2차원 배열(대괄호 두 번 사용)로 출력한다.

```
df[['학교', '이름', '키']]
```

|   | 학교  | 이름  | 키   |
|---|-----|-----|-----|
| 0 | 부산고 | 채치수 | 197 |
| 1 | 부산고 | 정대만 | 184 |
| 2 | 부산고 | 송태섭 | 168 |
| 3 | 부산고 | 서태웅 | 187 |
| 4 | 부산고 | 강백호 | 188 |
| 5 | 능남고 | 변덕규 | 202 |
| 6 | 능남고 | 황태산 | 188 |
| 7 | 능남고 | 윤대협 | 190 |



- **shape** 속성을 이용해 몇 차원 배열인지 확인할 수 있다.

```
df[['학교', '이름', '키']].shape
```

```
(8, 3)
```

- 데이터프레임(df)에서 **index**를 '1번', '2번', '3번' ... 으로 변경하여 저장한다. 행수와 같게 **index**를 지정한다.

```
df = pd.DataFrame(data, index=['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

|    | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|----|-----|-----|-----|-----|-----|-----|----|----|------------|
| 1번 | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번 | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번 | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번 | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |            |
| 5번 | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |            |
| 6번 | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번 | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번 | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- **data** 중에서 원하는 '이름','학교','키' 열만 순서대로 읽어 데이터프레임(df)에 저장한다.

```
df = pd.DataFrame(data, columns=['이름', '학교', '키'])
```

|   | 이름  | 학교  | 키   |
|---|-----|-----|-----|
| 0 | 채치수 | 북산고 | 197 |
| 1 | 정대만 | 북산고 | 184 |
| 2 | 송태섭 | 북산고 | 168 |
| 3 | 서태웅 | 북산고 | 187 |
| 4 | 강백호 | 북산고 | 188 |
| 5 | 변덕규 | 능남고 | 202 |
| 6 | 황태산 | 능남고 | 188 |
| 7 | 윤대협 | 능남고 | 190 |

- **data** 중에서 원하는 '이름','학교','키', 'SW특기' 열만 순서대로 읽어 데이터프레임(df)에 저장한다.

```
df = pd.DataFrame(data, columns=['이름', '키', '학교', 'SW특기'])
```

|   | 이름  | 키   | 학교  | SW특기       |
|---|-----|-----|-----|------------|
| 0 | 채치수 | 197 | 북산고 | Python     |
| 1 | 정대만 | 184 | 북산고 | Java       |
| 2 | 송태섭 | 168 | 북산고 | Javascript |
| 3 | 서태웅 | 187 | 북산고 |            |
| 4 | 강백호 | 188 | 북산고 |            |
| 5 | 변덕규 | 202 | 능남고 | C          |
| 6 | 황태산 | 188 | 능남고 | PYTHON     |
| 7 | 윤대협 | 190 | 능남고 | C#         |

### 03. 인덱스(index)

- 새로운 파일 '03.인덱스.ipynb'를 생성한다.
- 변수(list) 세 개의 값을 저장한 후 데이터와 데이터의 type을 출력한다.

```
list = ['유재석', '하하', '정형돈']  
list, type(list)
```

```
('유재석', '하하', '정형돈', list)
```

```
list[1]
```

```
'하하'
```

```
list[0:2]
```

```
['유재석', '하하']
```

```
list[-1]
```

```
'정형돈'
```

- Pandas 라이브러리 이용해 data에 딕셔너리(dictionary)를 저장한 후 데이터프레임(df)를 생성한다.

```
import pandas as pd  
data = {  
    '이름' : ['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'],  
    '학교' : ['북산고', '북산고', '북산고', '북산고', '북산고', '능남고', '능남고', '능남고'],  
    '키' : [197, 184, 168, 187, 188, 202, 188, 190],  
    '국어' : [90, 40, 80, 40, 15, 80, 55, 100],  
    '영어' : [85, 35, 75, 60, 20, 100, 65, 85],  
    '수학' : [100, 50, 70, 70, 10, 95, 45, 90],  
    '과학' : [95, 55, 80, 75, 35, 85, 40, 95],  
    '사회' : [85, 25, 75, 80, 10, 80, 35, 95],  
    'SW특기' : ['Python', 'Java', 'Javascript', '', '', 'C', 'PYTHON', 'C#']  
}  
df = pd.DataFrame(data, index = ['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
```

|    | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|----|-----|-----|-----|-----|-----|-----|----|----|------------|
| 1번 | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번 | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번 | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번 | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |            |
| 5번 | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |            |
| 6번 | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번 | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번 | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)의 index 이름들을 출력해 본다.

```
df.index
```

```
Index(['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'], dtype='object')
```

- 데이터프레임(df)의 index 칼럼 이름을 '지원번호'로 새로 설정한다.

```
df.index.name = '지원번호'
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 |            |

- 데이터프레임(df)의 index 칼럼이 원래 값으로 초기화되고 '지원번호'는 새로운 칼럼으로 바뀐다.

```
df.reset_index()
```

|   | 지원번호 | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|------|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 2 | 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 3 | 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 |            |

- 데이터프레임(df)에 추가되었던 '지원번호' 칼럼이 삭제되어 보이지만 데이터에는 반영되지 않는다.

```
df.reset_index(drop=True)
```

|   | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 3 | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 |            |

- 데이터프레임(df)을 출력하면 '지원번호' index가 다시 출력된다.

```
df
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 |            |

- 데이터프레임(df)에 바로 적용하려면 inplace 옵션을 사용한다.

```
df.reset_index(drop=True, inplace=True)
```

|   | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |

- 데이터프레임(df)의 '이름' 컬럼으로 index를 설정한다.

```
df.set_index('이름')
```

|  | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|--|-----|-----|-----|----|----|-----|----|----|------------|
|  | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
|  | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
|  | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |

- 데이터프레임(df)에 변경한 index를 바로 적용하려면 inplace 옵션을 사용한다.

```
df.set_index('이름', inplace=True)
```

|  | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|--|-----|-----|-----|----|----|-----|----|----|------------|
|  | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
|  | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
|  | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |

- 데이터프레임(df)에서 '이름' index를 기준으로 오름차순 정렬을 한다.

```
df.sort_index()
```

|  | 이름  | 학교  | 키   | 국어 | 영어  | 수학 | 과학 | 사회 | SW특기 |
|--|-----|-----|-----|----|-----|----|----|----|------|
|  | 강백호 | 북산고 | 188 | 15 | 20  | 10 | 35 | 10 |      |
|  | 변덕규 | 능남고 | 202 | 80 | 100 | 95 | 85 | 80 | C    |
|  | 서태웅 | 북산고 | 187 | 40 | 60  | 70 | 75 | 80 |      |

- 데이터프레임(df)에서 index('이름')를 기준으로 내림차순 정렬을 한다.

```
df.sort_index(ascending=False)
```

|   | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |

## 04. 파일 저장 및 열기

DataFrame 객체를 excel, csv, txt 등 형태의 파일로 저장 및 열기를 실행해 본다.

- 새로운 파일 '04.파일저장과열기.ipynb'를 생성한다.

```
import pandas as pd

data = {
    '이름' : ['채치수', '정대만', '송태섭', '서태웅', '강백호', '변덕규', '황태산', '윤대협'],
    '학교' : ['북산고', '북산고', '북산고', '북산고', '북산고', '능남고', '능남고', '능남고'],
    '키' : [197, 184, 168, 187, 188, 202, 188, 190],
    '국어' : [90, 40, 80, 40, 15, 80, 55, 100],
    '영어' : [85, 35, 75, 60, 20, 100, 65, 85],
    '수학' : [100, 50, 70, 70, 10, 95, 45, 90],
    '과학' : [95, 55, 80, 75, 35, 85, 40, 95],
    '사회' : [85, 25, 75, 80, 10, 80, 35, 95],
    'SW특기' : ['Python', 'Java', 'Javascript', '', '', 'C', 'PYTHON', 'C#']
}

df = pd.DataFrame(data, index = ['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'])
df.index.name = '지원번호'
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |            |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |            |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)를 csv 파일로 저장한다. (엑셀 파일에서 한글이 깨지는 경우 encoding 지정)

```
df.to_csv('score.csv', encoding='utf-8-sig')
```

- 데이터프레임(df)를 지원번호를 생략하고 csv 파일로 저장한다.

```
df.to_csv('score.csv', encoding='utf-8-sig', index=False)
```

- 데이터프레임(df)을 텍스트(.txt) 파일로 저장한다. (칼럼을 'Tab'으로 구분한다.)

```
df.to_csv('score.txt', sep='\t')
```

- 데이터프레임(df)을 엑셀(.xlsx) 파일로 저장한다. (openpyxl 파일 미설치 pip install openpyxl)

```
df.to_excel('score.xlsx')
```

- 'score.csv' 파일을 읽어 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv')
```

|   | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|-----|-----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 3 | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 4 | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 5 | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 6 | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 7 | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 'score.csv' 파일을 읽어 상위 1개의 ROW를 제외(Skip)하고 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv', skiprows=1)
```

|   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85.1 | Python     |
|---|-----|-----|-----|-----|-----|-----|----|------|------------|
| 0 | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25   | Java       |
| 1 | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75   | Javascript |
| 2 | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80   | NaN        |
| 3 | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10   | NaN        |
| 4 | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80   | C          |
| 5 | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35   | PYTHON     |
| 6 | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95   | C#         |

- 'score.csv' 파일의 1, 3, 5행을 제외(skip)하고 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv', skiprows=[1, 3, 5])
```

|   | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기   |
|---|-----|-----|-----|-----|-----|----|----|----|--------|
| 0 | 정대만 | 북산고 | 184 | 40  | 35  | 50 | 55 | 25 | Java   |
| 1 | 서태웅 | 북산고 | 187 | 40  | 60  | 70 | 75 | 80 | NaN    |
| 2 | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C      |
| 3 | 황태산 | 능남고 | 188 | 55  | 65  | 45 | 40 | 35 | PYTHON |
| 4 | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#     |

- 'score.csv' 파일의 2, 4, 6, 8행을 제외(skip)하고 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv', skiprows=[2, 4, 6, 8])
```

|   | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 2 | 강백호 | 북산고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |
| 3 | 황태산 | 능남고 | 188 | 55 | 65 | 45  | 40 | 35 | PYTHON     |

- 'score.csv' 파일의 지정된 수만큼의 행만 가져와서 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv', nrows=4)
```

|   | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|---|-----|-----|-----|----|----|-----|----|----|------------|
| 0 | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 1 | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 2 | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |

- 'score.csv' 파일의 처음부터 2행까지 무시하고 이후에 4행을 가져와서 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.csv', skiprows=2, nrows=4)
```

|   | 정대만 | 북산고 | 184 | 40 | 35  | 50 | 55 | 25 | Java       |
|---|-----|-----|-----|----|-----|----|----|----|------------|
| 0 | 송태섭 | 북산고 | 168 | 80 | 75  | 70 | 80 | 75 | Javascript |
| 1 | 서태웅 | 북산고 | 187 | 40 | 60  | 70 | 75 | 80 | NaN        |
| 2 | 강백호 | 북산고 | 188 | 15 | 20  | 10 | 35 | 10 | NaN        |
| 3 | 변덕규 | 능남고 | 202 | 80 | 100 | 95 | 85 | 80 | C          |

- 'score.txt' 텍스트 파일을 'TAB'을 구분자로 읽어 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.txt', sep="\t")
```

|   | 지원번호 | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|---|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 0 | 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 1 | 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 2 | 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 3 | 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 4 | 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 5 | 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 6 | 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 7 | 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 'score.txt' 파일을 '지원번호'열을 index열로 지정하여 데이터프레임(df)에 저장한다.

```
df = pd.read_csv('score.txt', sep="\t", index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 'score.txt' 파일을 읽어 데이터프레임(df)에 저장한 후 지원번호를 인덱스로 지정하면 앞의 결과와 같다.

```
df = pd.read_csv('score.txt', sep="\t")
df.set_index('지원번호', inplace=True)
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 'score.xlsx' 엑셀 파일을 읽어 데이터프레임(df)에 저장한다.

```
df = pd.read_excel('score.xlsx')
df
```

|   | 지원번호 | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|---|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 0 | 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 1 | 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 2 | 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 3 | 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 4 | 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 5 | 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 6 | 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 7 | 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 'score.xlsx' 엑셀 파일을 읽어 index를 지원번호로 지정한 후 데이터를 읽어온다.

```
df = pd.read_excel('score.xlsx', index_col='지원번호')
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |



## 05. 데이터 확인

- 새로운 파일 '05.데이터확인.ipynb'를 생성한 후 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 각종 계산이 가능한 열별로 데이터의 수, 평균, 표준편차, 최솟값, 최댓값 등의 정보를 보여준다.

```
df.describe()
```

|       | 키          | 국어         | 영어         | 수학         | 과학        | 사회        |
|-------|------------|------------|------------|------------|-----------|-----------|
| count | 8.000000   | 8.000000   | 8.000000   | 8.000000   | 8.000000  | 8.000000  |
| mean  | 188.000000 | 62.500000  | 65.625000  | 66.250000  | 70.000000 | 60.625000 |
| std   | 9.985704   | 29.519969  | 26.917533  | 30.325614  | 23.754699 | 32.120032 |
| min   | 168.000000 | 15.000000  | 20.000000  | 10.000000  | 35.000000 | 10.000000 |
| 25%   | 186.250000 | 40.000000  | 53.750000  | 48.750000  | 51.250000 | 32.500000 |
| 50%   | 188.000000 | 67.500000  | 70.000000  | 70.000000  | 77.500000 | 77.500000 |
| 75%   | 191.750000 | 82.500000  | 85.000000  | 91.250000  | 87.500000 | 81.250000 |
| max   | 202.000000 | 100.000000 | 100.000000 | 100.000000 | 95.000000 | 95.000000 |

- 데이터프레임(df)에서 각 열에 대한 데이터 타입을 확인한다.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8 entries, 1번 to 8번
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   이름        8 non-null     object
1   학교        8 non-null     object
2   키          8 non-null     int64
3   국어        8 non-null     int64
4   영어        8 non-null     int64
5   수학        8 non-null     int64
6   과학        8 non-null     int64
7   사회        8 non-null     int64
8   SW특기      6 non-null     object
dtypes: int64(6), object(3)
memory usage: 640.0+ bytes
```

- 데이터프레임(df)에서 처음 5행을 출력한다.

`df.head()`

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |

- 데이터프레임(df)에서 처음 4개의 행을 출력한다.

`df.head(4)`

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |

- 데이터프레임(df)에서 마지막 5개의 행을 출력한다.

`df.tail()`

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|-----|-----|----|----|----|--------|
| 지원번호 |     |     |     |     |     |    |    |    |        |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70 | 75 | 80 | NaN    |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10 | 35 | 10 | NaN    |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C      |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45 | 40 | 35 | PYTHON |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#     |

- 데이터프레임(df)에서 마지막 6개의 행을 출력한다.

`df.tail(6)`

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|----|----|----|------------|
| 지원번호 |     |     |     |     |     |    |    |    |            |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70 | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70 | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10 | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45 | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#         |

- 데이터프레임(df)의 값들을 2차원 배열로 출력한다.

```
df.values

array([[ '채지수', '북산고', 197, 90, 85, 100, 95, 85, 'Python'],
       [ '정대만', '북산고', 184, 40, 35, 50, 55, 25, 'Java'],
       [ '송태섭', '북산고', 168, 80, 75, 70, 80, 75, 'Javascript'],
       [ '서태웅', '북산고', 187, 40, 60, 70, 75, 80, nan],
       [ '강백호', '북산고', 188, 15, 20, 10, 35, 10, nan],
       [ '변덕규', '능남고', 202, 80, 100, 95, 85, 80, 'C'],
       [ '황태산', '능남고', 188, 55, 65, 45, 40, 35, 'PYTHON'],
       [ '윤대협', '능남고', 190, 100, 85, 90, 95, 95, 'C#']
      ], dtype=object)
```

- 데이터프레임(df)의 index 정보가 출력한다.

```
df.index

Index(['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'], dtype='object', name='직원번호')
```

- 데이터프레임(df)의 열 이름을 목록으로 출력된다.

```
df.columns

Index(['이름', '학교', '키', '국어', '영어', '수학', '과학', '사회', 'SW특기'], dtype='object')
```

- 데이터프레임(df)이 몇 개의 행과 열인지 출력한다.

```
df.shape

(8, 9)
```

- Series(일차원 배열) 키에 대한 모든 정보를 출력한다.

```
df['키'].describe()

count      8.000000
mean      188.000000
std         9.985704
min      168.000000
25%      186.250000
50%      188.000000
75%      191.750000
max      202.000000

Name: 키, dtype: float64
```

- 데이터프레임(df)에서 '키'열 데이터 중에서 최솟값을 출력한다.

```
df['키'].min()
```

```
168
```

- 데이터프레임(df)에서 '키'열 데이터 중에서 최댓값을 출력한다.

```
df['키'].max()
```

```
202
```

- 데이터프레임(df)에서 '키'가 큰사람 순서대로 3명의 데이터를 출력한다.

```
df['키'].nlargest(3)
```

```
지원번호
```

```
6번      202
```

```
1번      197
```

```
8번      190
```

```
Name: 키, dtype: int64
```

- 데이터프레임(df)에서 '키'열의 평균값을 출력한다.

```
df['키'].mean()
```

```
188.0
```

- 데이터프레임(df)에서 모든 '키'열에 대한 합계를 출력한다.

```
df['키'].sum()
```

```
1504
```

- 데이터프레임(df)에서 'SW특기'가 있는 사람들이 몇 명인지 출력한다.

```
df['SW특기'].count()
```

```
6
```

- 데이터프레임(df)에서 '학교'들을 중복 제거하고 출력한다.

```
df['학교'].unique()
```

```
array(['복산고', '능남고'], dtype=object)
```

- 데이터프레임(df)에서 데이터들이 전체 몇 개의 학교에 속해있는지를 출력한다.

```
df['학교'].nunique()
```

```
2
```

## 06. 데이터 선택(기본)

```
df[열], df[[열1, 열2]], df[열][:행], df[열][행:], df[열][행1:행2], df[[열1, 열2]][:행], df[[열1, 열2]][행:]  
df[[열1, 열2]][행1:행2], df[행:], df[:행], df[행1:행2]
```

- 새로운 파일 '06.데이터선택(기본).ipynb'를 생성한 후 'score.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd  
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 모든 인덱스에 대하여 '이름'열 데이터를 출력한다.

```
df['이름']
```

```
지원번호  
1번      채치수  
2번      정대만  
3번      송태섭  
4번      서태웅  
5번      강백호  
6번      변덕규  
7번      황태산  
8번      윤대협  
Name: 이름, dtype: object
```

- 데이터프레임(df)에서 모든 인덱스에 대한 '키'열 데이터를 출력한다.

```
df['키']
```

```
지원번호  
1번      197  
2번      184  
3번      168  
4번      187  
5번      188  
6번      202  
7번      188  
8번      190  
Name: 키, dtype: int64
```

- 데이터프레임(df)에서 모든 인덱스에 대한 '이름', '키'열을 2차원 배열로 출력한다.

```
df[['이름','키']]
```

|      | 이름  | 키   |
|------|-----|-----|
| 지원번호 |     |     |
| 1번   | 채치수 | 197 |
| 2번   | 정대만 | 184 |
| 3번   | 송태섭 | 168 |
| 4번   | 서태웅 | 187 |
| 5번   | 강백호 | 188 |
| 6번   | 변덕규 | 202 |
| 7번   | 황태산 | 188 |
| 8번   | 윤대협 | 190 |

- 데이터프레임(df)에서 열 이름들을 출력한다.

```
df.columns
```

```
Index(['이름', '학교', '키', '국어', '영어', '수학', '과학', '사회', 'SW특기'], dtype='object')
```

- 데이터프레임(df)에서 인덱스가 0열의 이름을 출력한다. (인덱스는 칼럼 이름을 모르는 경우 유용하다.)

```
df.columns[0]
```

```
'이름'
```

- 데이터프레임(df)에서 마지막 열 이름을 출력한다.

```
df.columns[-1]
```

```
'SW특기'
```

- 데이터프레임(df)에서 0열의 모든 데이터를 출력한다. df['이름']와 결과가 같다.

```
df[df.columns[0]]
```

```
지원번호
1번      채치수
2번      정대만
3번      송태섭
4번      서태웅
5번      강백호
6번      변덕규
7번      황태산
8번      윤대협
Name: 이름, dtype: object
```

- 데이터프레임(df)에서 열이 많은 경우 마지막의 열의 데이터 출력 시 유용하다.

```
df[df.columns[-1]]
```

지원번호

|    |            |
|----|------------|
| 1번 | Python     |
| 2번 | Java       |
| 3번 | Javascript |
| 4번 | NaN        |
| 5번 | NaN        |
| 6번 | C          |
| 7번 | PYTHON     |
| 8번 | C#         |

Name: SW특기, dtype: object

- 데이터프레임(df)에서 '영어'열 '1번'~'3번'행 데이터를 출력한다. df['영어'][0:3]

```
df['영어']['1번':'3번']
```

지원번호

|    |    |
|----|----|
| 1번 | 85 |
| 2번 | 35 |
| 3번 | 75 |

Name: 영어, dtype: int64

- 데이터프레임(df)에서 '이름', '키'열을 인덱스 처음~'5번' 학생 데이터를 출력한다. df[['이름', '키'][:5]]

```
df[['이름', '키'][:5]]
```

|      | 이름  | 키   |
|------|-----|-----|
| 지원번호 |     |     |
| 1번   | 채치수 | 197 |
| 2번   | 정대만 | 184 |
| 3번   | 송태섭 | 168 |
| 4번   | 서태웅 | 187 |
| 5번   | 강백호 | 188 |

- 데이터프레임(df)에서 모든 열의 인덱스 '2번'~마지막 데이터를 출력한다. df[1:]

```
df['2번':]
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|----|----|----|------------|
| 지원번호 |     |     |     |     |     |    |    |    |            |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50 | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70 | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70 | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10 | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45 | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#         |

## 07. 데이터 선택(loc)

```
df.loc[행], df.loc[행, 열], df.loc[[행1, 행2], 열], df.loc[[행1, 행2], [열1, 열2]], df.loc[행1:행2]
```

```
df.loc[행1:행2, [열1, 열2]], df.loc[[행1, 행2], 열1:열2], df.loc[행1:행2, 열1:열2]
```

- 새로운 파일 '**07.데이터선택(loc).ipynb**'를 생성하고 데이터프레임(df)에 index를 '지원번호'로 지정해 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 인덱스가 '1번'에 해당하는 모든 열 데이터를 출력한다.

```
df.loc['1번']
```

```
이름      채치수
학교      북산고
키        197
국어      90
영어      85
수학      100
과학      95
사회      85
SW특기    Python
Name: 1번, dtype: object
```

- 데이터프레임(df)에서 인덱스 '5번'에 해당하는 모든 열 데이터를 출력한다.

```
df.loc['5번']
```

```
이름      강백호
학교      북산고
키        188
국어      15
영어      20
수학      10
과학      35
사회      10
SW특기    NaN
Name: 5번, dtype: object
```



- 데이터프레임(df)에서 인덱스가 '1번'에 해당하는 '국어'열 데이터를 출력한다.

```
df.loc['1번', '국어']
```

```
90
```

- 데이터프레임(df)에서 인덱스가 '1번', '2번'에 해당하는 '영어'열 데이터를 출력한다.

```
df.loc[['1번', '2번'], '영어']
```

지원번호

1번 85

2번 35

Name: 영어, dtype: int64

```
type(df.loc[['1번', '2번'], '영어'])
```

```
pandas.core.series.Series
```

- 데이터프레임(df)에서 인덱스가 '1번', '3번', '5번'에 해당하는 '국어', '영어', '수학'열 데이터를 출력한다.

```
df3=df.loc[['1번','3번', '5번'], ['국어', '영어', '수학']]
```

|      | 국어 | 영어 | 수학  |
|------|----|----|-----|
| 지원번호 |    |    |     |
| 1번   | 90 | 85 | 100 |
| 3번   | 80 | 75 | 70  |
| 5번   | 15 | 20 | 10  |

```
type(df.loc[['1번','3번', '5번'], ['국어', '영어', '수학']])
```

```
pandas.core.frame.DataFrame
```

- 데이터프레임(df)에서 '1번'~'5번'인덱스, '국어'~'사회'열까지 데이터를 출력한다.

```
df.loc['1번':'5번', '국어':'사회']
```

|      | 국어 | 영어 | 수학  | 과학 | 사회 |
|------|----|----|-----|----|----|
| 지원번호 |    |    |     |    |    |
| 1번   | 90 | 85 | 100 | 95 | 85 |
| 2번   | 40 | 35 | 50  | 55 | 25 |
| 3번   | 80 | 75 | 70  | 80 | 75 |
| 4번   | 40 | 60 | 70  | 75 | 80 |
| 5번   | 15 | 20 | 10  | 35 | 10 |

## 08. 데이터 선택(iloc)

`df.iloc[행]`, `df.iloc[[행1, 행2]]`, `df.iloc[행1:행2]`, `df.iloc[행:]`, `df.iloc[:행]`, `df.iloc[행, 열]`, `df.iloc[행:, 열]`, `df.iloc[:행, 열]`  
`df.iloc[[행1, 행2], 열]`, `df.iloc[행1:행2, 열]`, `df.iloc[[행1, 행2], 열1:열2]`, `df.iloc[[행1, 행2], [열1, 열2]]`, `df.iloc[행, :열]`

- 새로운 파일 '08.데이터선택(iloc).ipynb'를 생성하고 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 index 0행의 모든 열 데이터를 출력한다.

```
df.iloc[0]
```

```
이름      채치수
학교      북산고
키        197
국어       90
영어       85
수학      100
과학       95
사회       85
SW특기     Python
Name: 1번, dtype: object
```

- 데이터프레임(df)에서 index 0~5행의 모든 열 데이터를 출력한다.

```
df.iloc[0:6]
```

|      | 이름  | 학교  | 키   | 국어 | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |    |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80 | 100 | 95  | 85 | 80 | C          |

- 데이터프레임(df)에서 index 0행(1번) 학생의 label 1열(학교) 데이터를 출력한다.

```
df.iloc[0, 1]
```

```
'북산고'
```

- 데이터프레임(df)에서 index 4행(5번) 학생의 label 2열(키) 데이터를 출력한다.

```
df.iloc[4, 2]
```

```
188
```

- 데이터프레임(df)에서 index 0, 1행 학생의 label 2열(키) 데이터를 출력한다.

```
df.iloc[[0, 1], 2]
```

```
지원번호
```

```
1번      197
```

```
2번      184
```

```
Name: 키, dtype: int64
```

```
type(df.iloc[[0,1], 2])
```

```
pandas.core.series.Series
```

- 데이터프레임(df)에서 index 0, 2, 4행 학생의 label 1, 3, 5열 데이터를 출력한다.

```
df.iloc[[0, 2, 4], [1, 3, 5]]
```

|      | 학교  | 국어 | 수학  |
|------|-----|----|-----|
| 지원번호 |     |    |     |
| 1번   | 북산고 | 90 | 100 |
| 3번   | 북산고 | 80 | 70  |
| 5번   | 북산고 | 15 | 10  |

- 데이터프레임(df)에서 index 0~6행 학생의 label 3~8열 데이터를 출력한다.

```
df.iloc[0:6, 1:8]
```

|      | 학교  | 키   | 국어 | 영어  | 수학  | 과학 | 사회 |
|------|-----|-----|----|-----|-----|----|----|
| 지원번호 |     |     |    |     |     |    |    |
| 1번   | 북산고 | 197 | 90 | 85  | 100 | 95 | 85 |
| 2번   | 북산고 | 184 | 40 | 35  | 50  | 55 | 25 |
| 3번   | 북산고 | 168 | 80 | 75  | 70  | 80 | 75 |
| 4번   | 북산고 | 187 | 40 | 60  | 70  | 75 | 80 |
| 5번   | 북산고 | 188 | 15 | 20  | 10  | 35 | 10 |
| 6번   | 능남고 | 202 | 80 | 100 | 95  | 85 | 80 |

## 09. 데이터 선택(조건)

- 새로운 파일 '09.데이터선택(조건).ipynb'를 생성하고 'score.xlsx'파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 학생들의 '키' 칼럼이 185 이상 여부를 True/False로 출력한다.

```
df['키'] >= 185
```

지원번호

|    |       |
|----|-------|
| 1번 | True  |
| 2번 | False |
| 3번 | False |
| 4번 | True  |
| 5번 | True  |
| 6번 | True  |
| 7번 | True  |
| 8번 | True  |

Name: 키, dtype: bool

- 데이터프레임(df)에서 학생들의 '키' 칼럼이 185 이상인 학생들의 정보를 2차원 배열로 출력한다.

```
filt = (df['키'] >= 185)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|-----|-----|-----|----|----|--------|
| 지원번호 |     |     |     |     |     |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN    |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN    |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C      |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#     |

- filter를 역으로 적용하여 '키'가 185 미만의 데이터를 출력한다.

```
df[~filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|----|----|----|------------|
| 지원번호 |     |     |     |    |    |    |    |    |            |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50 | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript |

- 조건식을 바로 배열에 입력하여 적용할 수 있다.

```
df[df['키'] >= 185]
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|-----|-----|-----|----|----|--------|
| 지원번호 |     |     |     |     |     |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN    |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN    |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C      |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#     |

- 데이터프레임(df)에서 '키'가 185이상 학생들의 수학 점수를 출력한다.

```
df.loc[df['키'] >= 185, '수학']
```

지원번호

1번 100  
4번 70  
5번 10  
6번 95  
7번 45  
8번 90

Name: 수학, dtype: int64

- 데이터프레임(df)에서 '키'가 185 이상 학생들의 이름, 수학, 과학 데이터를 출력한다.

```
df.loc[df['키'] >= 185, ['이름','수학','과학']]
```

|      | 이름  | 수학  | 과학 |
|------|-----|-----|----|
| 지원번호 |     |     |    |
| 1번   | 채치수 | 100 | 95 |
| 4번   | 서태웅 | 70  | 75 |
| 5번   | 강백호 | 10  | 35 |
| 6번   | 변덕규 | 95  | 85 |
| 7번   | 황태산 | 45  | 40 |
| 8번   | 윤대협 | 90  | 95 |

- 데이터프레임(df)에서 '키'가 185 이상인 '북산고' 학생들 데이터를 출력한다.

```
filt = (df['키'] >=185) & (df['학교'] == '북산고')
df.loc[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|----|----|-----|----|----|--------|
| 지원번호 |     |     |     |    |    |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN    |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN    |

- 데이터프레임(df)에서 '키'가 170보다 작거나, 200보다 큰 학생 데이터를 출력한다.

```
filt = (df['키'] < 170) | (df['키'] > 200)
df.loc[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어  | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|-----|----|----|----|------------|
| 지원번호 |     |     |     |    |     |    |    |    |            |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75  | 70 | 80 | 75 | Javascript |
| 6번   | 변덕규 | 능남고 | 202 | 80 | 100 | 95 | 85 | 80 | C          |

- 데이터프레임(df)에서 성이 '송'인 데이터를 출력한다.

```
filt = df['이름'].str.startswith('송')
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|----|----|----|------------|
| 지원번호 |     |     |     |    |    |    |    |    |            |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript |

```
df.loc[filt, ['이름', '학교']]
```

|      | 이름  | 학교  |
|------|-----|-----|
| 지원번호 |     |     |
| 3번   | 송태섭 | 북산고 |

- 데이터프레임(df)에서 이름에 '태'가 들어가는 데이터를 출력한다.

```
filt = df['이름'].str.contains('태')
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|----|----|----|------------|
| 지원번호 |     |     |     |    |    |    |    |    |            |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70 | 75 | 80 | NaN        |
| 7번   | 황태산 | 능남고 | 188 | 55 | 65 | 45 | 40 | 35 | PYTHON     |

- 데이터프레임(df)에서 이름에 '태'가 들어가는 사람을 제외하고 출력한다.

```
df[~filt]
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|-----|-----|-----|----|----|--------|
| 지원번호 |     |     |     |     |     |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java   |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN    |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C      |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#     |

```
df.loc[~filt, '이름':'수학']
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  |
|------|-----|-----|-----|-----|-----|-----|
| 지원번호 |     |     |     |     |     |     |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  |

- 데이터프레임(df)에서 'SW특기'가 'Python'이거나 'Java'인 데이터를 출력한다.

```
langs = ['Python', 'Java']
filt = df['SW특기'].isin(langs)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|----|----|-----|----|----|--------|
| 지원번호 |     |     |     |    |    |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java   |

- 데이터프레임(df)에서 'SW특기'를 모두 소문자로 바꾼 후 소문자끼리 비교하여 데이터를 출력한다.

```
langs = ['python', 'java']
filt = df['SW특기'].str.lower().isin(langs)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기   |
|------|-----|-----|-----|----|----|-----|----|----|--------|
| 지원번호 |     |     |     |    |    |     |    |    |        |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java   |
| 7번   | 황태산 | 능남고 | 188 | 55 | 65 | 45  | 40 | 35 | PYTHON |

- 데이터프레임(df)에서 'SW특기'를 모두 대문자로 바꾼 후 대문자끼리 비교하여 데이터를 출력한다.

```
langs = ['C', 'C#']
filt = df['SW특기'].str.upper().isin(langs)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기 |
|------|-----|-----|-----|-----|-----|----|----|----|------|
| 지원번호 |     |     |     |     |     |    |    |    |      |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C    |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#   |

- DataFrame(df)에서 'SW특기'에 'Java'가 들어가는 학생들의 True, False 값 출력한다.

```
df['SW특기'].str.contains('Java')
```

```
지원번호
1번      False
2번      True
3번      True
4번      NaN
5번      NaN
6번      False
7번      False
8번      False
Name: SW특기, dtype: bool
```

- filtering을 하면 NaN(널)값이 있어 오류가 발생한다.

```
filt = df['SW특기'].str.contains('Java')
df[filt]
```

- DataFrame(df)에서 'NaN'을 False로 설정한다. (NaN행은 출력되지 않는다.)

```
filt = df['SW특기'].str.contains('Java', na=False)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|----|----|----|------------|
| 지원번호 |     |     |     |    |    |    |    |    |            |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50 | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript |

- 'NaN'을 True로 설정하면 NaN행도 출력된다. [구글]-[pandas string]

```
filt = df['SW특기'].str.contains('Java', na=True)
df[filt]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|----|----|----|------------|
| 지원번호 |     |     |     |    |    |    |    |    |            |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50 | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70 | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20 | 10 | 35 | 10 | NaN        |



## 10. 결측치(비어있는 데이터)

- 새로운 파일 '10.결측치.ipynb'를 생성하여 'score.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('data/score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 'SW특기'열의 'NaN'값을 빈칸으로 출력한다.

```
df.fillna({'SW특기':''})
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |            |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |            |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 'SW특기'열의 'NaN'값을 '없음'으로 출력한다.

```
df.fillna({'SW특기':'없음'})
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | 없음         |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | 없음         |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 '학교'열 전체를 'NaN'으로 채운다. numpy는 수학 관련 라이브러리이다.

```
import numpy as np
df['학교'] = np.nan
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | NaN | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | NaN | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | NaN | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | NaN | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | NaN | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | NaN | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | NaN | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | NaN | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 모든 열의 'NaN'값을 모두 '모름'으로 출력한다.

```
df.fillna('모름')
```

|      | 이름  | 학교 | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |    |     |     |     |     |    |    |            |
| 1번   | 채치수 | 모름 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 모름 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 모름 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 모름 | 187 | 40  | 60  | 70  | 75 | 80 | 모름         |
| 5번   | 강백호 | 모름 | 188 | 15  | 20  | 10  | 35 | 10 | 모름         |
| 6번   | 변덕규 | 모름 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 모름 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 모름 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 '학교'열의 'NaN'값은 '무슨고', 'SW특기'열의 'NaN'값은 '없음'으로 출력한다.

```
fill_dict = {'학교':'무슨고', 'SW특기':'없음'}
df.fillna(fill_dict)
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 무슨고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 무슨고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 무슨고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 무슨고 | 187 | 40  | 60  | 70  | 75 | 80 | 없음         |
| 5번   | 강백호 | 무슨고 | 188 | 15  | 20  | 10  | 35 | 10 | 없음         |
| 6번   | 변덕규 | 무슨고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 무슨고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 무슨고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 'SW특기'열 값을 'NaN'에 대해서 '확인 중'으로 채워 저장한다.

```
df = pd.read_excel('data/score.xlsx', index_col='지원번호')
```

```
df.fillna({'SW특기':'확인 중'}, inplace=True)
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | 확인 중       |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | 확인 중       |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- '5번' index, '학교' 열값을 'NaN'값으로 수정한다.

```
df = pd.read_excel('data/score.xlsx', index_col='지원번호')
```

```
df.loc['5번', '학교'] = np.nan
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | NaN | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- ffill() 함수는 forward fill의 약자로 'NaN'값을 바로 위의 값으로 출력한다. (처음 행이 'NaN'이면 'NaN')

```
df.ffill()
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | Javascript |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | Javascript |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- `bfill()` 함수는 **backward fill**의 약자로 'NaN'값을 바로 아래 값으로 출력한다. (마지막행이 'NaN'이면 'NaN')

`df.bfill()`

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | C          |
| 5번   | 강백호 | 능남고 | 188 | 15  | 20  | 10  | 35 | 10 | C          |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- '1번' 'index', '학교' 열값을 'NaN'값으로 수정한다.

`df.loc['1번', '학교'] = np.nan`

`df`

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | NaN | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | NaN | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 전체 데이터 중에서 'NaN'을 포함하는 데이터를 삭제한다.

`df.dropna(inplace=True)`

`df`

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|----|----|----|------------|
| 지원번호 |     |     |     |     |     |    |    |    |            |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50 | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70 | 80 | 75 | Javascript |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95 | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45 | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90 | 95 | 95 | C#         |

- 데이터를 다시 불러와 데이터프레임(df)에 저장한다.

`df = pd.read_excel('data/score.xlsx', index_col='지원번호')`

- 데이터프레임(df)에서 'NaN'이 하나라도 있는 인덱스 행을 제외하고 출력한다.

1) axis: (index or columns)

2) how:(any or all)

```
df.dropna(axis='index', how='any')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 'NaN'이 하나라도 있는 칼럼을 제외하고 출력된다. (how는 생각하면 any)

```
df.dropna(axis='columns')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 |
|------|-----|-----|-----|-----|-----|-----|----|----|
| 지원번호 |     |     |     |     |     |     |    |    |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 |

- 데이터프레임(df)에서 '9번' 'index' 행에 열값을 모두 'NaN'으로 채운다.

```
import numpy as np
df.loc['9번'] = [np.nan] * 9
df
```

|      | 이름  | 학교  | 키     | 국어    | 영어    | 수학    | 과학   | 사회   | SW특기       |
|------|-----|-----|-------|-------|-------|-------|------|------|------------|
| 지원번호 |     |     |       |       |       |       |      |      |            |
| 1번   | 채치수 | 북산고 | 197.0 | 90.0  | 85.0  | 100.0 | 95.0 | 85.0 | Python     |
| 2번   | 정대만 | 북산고 | 184.0 | 40.0  | 35.0  | 50.0  | 55.0 | 25.0 | Java       |
| 3번   | 송태섭 | 북산고 | 168.0 | 80.0  | 75.0  | 70.0  | 80.0 | 75.0 | Javascript |
| 4번   | 서태웅 | 북산고 | 187.0 | 40.0  | 60.0  | 70.0  | 75.0 | 80.0 | NaN        |
| 5번   | 강백호 | 북산고 | 188.0 | 15.0  | 20.0  | 10.0  | 35.0 | 10.0 | NaN        |
| 6번   | 변덕규 | 능남고 | 202.0 | 80.0  | 100.0 | 95.0  | 85.0 | 80.0 | C          |
| 7번   | 황태산 | 능남고 | 188.0 | 55.0  | 65.0  | 45.0  | 40.0 | 35.0 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190.0 | 100.0 | 85.0  | 90.0  | 95.0 | 95.0 | C#         |
| 9번   | NaN | NaN | NaN   | NaN   | NaN   | NaN   | NaN  | NaN  | NaN        |

- 데이터프레임(df)에서 새로운 '학년' 열 모든 행에 'NaN'으로 채운다.

```
df['학년'] = np.nan
```

```
df
```

|      | 이름  | 학교  | 키     | 국어    | 영어    | 수학    | 과학   | 사회   | SW특기       | 학년  |
|------|-----|-----|-------|-------|-------|-------|------|------|------------|-----|
| 지원번호 |     |     |       |       |       |       |      |      |            |     |
| 1번   | 채치수 | 북산고 | 197.0 | 90.0  | 85.0  | 100.0 | 95.0 | 85.0 | Python     | NaN |
| 2번   | 정대만 | 북산고 | 184.0 | 40.0  | 35.0  | 50.0  | 55.0 | 25.0 | Java       | NaN |
| 3번   | 송태섭 | 북산고 | 168.0 | 80.0  | 75.0  | 70.0  | 80.0 | 75.0 | Javascript | NaN |
| 4번   | 서태웅 | 북산고 | 187.0 | 40.0  | 60.0  | 70.0  | 75.0 | 80.0 | NaN        | NaN |
| 5번   | 강백호 | 북산고 | 188.0 | 15.0  | 20.0  | 10.0  | 35.0 | 10.0 | NaN        | NaN |
| 6번   | 변덕규 | 능남고 | 202.0 | 80.0  | 100.0 | 95.0  | 85.0 | 80.0 | C          | NaN |
| 7번   | 황태산 | 능남고 | 188.0 | 55.0  | 65.0  | 45.0  | 40.0 | 35.0 | PYTHON     | NaN |
| 8번   | 윤대협 | 능남고 | 190.0 | 100.0 | 85.0  | 90.0  | 95.0 | 95.0 | C#         | NaN |
| 9번   | NaN | NaN | NaN   | NaN   | NaN   | NaN   | NaN  | NaN  | NaN        | NaN |

- 데이터프레임(df)에서 열 데이터 전체가 'NaN'인 경우에만 열을 삭제한다.

```
df.dropna(axis='columns', how='all', inplace=True)
```

|      | 이름  | 학교  | 키     | 국어    | 영어    | 수학    | 과학   | 사회   | SW특기       |
|------|-----|-----|-------|-------|-------|-------|------|------|------------|
| 지원번호 |     |     |       |       |       |       |      |      |            |
| 1번   | 채치수 | 북산고 | 197.0 | 90.0  | 85.0  | 100.0 | 95.0 | 85.0 | Python     |
| 2번   | 정대만 | 북산고 | 184.0 | 40.0  | 35.0  | 50.0  | 55.0 | 25.0 | Java       |
| 3번   | 송태섭 | 북산고 | 168.0 | 80.0  | 75.0  | 70.0  | 80.0 | 75.0 | Javascript |
| 4번   | 서태웅 | 북산고 | 187.0 | 40.0  | 60.0  | 70.0  | 75.0 | 80.0 | NaN        |
| 5번   | 강백호 | 북산고 | 188.0 | 15.0  | 20.0  | 10.0  | 35.0 | 10.0 | NaN        |
| 6번   | 변덕규 | 능남고 | 202.0 | 80.0  | 100.0 | 95.0  | 85.0 | 80.0 | C          |
| 7번   | 황태산 | 능남고 | 188.0 | 55.0  | 65.0  | 45.0  | 40.0 | 35.0 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190.0 | 100.0 | 85.0  | 90.0  | 95.0 | 95.0 | C#         |
| 9번   | NaN | NaN | NaN   | NaN   | NaN   | NaN   | NaN  | NaN  | NaN        |

- 데이터프레임(df)에서 index 행 데이터 전체가 'NaN'인 경우에만 행을 삭제한다.

```
df.dropna(axis='index', how='all', inplace=True)
```

|      | 이름  | 학교  | 키     | 국어    | 영어    | 수학    | 과학   | 사회   | SW특기       |
|------|-----|-----|-------|-------|-------|-------|------|------|------------|
| 지원번호 |     |     |       |       |       |       |      |      |            |
| 1번   | 채치수 | 북산고 | 197.0 | 90.0  | 85.0  | 100.0 | 95.0 | 85.0 | Python     |
| 2번   | 정대만 | 북산고 | 184.0 | 40.0  | 35.0  | 50.0  | 55.0 | 25.0 | Java       |
| 3번   | 송태섭 | 북산고 | 168.0 | 80.0  | 75.0  | 70.0  | 80.0 | 75.0 | Javascript |
| 4번   | 서태웅 | 북산고 | 187.0 | 40.0  | 60.0  | 70.0  | 75.0 | 80.0 | NaN        |
| 5번   | 강백호 | 북산고 | 188.0 | 15.0  | 20.0  | 10.0  | 35.0 | 10.0 | NaN        |
| 6번   | 변덕규 | 능남고 | 202.0 | 80.0  | 100.0 | 95.0  | 85.0 | 80.0 | C          |
| 7번   | 황태산 | 능남고 | 188.0 | 55.0  | 65.0  | 45.0  | 40.0 | 35.0 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190.0 | 100.0 | 85.0  | 90.0  | 95.0 | 95.0 | C#         |

## 11. 데이터 정렬

- 새로운 파일 '11.데이터정렬.ipynb'를 생성하고 'score.xlsx' 파일을 생성하고 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 '키' 기준으로 '오름차순' 정렬한다.

```
df.sort_values('키')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |

- 데이터프레임(df)에서 '키'가 큰 순으로 '내림차순' 정렬한다.

```
df.sort_values('키', ascending=False)
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |

- 데이터프레임(df)에서 '수학', '영어' 점수 기준으로 '오름차순' 정렬한다.

```
df.sort_values(['수학', '영어'])
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |

- 데이터프레임(df)에서 '수학', '영어' 점수 기준으로 '내림차순' 정렬한다.

```
df.sort_values(['수학', '영어'], ascending=False)
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |

- 데이터프레임(df)에서 '수학' 점수는 '오름차순'으로, '영어' 점수는 '내림차순'으로 정렬한다.

```
df.sort_values(['수학', '영어'], ascending=[True, False])
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |



- 데이터프레임(df)에서 '키' 순으로 '오름차순' 정렬한 후 '키' 데이터만 출력한다.

```
df['키'].sort_values()
```

```
지원번호
3번    168
2번    184
4번    187
5번    188
7번    188
8번    190
1번    197
6번    202
Name: 키, dtype: int64
```

- 데이터프레임(df)에서 '키' 순으로 '내림차순' 정렬한 후 '키' 데이터만 출력한다.

```
df['키'].sort_values(ascending=False)
```

```
지원번호
6번    202
1번    197
8번    190
5번    188
7번    188
4번    187
2번    184
3번    168
Name: 키, dtype: int64
```

- 데이터프레임(df)에서 index(지원번호) 순으로 '오름차순' 정렬한다.

```
df.sort_index()
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 index(지원번호) 순으로 '내림차순' 정렬한다.

```
df.sort_index(ascending=False)
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

## 12. 데이터 수정

- 새로운 파일 '12.데이터수정.ipynb'를 생성하고 'score.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 학교 이름 '북산고'를 '상북고'로, '능남고'는 '무슨고'로 출력한다.

```
df['학교'].replace({'북산고':'상북고', '능남고':'무슨고'})
```

```
지원번호
1번    상북고
2번    상북고
3번    상북고
4번    상북고
5번    상북고
6번    무슨고
7번    무슨고
8번    무슨고
Name: 학교, dtype: object
```

- 데이터프레임(df)에서 학교 이름 '북산고'를 '상북고'로, '능남고'는 '무슨고'로 수정한다.

```
df['학교'].replace({'북산고':'상북고', '능남고':'무슨고'}, inplace=True)
df.head()
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 상북고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 상북고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 상북고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 상북고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 상북고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |

- 위 작업과 동일하게 수정된다.

```
df.replace({'학교':{'북산고':'상북고', '능남고':'무슨고'}}, inplace=True)
```

- 데이터프레임(df)에서 'SW특기'를 소문자로 출력한다.

```
df['SW특기'].str.lower()
```

지원번호

```
1번      python
2번      java
3번      javascript
4번      NaN
5번      NaN
6번      c
7번      python
8번      c#
```

Name: SW특기, dtype: object

- 데이터프레임(df)에서 'SW특기'를 모두 소문자로 바꾸어 'SW특기'열에 저장한다.

```
df['SW특기'] = df['SW특기'].str.lower()
```

```
df.head(3)
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 상북고 | 197 | 90 | 85 | 100 | 95 | 85 | python     |
| 2번   | 정대만 | 상북고 | 184 | 40 | 35 | 50  | 55 | 25 | java       |
| 3번   | 송태섭 | 상북고 | 168 | 80 | 75 | 70  | 80 | 75 | javascript |

- 데이터프레임(df)에서 'SW특기'를 대문자로 바꾸어 'SW특기'열에 저장한다.

```
df['SW특기'] = df['SW특기'].str.upper()
```

```
df.head(5)
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 상북고 | 197 | 90 | 85 | 100 | 95 | 85 | PYTHON     |
| 2번   | 정대만 | 상북고 | 184 | 40 | 35 | 50  | 55 | 25 | JAVA       |
| 3번   | 송태섭 | 상북고 | 168 | 80 | 75 | 70  | 80 | 75 | JAVASCRIPT |
| 4번   | 서태웅 | 상북고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 상북고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |

- 데이터프레임(df)에서 'SW특기'를 첫 글자만 대문자로 바꾸어 'SW특기'열에 저장한다.

```
df['SW특기'] = df['SW특기'].str.capitalize()
```

```
df.head(5)
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 상북고 | 197 | 90 | 85 | 100 | 95 | 85 | PYTHON     |
| 2번   | 정대만 | 상북고 | 184 | 40 | 35 | 50  | 55 | 25 | JAVA       |
| 3번   | 송태섭 | 상북고 | 168 | 80 | 75 | 70  | 80 | 75 | JAVASCRIPT |
| 4번   | 서태웅 | 상북고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 상북고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |

- 데이터프레임(df)에서 '학교' 칼럼에 '등학교' 단어를 추가해 저장한다.

```
df['학교'] = df['학교'] + '등학교'
df.head(5)
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |        |     |     |     |     |    |    |            |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | PYTHON     |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | JAVA       |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | JAVASCRIPT |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 무슨고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 무슨고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 무슨고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 모든 과목의 합계를 구하여 새로운 '총합' 열에 저장한다.

```
df['총합'] = df['국어'] + df['영어'] + df['수학'] + df['과학'] + df['사회']
df
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 총합  |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|-----|
| 지원번호 |     |        |     |     |     |     |    |    |            |     |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python     | 455 |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | Java       | 205 |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | 380 |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        | 325 |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        | 90  |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          | 440 |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | Python     | 240 |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | 465 |

- 데이터프레임(df)에서 '결과' 열을 추가하고 'Fail'값을 저장한다.

```
df['결과'] = 'Fail'
df
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|-----|------|
| 지원번호 |     |        |     |     |     |     |    |    |            |     |      |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python     | 455 | Fail |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | Java       | 205 | Fail |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | 380 | Fail |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        | 325 | Fail |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        | 90  | Fail |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          | 440 | Fail |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | Python     | 240 | Fail |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | 465 | Fail |

- 데이터프레임(df)에서 '총합'이 400보다 큰 결과를 'Pass'로 저장한다.

```
filt = df['총합'] > 400
df.loc[filt, '결과'] = 'Pass'
df
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|-----|------|
| 지원번호 |     |        |     |     |     |     |    |    |            |     |      |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python     | 455 | Pass |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | Java       | 205 | Fail |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | 380 | Fail |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        | 325 | Fail |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        | 90  | Fail |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          | 440 | Pass |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | Python     | 240 | Fail |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | 465 | Pass |

- 데이터프레임(df)에서 '총합' 열을 제외한 데이터를 출력한다.

```
df.drop(columns=['총합'])
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 결과   |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|------|
| 지원번호 |     |        |     |     |     |     |    |    |            |      |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python     | Pass |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | Java       | Fail |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | Fail |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        | Fail |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        | Fail |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          | Pass |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | Python     | Fail |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | Pass |

- 데이터프레임(df)에서 '키', 'SW특기' 열을 제외한 데이터를 출력한다.

```
df.drop(columns=['키', 'SW특기'])
```

|      | 이름  | 학교     | 국어  | 영어  | 수학  | 과학 | 사회 | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|----|----|-----|------|
| 지원번호 |     |        |     |     |     |    |    |     |      |
| 1번   | 채치수 | 상북고등학교 | 90  | 85  | 100 | 95 | 85 | 455 | Pass |
| 2번   | 정대만 | 상북고등학교 | 40  | 35  | 50  | 55 | 25 | 205 | Fail |
| 3번   | 송태섭 | 상북고등학교 | 80  | 75  | 70  | 80 | 75 | 380 | Fail |
| 4번   | 서태웅 | 상북고등학교 | 40  | 60  | 70  | 75 | 80 | 325 | Fail |
| 5번   | 강백호 | 상북고등학교 | 15  | 20  | 10  | 35 | 10 | 90  | Fail |
| 6번   | 변덕규 | 무슨고등학교 | 80  | 100 | 95  | 85 | 80 | 440 | Pass |
| 7번   | 황태산 | 무슨고등학교 | 55  | 65  | 45  | 40 | 35 | 240 | Fail |
| 8번   | 윤대협 | 무슨고등학교 | 100 | 85  | 90  | 95 | 95 | 465 | Pass |

- 데이터프레임(df)에서 index '4번' 행을 제외한 데이터를 출력한다.

```
df.drop(index='4번')
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|-----|----|----|------------|-----|------|
| 지원번호 |     |        |     |     |     |     |    |    |            |     |      |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python     | 455 | Pass |
| 2번   | 정대만 | 상북고등학교 | 184 | 40  | 35  | 50  | 55 | 25 | Java       | 205 | Fail |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript | 380 | Fail |
| 5번   | 강백호 | 상북고등학교 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        | 90  | Fail |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C          | 440 | Pass |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45  | 40 | 35 | Python     | 240 | Fail |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#         | 465 | Pass |

- 데이터프레임(df)에서 index 0~5 연속되는 행을 제외한 데이터를 출력한다.

```
df.index
```

```
Index(['1번', '2번', '3번', '4번', '5번', '6번', '7번', '8번'], dtype='object', name='지원번호')
```

```
df.drop(index=df.index[0:5])
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기   | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|----|----|----|--------|-----|------|
| 지원번호 |     |        |     |     |     |    |    |    |        |     |      |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95 | 85 | 80 | C      | 440 | Pass |
| 7번   | 황태산 | 능남고등학교 | 188 | 55  | 65  | 45 | 40 | 35 | Python | 240 | Fail |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90 | 95 | 95 | C#     | 465 | Pass |

- 데이터프레임(df)에서 '수학' 점수가 80점미만 데이터만 추출하여 출력한다.

```
filt = df['수학'] < 80
```

```
df[filt]
```

|      | 이름  | 학교     | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기       | 총합  | 결과   |
|------|-----|--------|-----|----|----|----|----|----|------------|-----|------|
| 지원번호 |     |        |     |    |    |    |    |    |            |     |      |
| 2번   | 정대만 | 상북고등학교 | 184 | 40 | 35 | 50 | 55 | 25 | Java       | 205 | Fail |
| 3번   | 송태섭 | 상북고등학교 | 168 | 80 | 75 | 70 | 80 | 75 | Javascript | 380 | Fail |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40 | 60 | 70 | 75 | 80 | NaN        | 325 | Fail |
| 5번   | 강백호 | 상북고등학교 | 188 | 15 | 20 | 10 | 35 | 10 | NaN        | 90  | Fail |
| 7번   | 황태산 | 능남고등학교 | 188 | 55 | 65 | 45 | 40 | 35 | Python     | 240 | Fail |

- 위 조건을 만족하는 인덱스 값을 출력한다.

```
df[filt].index
```

```
Index(['2번', '3번', '4번', '5번', '7번'], dtype='object', name='지원번호')
```

- 위 조건을 만족하는 행을 제외한 데이터를 출력한다.

```
df.drop(index=df[filt].index)
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기   | 총합  | 결과   |
|------|-----|--------|-----|-----|-----|-----|----|----|--------|-----|------|
| 지원번호 |     |        |     |     |     |     |    |    |        |     |      |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python | 455 | Pass |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C      | 440 | Pass |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#     | 465 | Pass |

- 위 조건을 만족하는 행과 '총합', '결과' 열을 제외한 데이터를 출력한다.

```
df.drop(index=df[filt].index, columns=['총합', '결과'])
```

|      | 이름  | 학교     | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기   |
|------|-----|--------|-----|-----|-----|-----|----|----|--------|
| 지원번호 |     |        |     |     |     |     |    |    |        |
| 1번   | 채치수 | 상북고등학교 | 197 | 90  | 85  | 100 | 95 | 85 | Python |
| 6번   | 변덕규 | 능남고등학교 | 202 | 80  | 100 | 95  | 85 | 80 | C      |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85  | 90  | 95 | 95 | C#     |

- 데이터프레임(df)에서 index '9번' 행에 새로운 학생 데이터를 추가한다.

```
df.loc['9번'] = ['이정환', '해남고등학교', 184, 90, 90, 90, 90, 90, 90, 'Kotlin', 450, 'Pass']
df.tail(2)
```

|      | 이름  | 학교     | 키   | 국어  | 영어 | 수학 | 과학 | 사회 | SW특기   | 총합  | 결과   |
|------|-----|--------|-----|-----|----|----|----|----|--------|-----|------|
| 지원번호 |     |        |     |     |    |    |    |    |        |     |      |
| 8번   | 윤대협 | 능남고등학교 | 190 | 100 | 85 | 90 | 95 | 95 | C#     | 465 | Pass |
| 9번   | 이정환 | 해남고등학교 | 184 | 90  | 90 | 90 | 90 | 90 | Kotlin | 450 | Pass |

- 데이터프레임(df)에서 '4번' 학생의 'SW특기'를 'Python'으로 변경한다.

```
df.loc['4번', 'SW특기'] = 'Python'
df.head(4)
```

|      | 이름  | 학교     | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기   | 총합  | 결과   |
|------|-----|--------|-----|----|----|----|----|----|--------|-----|------|
| 지원번호 |     |        |     |    |    |    |    |    |        |     |      |
| 4번   | 서태웅 | 상북고등학교 | 187 | 40 | 60 | 70 | 75 | 80 | Python | 325 | Fail |

- 데이터프레임(df)에서 '5번' 학생의 학교는 '능남고등학교'로, 'SW특기'는 'C'로 변경한다.

```
df.loc['5번', ['학교', 'SW특기']] = ['능남고등학교', 'C']
df.loc[['5번']]
```

|      | 이름  | 학교     | 키   | 국어 | 영어 | 수학 | 과학 | 사회 | SW특기 | 총합 | 결과   |
|------|-----|--------|-----|----|----|----|----|----|------|----|------|
| 지원번호 |     |        |     |    |    |    |    |    |      |    |      |
| 5번   | 강백호 | 능남고등학교 | 188 | 15 | 20 | 10 | 35 | 10 | C    | 90 | Fail |

- 데이터프레임(df)에서 열 이름들을 list 타입으로 변경하여 저장 후 출력한다.

```
cols = list(df.columns)
cols
```

```
['이름', '학교', '키', '국어', '영어', '수학', '과학', '사회', 'SW특기', '총합', '결과']
```

- 맨 뒤에 있는 '결과'열을 앞으로 가져오고, 나머지 열들과 합쳐서 순서를 변경한 후 새로운 데이터프레임(df1)을 만든다.

```
df1 = df[[cols[-1]] + cols[0:-1]]
df1.head()
```

|      | 결과   | 이름  | 학교     | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       | 총합  |
|------|------|-----|--------|-----|----|----|-----|----|----|------------|-----|
| 지원번호 |      |     |        |     |    |    |     |    |    |            |     |
| 1번   | Pass | 채치수 | 상북고등학교 | 197 | 90 | 85 | 100 | 95 | 85 | Python     | 455 |
| 2번   | Fail | 정대만 | 상북고등학교 | 184 | 40 | 35 | 50  | 55 | 25 | Java       | 205 |
| 3번   | Fail | 송태섭 | 상북고등학교 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript | 380 |
| 4번   | Fail | 서태웅 | 상북고등학교 | 187 | 40 | 60 | 70  | 75 | 80 | Python     | 325 |
| 5번   | Fail | 강백호 | 능남고등학교 | 188 | 15 | 20 | 10  | 35 | 10 | C          | 90  |

- 데이터프레임(df)에서 '결과', '이름', '학교' 열을 이용해 새로운 데이터프레임(df2)을 만든다.

```
df2 = df[['결과', '이름', '학교']]
df2.head(3)
```

|      | 결과   | 이름  | 학교     |
|------|------|-----|--------|
| 지원번호 |      |     |        |
| 1번   | Pass | 채치수 | 상북고등학교 |
| 2번   | Fail | 정대만 | 상북고등학교 |
| 3번   | Fail | 송태섭 | 상북고등학교 |

- 데이터프레임(df2)의 열 이름을 출력한다.

```
df2.columns
```

```
Index(['결과', '이름', '학교'], dtype='object')
```

- 데이터프레임(df2)의 열 이름을 'Result', 'Name', 'School'로 변경한다.

```
df2.columns = ['Result', 'Name', 'School']
df2.head(3)
```

|      | Result | Name | School |
|------|--------|------|--------|
| 지원번호 |        |      |        |
| 1번   | Pass   | 채치수  | 상북고등학교 |
| 2번   | Fail   | 정대만  | 상북고등학교 |
| 3번   | Fail   | 송태섭  | 상북고등학교 |



### 13. 함수 적용

- 새로운 파일 '13.함수적용.ipynb'를 생성하고 'score.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- '키'열이 정수이면 'cm'를 연결하고 아니면 'cm'를 제거한 후 정수로 변환하는 함수를 작성한 후 적용한다.

```
def add_cm(height):
    if isinstance(height, int):
        return str(height) + 'cm'
    else:
        return int(height.replace('cm', ''))

df['키'] = df['키'].apply(add_cm)
df.head(3)
```

|      | 이름  | 학교  | 키     | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-------|----|----|-----|----|----|------------|
| 지원번호 |     |     |       |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197cm | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184cm | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168cm | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187cm | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188cm | 15 | 20 | 10  | 35 | 10 | NaN        |

- 위 함수를 익명의 람다함수로 작성하면 아래와 같다.

```
df['키'] = df['키'].apply(lambda x:str(x) + 'cm' if isinstance(x, int) else int(x.replace('cm', '')))
df.head(3)
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|----|----|-----|----|----|------------|
| 지원번호 |     |     |     |    |    |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20 | 10  | 35 | 10 | NaN        |

- 'SW특기'에 첫 글자를 대문자로 변환하여 함수를 정의한 후 'SW특기'에 적용한다.

```
def capitalize(lang):
    if pd.notnull(lang):
        return lang.capitalize()
    return lang

df['SW특기'] = df['SW특기'].apply(capitalize)
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | Python     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 위 함수를 익명의 람다함수로 작성하면 아래와 같다.

```
df['SW특기'] = df['SW특기'].apply(lambda x: x.capitalize() if pd.notna(x) else x)
df
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | Python     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 '국어'~'사회' Column의 모든 행을 새로운 데이터프레임(df\_score)에 저장한다.

```
df_score = df.loc[:, '국어':'사회']
df_score.head()
```

|      | 국어 | 영어 | 수학  | 과학 | 사회 |
|------|----|----|-----|----|----|
| 지원번호 |    |    |     |    |    |
| 1번   | 90 | 85 | 100 | 95 | 85 |
| 2번   | 40 | 35 | 50  | 55 | 25 |
| 3번   | 80 | 75 | 70  | 80 | 75 |
| 4번   | 40 | 60 | 70  | 75 | 80 |
| 5번   | 15 | 20 | 10  | 35 | 10 |

- 모든 행의 '국어'~'사회' 합을 구하여 새로운 Column(axis=1) '총점'에 저장한다.

```
df_score['총점'] = df_score.apply(lambda x:sum(x['국어':'사회']), axis=1)
df_score
```

|      | 국어  | 영어  | 수학  | 과학 | 사회 | 총점  |
|------|-----|-----|-----|----|----|-----|
| 지원번호 |     |     |     |    |    |     |
| 1번   | 90  | 85  | 100 | 95 | 85 | 455 |
| 2번   | 40  | 35  | 50  | 55 | 25 | 205 |
| 3번   | 80  | 75  | 70  | 80 | 75 | 380 |
| 4번   | 40  | 60  | 70  | 75 | 80 | 325 |
| 5번   | 15  | 20  | 10  | 35 | 10 | 90  |
| 6번   | 80  | 100 | 95  | 85 | 80 | 440 |
| 7번   | 55  | 65  | 45  | 40 | 35 | 240 |
| 8번   | 100 | 85  | 90  | 95 | 95 | 465 |

- 데이터프레임(df\_score)의 모든 각 행의 '총점'/5의 평균을 구하여 새로운 열(axis=1) '평균'에 저장한다.

```
df_score['평균'] = df_score.apply(lambda row:row['총점']/5, axis=1)
```

|      | 국어  | 영어  | 수학  | 과학 | 사회 | 총점  | 평균   |
|------|-----|-----|-----|----|----|-----|------|
| 지원번호 |     |     |     |    |    |     |      |
| 1번   | 90  | 85  | 100 | 95 | 85 | 455 | 91.0 |
| 2번   | 40  | 35  | 50  | 55 | 25 | 205 | 41.0 |
| 3번   | 80  | 75  | 70  | 80 | 75 | 380 | 76.0 |
| 4번   | 40  | 60  | 70  | 75 | 80 | 325 | 65.0 |
| 5번   | 15  | 20  | 10  | 35 | 10 | 90  | 18.0 |
| 6번   | 80  | 100 | 95  | 85 | 80 | 440 | 88.0 |
| 7번   | 55  | 65  | 45  | 40 | 35 | 240 | 48.0 |
| 8번   | 100 | 85  | 90  | 95 | 95 | 465 | 93.0 |

- 데이터프레임(df\_score)의 모든 각 열의 합계를 구하여 새로운 행(axis=0) '합계'에 저장한다.

```
import numpy as np
df_score.loc['합계'] = df_score.apply(np.sum, axis=0)
df_score.tail(2)
```

|      | 국어    | 영어    | 수학    | 과학    | 사회    | 총점     | 평균    |
|------|-------|-------|-------|-------|-------|--------|-------|
| 지원번호 |       |       |       |       |       |        |       |
| 1번   | 90.0  | 85.0  | 100.0 | 95.0  | 85.0  | 455.0  | 91.0  |
| 2번   | 40.0  | 35.0  | 50.0  | 55.0  | 25.0  | 205.0  | 41.0  |
| 3번   | 80.0  | 75.0  | 70.0  | 80.0  | 75.0  | 380.0  | 76.0  |
| 4번   | 40.0  | 60.0  | 70.0  | 75.0  | 80.0  | 325.0  | 65.0  |
| 5번   | 15.0  | 20.0  | 10.0  | 35.0  | 10.0  | 90.0   | 18.0  |
| 6번   | 80.0  | 100.0 | 95.0  | 85.0  | 80.0  | 440.0  | 88.0  |
| 7번   | 55.0  | 65.0  | 45.0  | 40.0  | 35.0  | 240.0  | 48.0  |
| 8번   | 100.0 | 85.0  | 90.0  | 95.0  | 95.0  | 465.0  | 93.0  |
| 합계   | 500.0 | 525.0 | 530.0 | 560.0 | 485.0 | 2600.0 | 520.0 |

## 14. 데이터그룹화

- 새로운 파일 '14.데이터그룹화.ipynb'를 생성하고 'score.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd
df = pd.read_excel('score.xlsx', index_col='지원번호')
```

|      | 이름  | 학교  | 키   | 국어  | 영어  | 수학  | 과학 | 사회 | SW특기       |
|------|-----|-----|-----|-----|-----|-----|----|----|------------|
| 지원번호 |     |     |     |     |     |     |    |    |            |
| 1번   | 채치수 | 북산고 | 197 | 90  | 85  | 100 | 95 | 85 | Python     |
| 2번   | 정대만 | 북산고 | 184 | 40  | 35  | 50  | 55 | 25 | Java       |
| 3번   | 송태섭 | 북산고 | 168 | 80  | 75  | 70  | 80 | 75 | Javascript |
| 4번   | 서태웅 | 북산고 | 187 | 40  | 60  | 70  | 75 | 80 | NaN        |
| 5번   | 강백호 | 북산고 | 188 | 15  | 20  | 10  | 35 | 10 | NaN        |
| 6번   | 변덕규 | 능남고 | 202 | 80  | 100 | 95  | 85 | 80 | C          |
| 7번   | 황태산 | 능남고 | 188 | 55  | 65  | 45  | 40 | 35 | PYTHON     |
| 8번   | 윤대협 | 능남고 | 190 | 100 | 85  | 90  | 95 | 95 | C#         |

- 데이터프레임(df)에서 값을 '학교'별로 그룹화한 후 '능남고' 학생들만 출력한다.

```
df.groupby('학교').get_group('능남고')
```

|      | 이름  | 학교  | 키     | 국어  | 영어  | 수학 | 과학 | 사회 | SW특기   | 학년 |
|------|-----|-----|-------|-----|-----|----|----|----|--------|----|
| 지원번호 |     |     |       |     |     |    |    |    |        |    |
| 6번   | 변덕규 | 능남고 | 202cm | 80  | 100 | 95 | 85 | 80 | C      | 3  |
| 7번   | 황태산 | 능남고 | 188cm | 55  | 65  | 45 | 40 | 35 | Python | 2  |
| 8번   | 윤대협 | 능남고 | 190cm | 100 | 85  | 90 | 95 | 95 | C#     | 2  |

- 데이터프레임(df)에서 값을 '학교'별로 그룹화한 후 '북산고' 학생들만 출력한다.

```
df.groupby('학교').get_group('북산고')
```

|      | 이름  | 학교  | 키     | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       | 학년 |
|------|-----|-----|-------|----|----|-----|----|----|------------|----|
| 지원번호 |     |     |       |    |    |     |    |    |            |    |
| 1번   | 채치수 | 북산고 | 197cm | 90 | 85 | 100 | 95 | 85 | Python     | 3  |
| 2번   | 정대만 | 북산고 | 184cm | 40 | 35 | 50  | 55 | 25 | Java       | 3  |
| 3번   | 송태섭 | 북산고 | 168cm | 80 | 75 | 70  | 80 | 75 | Javascript | 3  |
| 4번   | 서태웅 | 북산고 | 187cm | 40 | 60 | 70  | 75 | 80 | NaN        | 1  |
| 5번   | 강백호 | 북산고 | 188cm | 15 | 20 | 10  | 35 | 10 | NaN        | 1  |

- 데이터프레임(df)에서 '학교'별로 그룹화한 후 '키'의 평균값을 출력한다.

```
df.groupby('학교')[['키']].mean()
```

|     | 키          |
|-----|------------|
| 학교  |            |
| 능남고 | 193.333333 |
| 북산고 | 184.800000 |

- 데이터프레임(df)에서 '학교'별로 그룹화한 후 '국어', '영어', '수학' 평균값을 출력한다.

```
df.groupby('학교')[['국어', '영어', '수학']].mean()
```

|     | 국어        | 영어        | 수학        |
|-----|-----------|-----------|-----------|
| 학교  |           |           |           |
| 능남고 | 78.333333 | 83.333333 | 76.666667 |
| 북산고 | 53.000000 | 55.000000 | 60.000000 |

- 데이터프레임(df)에 새로운 '학년' 칼럼을 추가하고 새로운 데이터를 입력한다.

```
df['학년'] = [3, 3, 3, 1, 1, 3, 2, 2]
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       | 학년 |
|------|-----|-----|-----|----|----|-----|----|----|------------|----|
| 지원번호 |     |     |     |    |    |     |    |    |            |    |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     | 3  |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       | 3  |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript | 3  |

- 데이터프레임(df)에서 '학교'별, '학년'별로 그룹화한 후 '국어', '영어', '수학', '과학', '사회'의 평균값을 출력한다.

```
df.groupby(['학교', '학년'])[['국어', '영어', '수학', '과학', '사회']].mean()
```

|     |    | 국어   | 영어    | 수학        | 과학        | 사회        |
|-----|----|------|-------|-----------|-----------|-----------|
| 학교  | 학년 |      |       |           |           |           |
| 능남고 | 2  | 77.5 | 75.0  | 67.500000 | 67.500000 | 65.000000 |
|     | 3  | 80.0 | 100.0 | 95.000000 | 85.000000 | 80.000000 |
| 북산고 | 1  | 27.5 | 40.0  | 40.000000 | 55.000000 | 45.000000 |
|     | 3  | 70.0 | 65.0  | 73.333333 | 76.666667 | 61.666667 |

- 데이터프레임(df)에서 '학년'별 그룹화한 후 '키'의 평균값을 출력한다.

```
df.groupby('학년')[['키']].mean()
```

|    | 키      |
|----|--------|
| 학년 |        |
| 1  | 187.50 |
| 2  | 189.00 |
| 3  | 187.75 |

- 데이터프레임(df)에서 '학년'별 그룹화한 후 '키'의 평균이 큰 순으로 '내림차순' 정렬한 결과를 출력한다.

```
df.groupby('학년')[['키']].mean().sort_values('키', ascending=False)
```

|    | 키      |
|----|--------|
| 학년 |        |
| 2  | 189.00 |
| 3  | 187.75 |
| 1  | 187.50 |

- 데이터프레임(df)에서 '학년'별 그룹화한 후 index '학년'이 큰 순으로 '내림차순' 정렬한 결과를 출력한다.

```
df.groupby('학년')[['키']].mean().sort_index(ascending=False)
```

|    | 키      |
|----|--------|
| 학년 |        |
| 3  | 187.75 |
| 2  | 189.00 |
| 1  | 187.50 |

- '학년'으로 그룹화해 '국어', '영어' 평균을 구한 후 '국어'를 내림차순, '국어'가 같은 경우 '영어'로 오름차순 정렬한다.

```
df.groupby('학년')[['국어', '영어']].mean().sort_values(['국어', '영어'], ascending=[False, True])
```

|    | 국어   | 영어    |
|----|------|-------|
| 학년 |      |       |
| 1  | 27.5 | 40.00 |
| 3  | 72.5 | 73.75 |
| 2  | 77.5 | 75.00 |

- '학교'별, '학년'별 그룹화한 후 '키'의 평균을 구해 index '학교' 오름차순, index '학년'는 내림차순으로 정렬한다.

```
df.groupby(['학교', '학년'])[['키']].mean().sort_index(ascending=[True, False])
```

|     |    | 키     |
|-----|----|-------|
| 학교  | 학년 |       |
| 능남고 | 3  | 202.0 |
|     | 2  | 189.0 |
| 북산고 | 3  | 183.0 |
|     | 1  | 187.5 |

- 데이터프레임(df)에서 각 '학교'별로 그룹화한 후 각 학교별 학생 수를 출력한다.

```
df.groupby('학교').size()
```

```
학교
능남고    3
북산고    5
dtype: int64
```

- 데이터프레임(df)에서 '학교'별로 그룹화한 후 '능남고' 학생 수를 출력한다.

```
df.groupby('학교').size()['능남고']
```

```
3
```

- 데이터프레임(df)에서 '학교'로 그룹화한 후 각 '학교'별 학생 수와 'SW특기'를 가진 학생 수를 출력한다.

```
df.groupby('학교').count()[['이름','SW특기']]
```

|     | 이름 | SW특기 |
|-----|----|------|
| 학교  |    |      |
| 능남고 | 3  | 3    |
| 북산고 | 5  | 3    |

- 데이터프레임(df)에서 '학교'별, '학년'별로 그룹화한 후 학생 수를 출력한다.

```
df.groupby(['학교', '학년']).size().to_frame('학생수')
```

|     |    | 학생수 |
|-----|----|-----|
| 학교  | 학년 |     |
| 능남고 | 2  | 2   |
|     | 3  | 1   |
| 북산고 | 1  | 2   |
|     | 3  | 3   |

- 데이터프레임(df)에서 'SW특기' 종류를 중복 배제한 후 출력한다.

```
df['SW특기'].unique()
```

```
array(['Python', 'Java', 'Javascript', nan, 'C', 'C#'], dtype=object)
```

- 데이터프레임(df)에서 'SW특기'별 학생 수를 출력한다.

```
df['SW특기'].value_counts().to_frame('학생수')
```

|            | 학생수 |
|------------|-----|
| SW특기       |     |
| Python     | 2   |
| Java       | 1   |
| Javascript | 1   |
| C          | 1   |
| C#         | 1   |

- 데이터프레임(df)에서 'SW특기'별 학생 수를 비율로 출력한다.

```
df['SW특기'].value_counts(normalize=True)
```

|            | 비율       |
|------------|----------|
| SW특기       |          |
| Python     | 0.333333 |
| Java       | 0.166667 |
| Javascript | 0.166667 |
| C          | 0.166667 |
| C#         | 0.166667 |

- '학교'별 그룹화 한 후 'SW특기' 종류를 중복 배제한 후 출력한다.

```
df.groupby('학교')['SW특기'].unique().to_frame('SW특기')
```

| SW특기 |                                 |
|------|---------------------------------|
| 학교   |                                 |
| 능남고  | [C, PYTHON, C#]                 |
| 북산고  | [Python, Java, Javascript, nan] |

- 데이터프레임(df) 'SW특기' null값을 아래와 같이 수정한다.

```
df.loc['4번', 'SW특기'] = 'Python'
df.loc['5번', 'SW특기'] = 'C#'
```

|      | 이름  | 학교  | 키   | 국어 | 영어 | 수학  | 과학 | 사회 | SW특기       | 학년 |
|------|-----|-----|-----|----|----|-----|----|----|------------|----|
| 지원번호 |     |     |     |    |    |     |    |    |            |    |
| 1번   | 채치수 | 북산고 | 197 | 90 | 85 | 100 | 95 | 85 | Python     | 3  |
| 2번   | 정대만 | 북산고 | 184 | 40 | 35 | 50  | 55 | 25 | Java       | 3  |
| 3번   | 송태섭 | 북산고 | 168 | 80 | 75 | 70  | 80 | 75 | Javascript | 3  |
| 4번   | 서태웅 | 북산고 | 187 | 40 | 60 | 70  | 75 | 80 | Python     | 1  |
| 5번   | 강백호 | 북산고 | 188 | 15 | 20 | 10  | 35 | 10 | C#         | 1  |

- '학교'별 그룹화 한 후 'SW특기' 종류를 중복 배제한 후 출력한다.

```
df.groupby('학교')['SW특기'].unique().to_frame('SW특기')
```

| SW특기 |                                |
|------|--------------------------------|
| 학교   |                                |
| 능남고  | [C, Python, C#]                |
| 북산고  | [Python, Java, Javascript, C#] |

- '학교'별 그룹화 한 후 'SW특기'별 학생 수를 구하여 새로운 데이터프레임(school)에 저장한다.

```
school = df.groupby('학교')['SW특기'].value_counts().to_frame('학생수')
```

| 학생수 |            |   |
|-----|------------|---|
| 학교  | SW특기       |   |
| 능남고 | C          | 1 |
|     | C#         | 1 |
|     | Python     | 1 |
| 북산고 | Python     | 2 |
|     | C#         | 1 |
|     | Java       | 1 |
|     | Javascript | 1 |

- '북산고' 학생 중 'SW특기'가 'Python'인 학생이 몇 명인지 출력한다.

```
school.loc[('북산고', 'Python')]
```

학생수 2



## 15. Pandas 퀴즈

다음은 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터이다. 주어진 코드를 이용하여 퀴즈를 풀어보시오.

- 새로운 파일 '15.Pandas퀴즈.ipynb'를 생성하여 아래 문제에 대한 답을 작성하시오.

```
import pandas as pd
data = {
    '영화' : ['명량', '극한직업', '신과함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270], # (단위 : 만 명)
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
df
```

|   | 영화        | 개봉 연도 | 관객 수 | 평점   |
|---|-----------|-------|------|------|
| 0 | 명량        | 2014  | 1761 | 8.88 |
| 1 | 극한직업      | 2019  | 1626 | 9.20 |
| 2 | 신과함께-죄와 벌 | 2017  | 1441 | 8.73 |
| 3 | 국제시장      | 2014  | 1426 | 9.16 |
| 4 | 괴물        | 2006  | 1301 | 8.62 |
| 5 | 도둑들       | 2012  | 1298 | 7.64 |
| 6 | 7번방의 선물   | 2013  | 1281 | 8.83 |
| 7 | 암살        | 2015  | 1270 | 9.10 |

- 데이터프레임(df)에서 '영화' 열만 5행 출력하시오.

```
df[['영화']].head()
```

|   | 영화        |
|---|-----------|
| 0 | 명량        |
| 1 | 극한직업      |
| 2 | 신과함께-죄와 벌 |
| 3 | 국제시장      |
| 4 | 괴물        |

- 데이터프레임(df)에서 '영화', '평점' 열 정보를 출력하시오.

```
df[['영화', '평점']]
```

|   | 영화        | 평점   |
|---|-----------|------|
| 0 | 명량        | 8.88 |
| 1 | 극한직업      | 9.20 |
| 2 | 신과함께-죄와 벌 | 8.73 |
| 3 | 국제시장      | 9.16 |
| 4 | 괴물        | 8.62 |
| 5 | 도둑들       | 7.64 |
| 6 | 7번방의 선물   | 8.83 |
| 7 | 암살        | 9.10 |

- '2015'년 이후에 개봉한 데이터 중에서 '영화', '개봉연도' 정보를 출력하시오.

```
filt = df['개봉 연도'] >= 2015
df.loc[filt, ['영화', '개봉 연도']]
```

|   | 영화        | 개봉 연도 |
|---|-----------|-------|
| 1 | 극한직업      | 2019  |
| 2 | 신과함께-죄와 벌 | 2017  |
| 7 | 암살        | 2015  |

- 주어진 계산식을 참고하여 '추천점수' 칼럼을 추가하시오. (추천점수 = (관객 수 \* 평점) // 1000)

```
df['추천 점수'] = (df['관객 수'] * df['평점']) // 100
df.head()
```

|   | 영화        | 개봉 연도 | 관객 수 | 평점   | 추천 점수 |
|---|-----------|-------|------|------|-------|
| 0 | 명량        | 2014  | 1761 | 8.88 | 156.0 |
| 1 | 극한직업      | 2019  | 1626 | 9.20 | 149.0 |
| 2 | 신과함께-죄와 벌 | 2017  | 1441 | 8.73 | 125.0 |
| 3 | 국제시장      | 2014  | 1426 | 9.16 | 130.0 |
| 4 | 괴물        | 2006  | 1301 | 8.62 | 112.0 |

- 데이터프레임(df)의 전체 데이터를 '개봉연도' 기준 '내림차순'으로 출력하시오.

```
df.sort_values('개봉 연도', ascending=False)
```

|   | 영화        | 개봉 연도 | 관객 수 | 평점   | 추천 점수 |
|---|-----------|-------|------|------|-------|
| 1 | 극한직업      | 2019  | 1626 | 9.20 | 149.0 |
| 2 | 신과함께-죄와 벌 | 2017  | 1441 | 8.73 | 125.0 |
| 7 | 암살        | 2015  | 1270 | 9.10 | 115.0 |
| 0 | 명량        | 2014  | 1761 | 8.88 | 156.0 |
| 3 | 국제시장      | 2014  | 1426 | 9.16 | 130.0 |
| 6 | 7번방의 선물   | 2013  | 1281 | 8.83 | 113.0 |
| 5 | 도둑들       | 2012  | 1298 | 7.64 | 99.0  |
| 4 | 괴물        | 2006  | 1301 | 8.62 | 112.0 |

- '개봉 연도'별로 그룹화 한 후 '관객 수', '평점'의 평균을 구해 '개봉 연도' 내림차순으로 정렬 후 5위까지 출력한다.

```
movies = df.groupby('개봉 연도')[['관객 수', '평점']].mean()
movies.sort_index(ascending=False).head(5)
```

|       | 관객 수   | 평점   |
|-------|--------|------|
| 개봉 연도 |        |      |
| 2019  | 1626.0 | 9.20 |
| 2017  | 1441.0 | 8.73 |
| 2015  | 1270.0 | 9.10 |
| 2014  | 1593.5 | 9.02 |
| 2013  | 1281.0 | 8.83 |

## 16. 타이타닉(EDA:Exploratory Data Analysis)

EDA는 탐색적 데이터 분석(Exploratory Data Analysis)의 약자로 데이터 분석 초기에 데이터를 다양한 관점에서 관찰하고 이해하는 과정을 의미한다. 데이터 분석을 할 때 아래와 같은 이유로 타이타닉 탑승자 데이터를 많이 사용된다.

- 1) 테스트를 위해 만든 임의의 데이터가 아닌 실제 데이터이다.
- 2) 다양한 분석 기법을 사용해서 정확도를 올릴 수 있을 만한 적당한 변수가 있고 데이터 포맷도 꽤 깔끔한 편이다.
- 3) 너무 많지도 적지도 않은 데이터이다.

▪ 새로운 파일 '16.타이타닉.ipynb'를 생성한다.

▪ 아래 kaggle 사이트에서 데이터셋을 다운로드 받는다.

<https://www.kaggle.com/c/titanic/data>

▪ train 데이터셋을 읽어 train 데이터 프레임 변수에 읽어 들인다.

```
import pandas as pd
train = pd.read_csv('data/train.csv')
```

▪ train 데이터를 확인해 본다.

| 칼럼 이름    | 칼럼 설명                               |
|----------|-------------------------------------|
| Survived | 생존유무(0 = 사망, 1 = 생존)                |
| Name     | 탑승객 성명                              |
| Pclass   | 티켓 클래스 (1 = 1st, 2 = 2nd, 3 = 3rd ) |
| Sex      | 성별                                  |
| Age      | 나이(세)                               |
| SibSp    | 동승한 형제자매, 배우자 수 총합                  |
| Parch    | 동승한 부모, 자녀 수 총합                     |
| Embarked | 탑승 항구                               |
| Fare     | 탑승 요금                               |
| Ticket   | 티켓 넘버                               |
| Cabin    | 객실 넘버                               |

train.head()

| PassengerId | Survived | Pclass | Name | Sex                                               | Age    | SibSp | Parch | Ticket | Fare             | Cabin   | Embarked |   |
|-------------|----------|--------|------|---------------------------------------------------|--------|-------|-------|--------|------------------|---------|----------|---|
| 0           | 1        | 0      | 3    | Braund, Mr. Owen Harris                           | male   | 22.0  | 1     | 0      | A/5 21171        | 7.2500  | NaN      | S |
| 1           | 2        | 1      | 1    | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0  | 1     | 0      | PC 17599         | 71.2833 | C85      | C |
| 2           | 3        | 1      | 3    | Heikkinen, Miss. Laina                            | female | 26.0  | 0     | 0      | STON/O2. 3101282 | 7.9250  | NaN      | S |
| 3           | 4        | 1      | 1    | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0  | 1     | 0      | 113803           | 53.1000 | C123     | S |
| 4           | 5        | 0      | 3    | Allen, Mr. William Henry                          | male   | 35.0  | 0     | 0      | 373450           | 8.0500  | NaN      | S |

▪ 891개의 행과 12 칼럼이 존재한다.

train.shape

(891, 12)

- train data의 결측치를 확인한다. train data에는 Age, Cabin, Embarked 칼럼에 결측치가 존재 한다

```
train.isnull().sum()
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            177
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
dtype: int64
```

- 사망, 생존 비율을 구한다. 사망 비율은 61.6%, 생존 비율은 38.4%로 사망한 탑승객의 수가 더 많은 것이 확인 된다.

```
train.value_counts('Survived', normalize=True).to_frame('비율')
```

|          | 비율       |
|----------|----------|
| Survived |          |
| 0        | 0.616162 |
| 1        | 0.383838 |

- 성별 생존율, 사망률을 구한다. 여성의 생존율은 약 74.2%, 남성의 생존율은 약 18.9%로 여성의 생존율이 압도적으로 높은 것이 확인된다. 남성보다 여성을 먼저 구조한 것으로 보인다.

```
sex = train.groupby('Sex').size().to_frame('전체인원')
sex_survived = train[train['Survived']==1].groupby('Sex').size().to_frame('생존인원')
sex = sex.merge(sex_survived, on='Sex')
sex['생존율'] = sex['생존인원'] / sex['전체인원']
```

|        | 전체인원 | 생존인원 | 생존율      |
|--------|------|------|----------|
| Sex    |      |      |          |
| female | 314  | 233  | 0.742038 |
| male   | 577  | 109  | 0.188908 |

- 티켓 클래스(Pclass)별로 생존율과 사망률을 구한다. 3등석 탑승객들이 많이 사망한 것으로 보인다.

```
pclass = train.groupby('Pclass').size().to_frame('전체인원')
pclass_survived = train[train['Survived']==1].groupby('Pclass').size().to_frame('생존인원')
pclass = pclass.merge(pclass_survived, on='Pclass')
pclass['생존율'] = pclass['생존인원'] / pclass['전체인원']
```

|        | 전체인원 | 생존인원 | 생존율      |
|--------|------|------|----------|
| Pclass |      |      |          |
| 1      | 216  | 136  | 0.629630 |
| 2      | 184  | 87   | 0.472826 |
| 3      | 491  | 119  | 0.242363 |

- 탑승객들의 평균 나이는 약 30세이고 주로 20~40세에 분포하고 있다.

```
train['Age'].describe()
```

```
count      714.000000
mean       29.699118
std        14.526497
min         0.420000
25%        20.125000
50%        28.000000
75%        38.000000
max        80.000000
Name: Age, dtype: float64
```

- 탑승객 연령대별(step 10) 인원수를 구한다. 20~30대 인원수가 가장 많다.

```
ages = []
for age in range(0, int(train['Age'].max()+1), 10):
    filt = (train['Age'] >= age) & (train['Age'] < age+10)
    count = train[filt]['Age'].count()
    ages.append({'나이':f'{age}이상~{age+10}미만', '인원수':count})
df = pd.DataFrame(ages)
df
```

|   | 나이        | 인원수 |
|---|-----------|-----|
| 0 | 0이상~10미만  | 62  |
| 1 | 10이상~20미만 | 102 |
| 2 | 20이상~30미만 | 220 |
| 3 | 30이상~40미만 | 167 |
| 4 | 40이상~50미만 | 89  |
| 5 | 50이상~60미만 | 48  |
| 6 | 60이상~70미만 | 19  |
| 7 | 70이상~80미만 | 6   |
| 8 | 80이상~90미만 | 1   |

- 탑승객 연령대별(step 20) 인원수를 구한다. 20대~30대 인원수가 가장 많다.

```
ages = []
for age in range(0, int(train['Age'].max()+1), 20):
    filt = (train['Age'] >= age) & (train['Age'] < age+20)
    count = train[filt]['Age'].count()
    ages.append({'나이':f'{age}이상~{age+10}미만', '인원수':count})
df = pd.DataFrame(ages)
df
```

|   | 나이        | 인원수 |
|---|-----------|-----|
| 0 | 0이상~10미만  | 164 |
| 1 | 20이상~30미만 | 387 |
| 2 | 40이상~50미만 | 137 |
| 3 | 60이상~70미만 | 25  |
| 4 | 80이상~90미만 | 1   |

- SibSp(동승한 형제자매, 배우자 수)별 생존율을 구한다. 혼자 탑승한 사람보다 1명, 2명 SibSp와 탑승한 탑승객들의 사망률이 낮고 3명, 4명 보다는 사망률이 높다.

```
sibsp = train.groupby('SibSp').size()
sibsp = sibsp.to_frame('전체인원')
filt = train['Survived'] == 1
sibsp_survived = train[filt].groupby('SibSp').size()
sibsp_survived = sibsp_survived.to_frame('생존인원')
sibsp = sibsp.merge(sibsp_survived, how='left', on='SibSp')
sibsp['생존율'] = sibsp['생존인원']/sibsp['전체인원']
sibsp.fillna(0, inplace=True)
sibsp.index.name='동승자수'
sibsp
```

|       | 전체인원 | 생존인원  | 생존율      |
|-------|------|-------|----------|
| SibSp |      |       |          |
| 0     | 608  | 210.0 | 0.345395 |
| 1     | 209  | 112.0 | 0.535885 |
| 2     | 28   | 13.0  | 0.464286 |
| 3     | 16   | 4.0   | 0.250000 |
| 4     | 18   | 3.0   | 0.166667 |
| 5     | 5    | 0.0   | 0.000000 |
| 8     | 7    | 0.0   | 0.000000 |

- Parch(동승한 부모, 자녀 수)별 생존율을 구한다. 생존율을 구한 결과 부모, 자녀 없이 혼자 탑승한 사람의 사망률이 1명, 2명, 3명과 함께 탑승한 사망률 보다 높다.

```
parch = train.groupby('Parch').size()
parch = parch.to_frame('전체인원')
filt = train['Survived'] == 1
parch_survived = train[filt].value_counts('Parch')
parch_survived = parch_survived.to_frame('생존인원')
parch = parch.merge(parch_survived, how='left', on='Parch')
parch['생존율'] = parch['생존인원'] / parch['전체인원']
parch.fillna(0, inplace=True)
parch.index.name='동승자수'
parch
```

|       | 전체인원 | 생존인원  | 생존율      |
|-------|------|-------|----------|
| Parch |      |       |          |
| 0     | 678  | 233.0 | 0.343658 |
| 1     | 118  | 65.0  | 0.550847 |
| 2     | 80   | 40.0  | 0.500000 |
| 3     | 5    | 3.0   | 0.600000 |
| 4     | 4    | 0.0   | 0.000000 |
| 5     | 5    | 1.0   | 0.200000 |
| 6     | 1    | 0.0   | 0.000000 |

- 탑승 항구별 사망 비율을 확인한다. S항구 탑승객의 사망률이 가장 높다

```
embarked = train.groupby('Embarked').size().to_frame('전체인원')
filt = train['Survived'] == 1
embarked_survived = train[filt].groupby('Embarked').size().to_frame('생존인원')
embarked = embarked.merge(embarked_survived, on='Embarked')
embarked['생존율'] = embarked['생존인원'] / embarked['전체인원']
embarked
```

|          | 전체인원 | 생존인원 | 생존율      |
|----------|------|------|----------|
| Embarked |      |      |          |
| C        | 168  | 93   | 0.553571 |
| Q        | 77   | 30   | 0.389610 |
| S        | 644  | 217  | 0.336957 |

- 항구별 티켓 클래스별 탑승객수를 구한다. S항구에서 3등석 탑승객들이 많은 탑승한 것을 확인할 수 있다. 위에서 3등석의 사망률이 가장 높았듯이 S항구에서 3등석 탑승객들이 많이 때문에 S항구 탑승객들의 사망률이 높은 것으로 보인다.

```
group = train.groupby(['Embarked', 'Pclass']).size()
c = group['C']
s = group['S']
q = group['Q']
df = pd.DataFrame([c, s, q], index=['C항구', 'S항구', 'Q항구'])
df.index.name = 'Embarked'
df.columns=['1등석', '2등석', '3등석']
```

|          | 1등석 | 2등석 | 3등석 |
|----------|-----|-----|-----|
| Embarked |     |     |     |
| C항구      | 85  | 17  | 66  |
| S항구      | 127 | 164 | 353 |
| Q항구      | 2   | 3   | 72  |

- 티켓 클래스별 탑승 항구별 탑승객수를 구한다. 3등석은 S항구에서 가장 많이 탑승한 것을 확인할 수 있다. 위에서 3등석의 사망률이 가장 높았듯이 3등석에 S항구 탑승객들이 많았으므로 S항구 탑승객들의 사망률이 높은 것으로 보인다.

```
group = train.groupby(['Pclass', 'Embarked']).size()
p1 = group[1]
p2 = group[2]
p3 = group[3]
df = pd.DataFrame([p1, p2, p3], index=['1등석', '2등석', '3등석'])
df.index.name = 'Pclass'
df.columns = ['C항구', 'Q항구', 'S항구']
```

|        | C항구 | Q항구 | S항구 |
|--------|-----|-----|-----|
| Pclass |     |     |     |
| 1등석    | 85  | 2   | 127 |
| 2등석    | 17  | 3   | 164 |
| 3등석    | 66  | 72  | 353 |

- 탑승 항구별 동승자별 탑승객 인원수를 구한다. 혼자 탑승한 탑승객의 수는 S항구 탑승객이 압도적으로 많았다.

```
group = train.groupby(['Embarked', 'SibSp']).size()
df = pd.DataFrame([group['C'], group['Q'], group['S']], index=['C항구', 'Q항구', 'S항구'])
df.index.name='탑승항구'
df.columns=[f'{x}명' for x in df.columns]
df.fillna(0, inplace=True)
```

|      | 0명    | 1명    | 2명   | 4명   | 3명   | 5명  | 8명  |
|------|-------|-------|------|------|------|-----|-----|
| 탑승항구 |       |       |      |      |      |     |     |
| C항구  | 109.0 | 53.0  | 6.0  | 0.0  | 0.0  | 0.0 | 0.0 |
| Q항구  | 59.0  | 11.0  | 3.0  | 4.0  | 0.0  | 0.0 | 0.0 |
| S항구  | 438.0 | 145.0 | 19.0 | 14.0 | 16.0 | 5.0 | 7.0 |

```
group = train.groupby(['Embarked', 'Parch']).size()
df = pd.DataFrame([group['C'], group['Q'], group['S']], index=['C항구', 'Q항구', 'S항구'])
df.index.name='탑승항구'
df.columns=[f'{x}명' for x in df.columns]
df.fillna(0, inplace=True)
```

|      | 0명    | 1명   | 2명   | 3명  | 5명  | 4명  | 6명  |
|------|-------|------|------|-----|-----|-----|-----|
| 탑승항구 |       |      |      |     |     |     |     |
| C항구  | 123.0 | 30.0 | 14.0 | 1.0 | 0.0 | 0.0 | 0.0 |
| Q항구  | 69.0  | 6.0  | 1.0  | 0.0 | 1.0 | 0.0 | 0.0 |
| S항구  | 484.0 | 82.0 | 65.0 | 4.0 | 4.0 | 4.0 | 1.0 |

- 탑승 금액이 적은 쪽은 사망률이 높은 것이 확인 된다. 그러므로 탑승 클래스, 탑승 금액에 따라 생존 여부에 영향을 끼치는 것으로 생각된다.

```
group = train.groupby('Fare').size()
total = train['Fare'].count()
fares = []
for fare in range(0, int(group.index.max())+1, 100):
    filt = (train['Fare']>=fare) & (train['Fare']<fare+100)
    survived = train[filt & (train['Survived']==1)]['Fare'].count()
    dead = train[filt & (train['Survived']==0)]['Fare'].count()
    fares.append({'요금': f'{fare}이상~{fare+100}미만', '생존자수':survived, '사망자수':dead})
df = pd.DataFrame(fares)
df['사망률'] = df['사망자수'] / total
df['생존율'] = df['생존자수'] / total
```

|   | 요금          | 생존자수 | 사망자수 | 사망률      | 생존율      |
|---|-------------|------|------|----------|----------|
| 0 | 0이상~100미만   | 303  | 535  | 0.600449 | 0.340067 |
| 1 | 100이상~200미만 | 25   | 8    | 0.008979 | 0.028058 |
| 2 | 200이상~300미만 | 11   | 6    | 0.006734 | 0.012346 |
| 3 | 300이상~400미만 | 0    | 0    | 0.000000 | 0.000000 |
| 4 | 400이상~500미만 | 0    | 0    | 0.000000 | 0.000000 |
| 5 | 500이상~600미만 | 3    | 0    | 0.000000 | 0.003367 |



## 17. 커피빈 매장 지도 출력

'데이터분석'에서 생성한 '커피빈.csv' 파일을 이용하여 지도에 매장위치를 마커로 출력해 본다.

- 새로운 파일 '17.커피빈맵.ipynb'를 생성하여 '커피빈.csv' 파일을 읽어 cb 변수에 저장한다.

```
import pandas as pd
cb = pd.read_csv('data/커피빈.csv', index_col=0)
cb.reset_index(inplace=True)
cb.head(3)
```

|   | seq | store     | address                         | phone        |
|---|-----|-----------|---------------------------------|--------------|
| 0 | 1   | 삼성봉은사거리점  | 서울시 강남구 영동대로 607 1,2층           | 02-3443-5618 |
| 1 | 2   | 코엑스웨스턴서울점 | 서울시 강남구 봉은사로 524 코엑스 인터컨티넨탈아케이드 | 02-569-3103  |
| 2 | 3   | 삼성오크우드점   | 서울 강남구 테헤란로87길 46 지하 2층         | 02-3466-8507 |

- 카카오 지도 API 이용하여 주소 검색함수로 정의한다.

```
import requests
def get_location(address):
    url = 'https://dapi.kakao.com/v2/local/search/address.json?query=' + address
    headers={"Authorization": 'KakaoAK 7dc21d5852274d162cc3c15163505083'}
    result = requests.get(url, headers=headers).json()
    return result
```

- 특정 주소를 입력하여 검색 결과를 출력한다.

```
result = get_location('서울특별시 강남구 영동대로 607')
result
```

```
{'documents': [{'address': {'address_name': '서울 강남구 삼성동 77-11',
...
'x': '127.059486615422',
'y': '37.5145473024381'}]},
'meta': {'is_end': True, 'pageable_count': 1, 'total_count': 1}]}
```

- 위 주소 검색함수를 수정하여 위도, 경도 값을 구하여 리턴 한다.

```
import requests
import numpy as np
def get_location(address):
    url = 'https://dapi.kakao.com/v2/local/search/address.json?query=' + address
    headers={"Authorization": 'KakaoAK xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'}
    result = requests.get(url, headers=headers).json()
    if result['documents']:
        x = result['documents'][0]['x']
        y = result['documents'][0]['y']
    else:
        x=np.nan
        y=np.nan
    return x, y
```

- 데이터프레임(cb)의 주소(address)를 이용해 위도, 경도 값을 구하여 y, x에 저장한다.

```
for i in range(1, len(cb)):
    address=cb.loc[i, 'address']
    x, y = get_location(address)
    cb.loc[i, 'x'] = x
    cb.loc[i, 'y'] = y
```

cb.head(3)

|   | seq | store     | address                          | phone        | x                | y                |
|---|-----|-----------|----------------------------------|--------------|------------------|------------------|
| 0 | 1   | 삼성봉은사거리점  | 서울시 강남구 영동대로 607 1,2층            | 02-3443-5618 | NaN              | NaN              |
| 1 | 2   | 코엑스웨스턴서울점 | 서울시 강남구 봉은사로 524 코엑스 인터컨티넨탈 아케이드 | 02-569-3103  | 127.057250899584 | 37.5128320848839 |
| 2 | 3   | 삼성오코우드점   | 서울 강남구 테헤란로87길 46 지하 2층          | 02-3466-8507 | 127.058205573267 | 37.5109823973192 |

- 각 칼럼에 NaN이 몇 개 있는지 출력한다. (x 칼럼에 21개, y 칼럼에 21개가 존재한다.)

cb.isnull().sum()

```
seq      0
store    0
address  0
phone    0
x        21
y        21
dtype: int64
```

cb.shape

cb.shape

- 모든 칼럼을 검색해 NaN이면 행을 삭제한다.

cb.dropna(inplace=True)

cb.isnull().sum()

```
seq      0
store    0
address  0
phone    0
x        0
y        0
dtype: int64
```

cb.shape

cb.shape

- 각 칼럼의 정보를 출력한다.

```
cb.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 198 entries, 1 to 218
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   seq         198 non-null    int64
1   store       198 non-null    object
2   address     198 non-null    object
3   phone       198 non-null    object
4   x           198 non-null    object
5   y           198 non-null    object
dtypes: int64(1), object(5)
memory usage: 10.8+ KB
```

- 데이터프레임(cb)을 '커피빈좌표.csv' 파일로 저장 후 다시 읽어 저장한다.

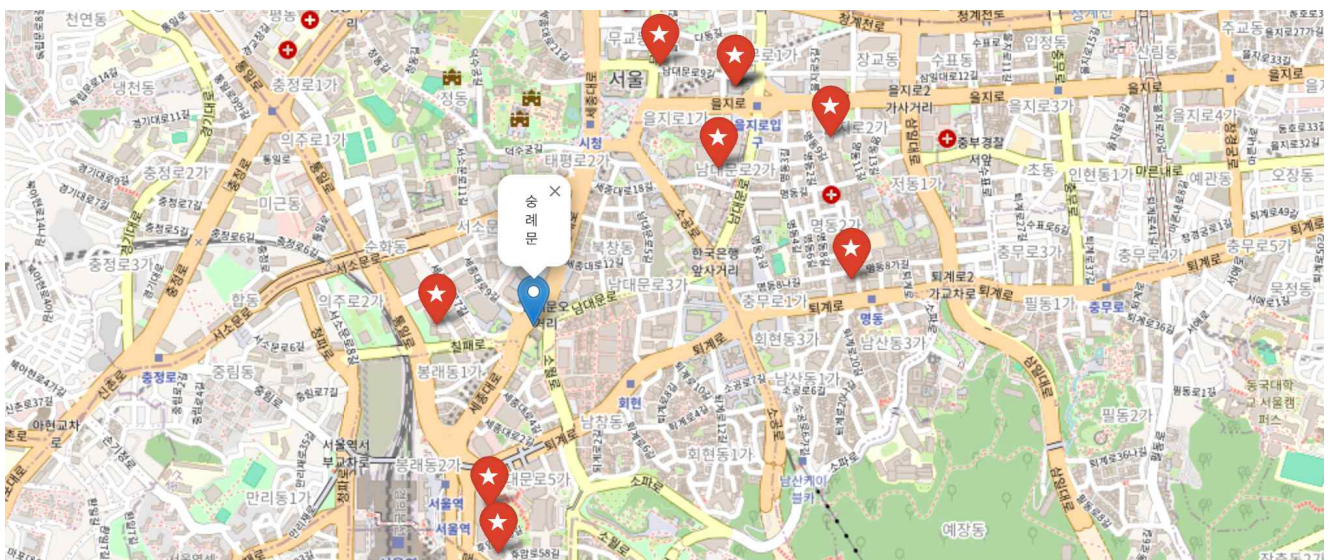
```
cb.to_csv('data/커피빈좌표.csv')
```

```
cb = pd.read_csv('data/커피빈좌표.csv')
```

- 승례문위치를 중심으로 인근 커피빈 위치를 마커로 출력한다.

```
import folium
map = folium.Map(location=[37.560163, 126.975200], zoom_start=15)
for i, store in cb.iterrows():
    folium.Marker(location=[store['y'], store['x']],
                  popup=store['store'],
                  icon=folium.Icon(color='red', icon='star')).add_to(map)
```

```
map.save('data/커피빈맵.html')
```



## 18. 행정구역주소

국가통계포털에서 행정구역 이름을 정제할 데이터를 다운로드하여 커피빈.csv의 주소를 행정구역 주소 체계에 맞게 변경한다.

### ■ 행정구역 주소 체계 데이터 수집하기

- 1) 국가통계포털 사이트(<http://kosis.kr>)에서 '행정구역'으로 데이터를 검색하여 행정구역 이름을 정제할 기준이 되는 데이터를 준비한다.
- 2) 검색 결과 중에서 '행정구역(시군구)별, 성별 인구수'를 선택한다.
- 3) 상위 레벨(시도)만 있고 하위 레벨(군구)은 보이지 않으므로 필요한 데이터를 조회하기 위해 [조회설정]버튼을 클릭한다.
- 4) [행정구역(시군구)별]을 선택하고 [2레벨]을 선택한다.
- 5) [시점]을 선택하고 [2024.01]을 선택한다. [조회] 버튼을 클릭하여 데이터를 조회한다.
- 6) 조회 결과 화면에서 [다운로드] 버튼을 클릭한다. [다운로드] 창이 나타나면 파일 형태를 EXCEL로 선택하고 [다운로드]를 클릭한다.
- 7) 다운로드 파일이름을 '행정구역\_202401.xlsx'로 수정한다.

### ■ 새로운 파일 '18.행정구역주소.ipynb'를 생성한 후 '행정구역\_202401.xlsx' 파일을 읽어 데이터프레임(df)에 저장한다.

```
import pandas as pd

import warnings
warnings.filterwarnings(action='ignore')

df=pd.read_excel('data/행정구역_202401.xlsx', skiprows=1)
df.head(2)
```

|   | 행정구역(시군구)별 | 총인구수 (명) | 남자인구수 (명) | 여자인구수 (명) |
|---|------------|----------|-----------|-----------|
| 0 | 전국         | 51313912 | 25558944  | 25754968  |
| 1 | 서울특별시      | 9384325  | 4538354   | 4845971   |

### ■ 컬럼이름들을 새로운 이름으로 변경하고 새로운 '행정구역1' 컬럼을 추가한다.

```
df1 = df.copy()
df1.columns=['행정구역2', '총인구수', '남자인구수', '여자인구수']
df1['행정구역1'] = ''
df1.head(2)
```

|   | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    | 행정구역1 |
|---|-------|----------|----------|----------|-------|
| 0 | 전국    | 51313912 | 25558944 | 25754968 |       |
| 1 | 서울특별시 | 9384325  | 4538354  | 4845971  |       |

### ■ 컬럼 순서를 아래와 같이 변경한다. 행정구역1이 가장 먼저 출력된다.

```
df2 = df1.copy()
cols = list(df1.columns)
df2 = df1[[cols[-1]] + cols[0:-1]]
df2.head(5)
```

|   | 행정구역1 | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    |
|---|-------|-------|----------|----------|----------|
| 0 |       | 전국    | 51313912 | 25558944 | 25754968 |
| 1 |       | 서울특별시 | 9384325  | 4538354  | 4845971  |

- 시/도 이름을 1레벨(행정구역1)로, 군/구 이름을 2레벨(행정구역2)로 열을 분리하는 작업을 한다. 2레벨 이름이 sides 배열에 있으면 1레벨에 저장하고 2레벨에는 '소계'를 저장한다.

```
sidos = ['특별시', '광역시', '자치시', '경기도', '남도', '북도', '자치도']
for i in range(1, len(df2)):
    for sido in sides:
        if sido in df2.loc[i, '행정구역2']:
            df2.loc[i, '행정구역1'] = df2.loc[i, '행정구역2']
            df2.loc[i, '행정구역2'] = '소계'
```

df2

|     | 행정구역1 | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    |
|-----|-------|-------|----------|----------|----------|
| 0   |       | 전국    | 51313912 | 25558944 | 25754968 |
| 1   | 서울특별시 | 소계    | 9384325  | 4538354  | 4845971  |
| 291 |       | 제주시   | 490888   | 245197   | 245691   |
| 292 |       | 서귀포시  | 183465   | 92381    | 91084    |

293 rows × 5 columns

- 레벨1의 빈자리에는 위행의 레벨1 값으로 채운다.

```
for i in range(2, len(df2)):
    if df2.loc[i, '행정구역1'] == '':
        df2.loc[i, '행정구역1'] = df2.loc[i-1, '행정구역1']
df2.head()
```

|   | 행정구역1 | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    |
|---|-------|-------|----------|----------|----------|
| 0 |       | 전국    | 51313912 | 25558944 | 25754968 |
| 1 | 서울특별시 | 소계    | 9384325  | 4538354  | 4845971  |
| 2 | 서울특별시 | 종로구   | 139378   | 67240    | 72138    |
| 3 | 서울특별시 | 중구    | 121322   | 58651    | 62671    |
| 4 | 서울특별시 | 용산구   | 212175   | 101793   | 110382   |

- 0행의 행정구역1 값을 '전국', 행정구역2 값을 '합계'로 변경한다.

```
df2.loc[0, '행정구역1'] = '전국'
df2.loc[0, '행정구역2'] = '합계'
df2.head()
```

|   | 행정구역1 | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    |
|---|-------|-------|----------|----------|----------|
| 0 | 전국    | 합계    | 51313912 | 25558944 | 25754968 |
| 1 | 서울특별시 | 소계    | 9384325  | 4538354  | 4845971  |
| 2 | 서울특별시 | 종로구   | 139378   | 67240    | 72138    |
| 3 | 서울특별시 | 중구    | 121322   | 58651    | 62671    |
| 4 | 서울특별시 | 용산구   | 212175   | 101793   | 110382   |

- 정제된 행정체계 데이터프레임(df2)을 '행정구역별인구수.csv'로 저장한다.

```
df2.to_csv('data/행정구역별인구수.csv')
```

- 커피빈 주소 데이터를 행정체계에 맞는 주소 데이터로 변경하기 위해 데이터프레임(cb)에 저장한다.

```
cb = pd.read_csv('data/커피빈.csv', index_col=0)
cb.reset_index(inplace=False)
cb.head()
```

|   | seq | store     | address                         | phone        |
|---|-----|-----------|---------------------------------|--------------|
| 0 | 1   | 삼성봉은사거리점  | 서울시 강남구 영동대로 607 1,2층           | 02-3443-5618 |
| 1 | 2   | 코엑스웨스턴서울점 | 서울시 강남구 봉은사로 524 코엑스 인터컨티넨탈아케이드 | 02-569-3103  |
| 2 | 3   | 삼성오크우드점   | 서울 강남구 테헤란로87길 46 지하 2층         | 02-3466-8507 |
| 3 | 4   | 삼성봉은사로점   | 서울시 강남구 봉은사로 628 엘스빌딩1층         | 02-538-4523  |
| 4 | 5   | 도심공항타워점   | 서울시 강남구 테헤란로87길 36 공항타워 1층      | 02-2016-5330 |

- 모든 행의 address 칼럼 값을 띄어쓰기 기준으로 분리한 후 addr 배열에 추가한다.

```
addr = []
for address in cb.address:
    addr.append(str(address).split())
addr
```

```
[['서울시', '강남구', '영동대로', '607', '1,2층'],
 ['서울시', '강남구', '봉은사로', '524', '코엑스', '인터컨티넨탈아케이드'],
 ['서울', '강남구', '테헤란로87길', '46', '지하', '2층'],
 ['서울시', '강남구', '봉은사로', '628', '엘스빌딩1층'],
 ['서울시', '강남구', '테헤란로87길', '36', '공항타워', '1층'],
 ['서울시', '강남구', '영동대로', '511', '트레이드타워', '지하1층'],
 ['서울시', '강남구', '테헤란로87길22', '도심공항터미널', '지하1층'],
 ['서울시', '강남구', '테헤란로521', '파르나스몰', '지하1층'],
 ['서울시', '강남구', '삼성로', '551', '1층,2층'],
 ...,
 ['서울특별시', '강동구', '동남로', '885', '양지빌딩', '1층'],
 ['서울특별시', '종로구', '새문안로', '91', '고려빌딩', '1,2층'],
 ['서울시', '종로구', '새문안로5가길', '7', '1층', '일부'],
 ['서울시', '종로구', '성균관로', '25-2', '성균관대학교', '법학관', 'B2'],
 ['서울시', '종로구', '새문안로3길', '23', '경희궁아침상가4단지105획']]
```

- 커피빈 주소 데이터를 행정구역 주소 체계에 맞게 정리하기 위해 '행정구역별인구수.csv' 파일을 읽어 저장한다.

```
population = pd.read_csv('data/행정구역별인구수.csv', index_col=0)
population.head()
```

|   | 행정구역1 | 행정구역2 | 총인구수     | 남자인구수    | 여자인구수    |
|---|-------|-------|----------|----------|----------|
| 0 | 전국    | 합계    | 51313912 | 25558944 | 25754968 |
| 1 | 서울특별시 | 소계    | 9384325  | 4538354  | 4845971  |
| 2 | 서울특별시 | 종로구   | 139378   | 67240    | 72138    |
| 3 | 서울특별시 | 중구    | 121322   | 58651    | 62671    |
| 4 | 서울특별시 | 용산구   | 212175   | 101793   | 110382   |

- 전국 0행을 제외한 행정구역1 유니크한 값을 구하여 `sidos` 변수에 저장한다.

```
sidos = population[1]['행정구역1'].unique()
sidos

array(['서울특별시', '부산광역시', '대구광역시', '인천광역시', '광주광역시', '대전광역시', '울산광역시',
       '세종특별자치시', '경기도', '강원특별자치도', '충청북도', '충청남도', '전북특별자치도', '전라남도',
       '경상북도', '경상남도', '제주특별자치도'], dtype=object)
```

- 분리한 커피빈 주소 데이터 0행 0열의 2글자를 출력한다.

```
addr[0][0][:2]

'서울'
```

- `addr` 각 행의 주소데이터 0열 앞 두 글자가 `sidos` 배열에 존재하면 0열값을 `sidos`에 속한 값으로 변경한다. 변경 후 각 행의 모든 열을 연결하여 String으로 변경한다. 연결할 경우 열과 열 사이를 한 칸 띄운다.

```
addr2 = []
for i in range(len(addr)):
    for sido in sides:
        if addr[i][0][:2] in sido:
            addr[i][0] = sido
    addr2.append(' '.join(addr[i]))
addr2

['서울특별시 강남구 영동대로 607 1,2층',
 '서울특별시 강남구 봉은사로 524 코엑스 인터컨티넨탈아케이드',
 '서울특별시 강남구 테헤란로87길 46 지하 2층',
 '서울특별시 강남구 봉은사로 628 엘스빌딩1층',
 '서울특별시 강남구 테헤란로87길 36 공향타워 1층',
 '서울특별시 강남구 영동대로 511 트레이드타워 지하1층',
 '서울특별시 강남구 테헤란로87길22 도심공향터미널 지하1층',
 ...,
 '서울특별시 강동구 동남로 885 양지빌딩 1층',
 '서울특별시 종로구 새문안로 91 고려빌딩 1,2층',
 '서울특별시 종로구 새문안로5가길 7 1층 일부',
 '서울특별시 종로구 성균관로 25-2 성균관대학교 법학관 B2',
 '서울특별시 종로구 새문안로3길 23 경희궁아침상가4단지105외']
```

- 위에서 변경한 `addr2` 배열의 값을 새로운 `address2` 칼럼에 저장한다.

```
#cb['address2'] = addr2
addr2 = pd.DataFrame(addr2, columns=['address2'])
cb = pd.concat([cb, addr2], axis=1)
cb.head(3)
```

|   | seq | store     | address                         | phone        | address2                          |
|---|-----|-----------|---------------------------------|--------------|-----------------------------------|
| 0 | 1   | 삼성봉은사 거리점 | 서울시 강남구 영동대로 607 1,2층           | 02-3443-5618 | 서울특별시 강남구 영동대로 607 1,2층           |
| 1 | 2   | 코엑스웨스틴서울점 | 서울시 강남구 봉은사로 524 코엑스 인터컨티넨탈아케이드 | 02-569-3103  | 서울특별시 강남구 봉은사로 524 코엑스 인터컨티넨탈아케이드 |
| 2 | 3   | 삼성오크우드점   | 서울 강남구 테헤란로87길 46 지하 2층         | 02-3466-8507 | 서울특별시 강남구 테헤란로87길 46 지하 2층        |

## 19. 시계열 날씨

시계열 데이터는 시간 순서대로 나열된 데이터를 의미한다. 특정 시점의 값들이 시간의 흐름에 따라 어떻게 변화하는지 분석하여 패턴, 추세, 계절성 등을 파악하는 데 사용됩니다. 예를 들어, 주식 가격, 기온, 환율 등이 시계열 데이터의 대표적인 예시이다.

### ■ 지상 기상 데이터 수집하기

- 1) 기상자료 개방포털(<http://data.kma.go.kr>)에 접속하고 회원가입 후 로그인한다.
- 2) [데이터]-[기상관측]-[지상]을 차례로 선택하여 지상 기상관측 데이터를 조회하는 화면으로 이동한다.
- 3) [검색조건]에서 [자료형태]는 [일 자료], [기간]은 2024년 1월 1일에서 2024년 12월31일까지를 선택한다.
- 4) [지점]은 [전체], 오른쪽 데이터 종류 목록에서 [기온]에서 [평균기온], [최저기온], [최고기온] [강수]에서 [일강수량]을 클릭한다.
- 5) [조회] 버튼 클릭 후 [자료구조]에서 내용을 확인한다.
- 6) [CSV]를 클릭하여 데이터를 '전국기온강수량\_2024.csv' 파일로 저장한다.

- 새로운 파일 '19.시계열날씨.ipynb'를 생성한 후 '전국기온강수량\_2024.csv' 파일을 읽어 df\_tmp에 저장한다.

```
import pandas as pd
df_tmp = pd.read_csv('data/전국기온강수량_2024.csv', encoding='ANSI')
```

```
df_tmp.head()
```

|   | 지점 | 지점명 | 일시         | 평균기온(°C) | 최저기온(°C) | 최고기온(°C) | 일강수량(mm) |
|---|----|-----|------------|----------|----------|----------|----------|
| 0 | 90 | 속초  | 2024-01-01 | 2.7      | 0.9      | 6.1      | 0.4      |
| 1 | 90 | 속초  | 2024-01-02 | NaN      | 1.1      | 7.3      | 0.0      |
| 2 | 90 | 속초  | 2024-01-03 | 3.1      | 0.2      | 6.7      | NaN      |
| 3 | 90 | 속초  | 2024-01-04 | 2.9      | -2.3     | 6.6      | NaN      |
| 4 | 90 | 속초  | 2024-01-05 | 7.7      | 4.7      | 10.3     | NaN      |

```
df_tmp.shape
```

```
(35484, 7)
```

- '일시' 컬럼 타입을 확인 후 날짜 포맷으로 변경한다.

- 1) 데이터프레임(df\_tmp) 첫 행 '일시' 컬럼의 type을 확인한다.

```
type(df_tmp.loc[0, '일시'])
```

```
str
```

- 2) 데이터프레임(df\_tmp) 모든 '일시' 컬럼의 데이터 타입을 날짜 타입으로 변경한다.

```
df_tmp['일시'] = pd.to_datetime(df_tmp['일시'])
```

- 3) 데이터프레임(df\_tmp) 첫 행 '일시' 컬럼의 type을 확인한다.

```
type(df_tmp.loc[0, '일시'])
```

```
pandas._libs.tslibs.timestamps.Timestamp
```



- 데이터프레임(df\_tmp)에 '년', '월', '일' 칼럼을 추가하고 칼럼 이름들을 변경한다.

1) 데이터프레임(df\_tmp)를 데이터프레임(df)로 복사하여 '일시' 칼럼을 index 칼럼으로 지정한다.

```
df = df_tmp.copy()
df.set_index('일시', inplace=True)
df.head()
```

|            | 지점 | 지점명 | 평균기온(°C) | 최저기온(°C) | 최고기온(°C) | 일강수량(mm) |
|------------|----|-----|----------|----------|----------|----------|
| 일시         |    |     |          |          |          |          |
| 2024-01-01 | 90 | 속초  | 2.7      | 0.9      | 6.1      | 0.4      |
| 2024-01-02 | 90 | 속초  | NaN      | 1.1      | 7.3      | 0.0      |
| 2024-01-03 | 90 | 속초  | 3.1      | 0.2      | 6.7      | NaN      |
| 2024-01-04 | 90 | 속초  | 2.9      | -2.3     | 6.6      | NaN      |
| 2024-01-05 | 90 | 속초  | 7.7      | 4.7      | 10.3     | NaN      |

2) index 칼럼을 년, 월, 일로 분리하여 새로운 칼럼을 추가한다.

```
df['년'] = df.index.year
df['월'] = df.index.month
df['일'] = df.index.day
df.head(3)
```

|            | 지점 | 지점명 | 평균기온(°C) | 최저기온(°C) | 최고기온(°C) | 일강수량(mm) | 년    | 월 | 일 |
|------------|----|-----|----------|----------|----------|----------|------|---|---|
| 일시         |    |     |          |          |          |          |      |   |   |
| 2024-01-01 | 90 | 속초  | 2.7      | 0.9      | 6.1      | 0.4      | 2024 | 1 | 1 |
| 2024-01-02 | 90 | 속초  | NaN      | 1.1      | 7.3      | 0.0      | 2024 | 1 | 2 |
| 2024-01-03 | 90 | 속초  | 3.1      | 0.2      | 6.7      | NaN      | 2024 | 1 | 3 |
| 2024-01-04 | 90 | 속초  | 2.9      | -2.3     | 6.6      | NaN      | 2024 | 1 | 4 |
| 2024-01-05 | 90 | 속초  | 7.7      | 4.7      | 10.3     | NaN      | 2024 | 1 | 5 |

3) 데이터프레임(df)의 칼럼 이름들을 아래와 같이 변경한다.

```
df.rename(columns={df.columns[2]:'평균기온',
                    df.columns[3]:'최저기온',
                    df.columns[4]:'최고기온',
                    df.columns[5]:'일강수량' }, inplace=True)
df.head()
```

|            | 지점 | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월 | 일 |
|------------|----|-----|------|------|------|------|------|---|---|
| 일시         |    |     |      |      |      |      |      |   |   |
| 2024-01-01 | 90 | 속초  | 2.7  | 0.9  | 6.1  | 0.4  | 2024 | 1 | 1 |
| 2024-01-02 | 90 | 속초  | NaN  | 1.1  | 7.3  | 0.0  | 2024 | 1 | 2 |
| 2024-01-03 | 90 | 속초  | 3.1  | 0.2  | 6.7  | NaN  | 2024 | 1 | 3 |
| 2024-01-04 | 90 | 속초  | 2.9  | -2.3 | 6.6  | NaN  | 2024 | 1 | 4 |
| 2024-01-05 | 90 | 속초  | 7.7  | 4.7  | 10.3 | NaN  | 2024 | 1 | 5 |

- 데이터프레임(df)의 데이터분석을 위해 결측치를 처리한다.

1) '평균기온', '최저기온', '최고기온', '일강수량'에 NaN 데이터가 존재하는 것을 확인한다.

```
df.isna().sum()
```

```
지점      0
지점명     0
평균기온   43
최저기온   12
최고기온   11
일강수량  20942
년         0
월         0
일         0
dtype: int64
```

2) '일강수량' 칼럼 값이 NaN이면 0.0값으로 채운다.

```
df['일강수량'] = df['일강수량'].fillna(0.0)
df.head(3)
```

|            | 지점 | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월 | 일 |
|------------|----|-----|------|------|------|------|------|---|---|
| 일시         |    |     |      |      |      |      |      |   |   |
| 2024-01-01 | 90 | 속초  | 2.7  | 0.9  | 6.1  | 0.4  | 2024 | 1 | 1 |
| 2024-01-02 | 90 | 속초  | NaN  | 1.1  | 7.3  | 0.0  | 2024 | 1 | 2 |
| 2024-01-03 | 90 | 속초  | 3.1  | 0.2  | 6.7  | 0.0  | 2024 | 1 | 3 |

3) '평균기온' 칼럼이 NaN인 경우 bfill() 함수를 사용해 다음 행과 같은 값으로 채운다. (ffill()함수: 이전 행 값)

```
df['평균기온'] = df['평균기온'].bfill()
df.head(3)
```

|            | 지점 | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월 | 일 |
|------------|----|-----|------|------|------|------|------|---|---|
| 일시         |    |     |      |      |      |      |      |   |   |
| 2024-01-01 | 90 | 속초  | 2.7  | 0.9  | 6.1  | 0.4  | 2024 | 1 | 1 |
| 2024-01-02 | 90 | 속초  | 3.1  | 1.1  | 7.3  | 0.0  | 2024 | 1 | 2 |
| 2024-01-03 | 90 | 속초  | 3.1  | 0.2  | 6.7  | 0.0  | 2024 | 1 | 3 |

4) '최저기온', '최고기온' 칼럼도 NaN인 경우 다음 행 값으로 채우고 NaN 수를 출력해 확인한다.

```
df['최저기온'] = df['최저기온'].bfill()
df['최고기온'] = df['최고기온'].bfill()
df.isna().sum()
```

```
지점      0
지점명     0
평균기온   0
최저기온   0
최고기온   0
일강수량   0
년         0
월         0
일         0
dtype: int64
```

■ 데이터프레임(df\_temp) '지점명' 칼럼을 이용하여 '시도' 칼럼 값을 구한다.

1) df 데이터프레임을 df\_temp 데이터프레임으로 복사한다.

```
df_temp = df.copy()
df_temp.head(3)
```

|   | 일시         | 지점 | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월 | 일 |
|---|------------|----|-----|------|------|------|------|------|---|---|
| 0 | 2024-01-01 | 90 | 속초  | 2.7  | 0.9  | 6.1  | 0.4  | 2024 | 1 | 1 |
| 1 | 2024-01-02 | 90 | 속초  | 3.1  | 1.1  | 7.3  | 0.0  | 2024 | 1 | 2 |
| 2 | 2024-01-03 | 90 | 속초  | 3.1  | 0.2  | 6.7  | 0.0  | 2024 | 1 | 3 |

2) '지점명' 칼럼의 고유향값들을 출력한다.

```
names=df_temp['지점명'].unique()
names
```

```
(97
array(['속초', '북춘천', '철원', '동두천', '파주', '대관령', '춘천', '백령도', '북강릉', '강릉',
      '동해', '서울', '인천', '원주', '울릉도', '수원', '영월', '충주', '서산', '울진', '청주',
      '대전', '추풍령', '안동', '상주', '포항', '군산', '대구', '전주', '울산', '창원', '광주',
      '부산', '통영', '목포', '여수', '흑산도', '완도', '고창', '순천', '홍성', '서청주', '제주',
      '고산', '성산', '서귀포', '진주', '강화', '양평', '이천', '인제', '홍천', '태백', '정선군',
      '제천', '보은', '천안', '보령', '부여', '금산', '세종', '부안', '임실', '정읍', '남원',
      '장수', '고창군', '영광군', '김제시', '순창군', '북창원', '양산시', '보성군', '강진군', '장흥',
      '해남', '고흥', '의령군', '함양군', '광양시', '진도군', '봉화', '영주', '문경', '청송군',
      '영덕', '의성', '구미', '영천', '경주시', '거창', '합천', '밀양', '산청', '거제', '남해',
      '북부산'], dtype=object))
```

3) 대한민국 행정구역의 '시도명'을 얻기 위해 Kakao 주소검색 API를 사용해 함수를 정의한다.

```
import requests
import numpy as np
def get_location(address):
    url ='https://dapi.kakao.com/v2/local/search/address.json?query=' + address
    headers={"Authorization": 'KakaoAK xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'}
    result = requests.get(url, headers=headers).json()
    if result['documents']:
        return result['documents'][0].get('address')
    else:
        return np.nan
```

4) '지점명' '속초'는 '시도명'이 '강원특별자치도'로 출력된다.

```
get_location('속초')
```

```
{'address_name': '강원특별자치도 속초시',
'b_code': '5121000000',
'h_code': '5121000000',
'main_address_no': '',
'mountain_yn': 'N',
'region_1depth_name': '강원특별자치도',
'region_2depth_name': '속초시',
'region_3depth_h_name': '',
'region_3depth_name': '',
'sub_address_no': '',
'x': '128.591938589225',
'y': '38.2068943352589'}
```

5) '지점명' 북강릉은 '시도명'이 NaN이 출력된다.

```
get_location('북강릉')
```

```
nan
```

6) 모든 '지점명' 값으로 '시도명'을 구하는 함수를 적용하여 '시도명'이 NaN인 '지점명'을 출력한다.

```
for name in names:
    region = get_location(name)
    if(pd.isna(region)):
        print(name)
```

```
북춘천
백령도
북강릉
울릉도
흑산도
서청주
북창원
북부산
```

7) 위에서 구한 '시도명'이 NaN인 '지점명'들을 새로운 '지점명'들로 변경한다. (웅진군, 울릉군, 신안은 새로 추가)

```
addr_aliases = {'백령도':'웅진군', '북강릉':'강릉', '북부산':'부산', '북창원':'창원',
                '북춘천':'춘천', '서청주':'청주', '울릉도':'울릉군', '흑산도':'신안'}
df_temp['지점명']=df_temp['지점명'].apply(lambda x:addr_aliases.get(x, x))
```

```
names=df_temp['지점명'].unique()
```

```
names
```

```
(92,
array(['속초', '춘천', '철원', '동두천', '파주', '대관령', '웅진군', '강릉', '동해', '서울',
      '인천', '원주', '울릉군', '수원', '영월', '충주', '서산', '울진', '청주', '대전', '추풍령',
      '안동', '상주', '포항', '군산', '대구', '전주', '울산', '창원', '광주', '부산', '통영',
      '목포', '여수', '신안', '완도', '고창', '순천', '홍성', '제주', '고산', '성산', '서귀포',
      '진주', '강화', '양평', '이천', '인제', '홍천', '태백', '정선군', '제천', '보은', '천안',
      '보령', '부여', '금산', '세종', '부안', '임실', '정읍', '남원', '장수', '고창군', '영광군',
      '김해시', '순창군', '양산시', '보성군', '강진군', '장흥', '해남', '고흥', '익령군', '함양군',
      '광양시', '진도군', '봉화', '영주', '문경', '청송군', '영덕', '의성', '구미', '영천',
      '경주시', '거창', '합천', '밀양', '산청', '거제', '남해'], dtype=object))
```

8) 모든 '지점명'에 해당하는 '시도명'을 구하여 딕셔너리(addr\_aliases)에 추가한다.

```
addr_cities = {}
for name in names:
    region = get_location(name)
    #print(name, region.get('region_1depth_name'))
    addr_cities[name] = region.get('region_1depth_name')
addr_cities
```

```
('속초': '강원특별자치도',
 '춘천': '강원특별자치도',
 '철원': '강원특별자치도',
 ...
 '합천': '경남',
 '밀양': '경남',
 '산청': '경남',
 '거제': '경남',
 '남해': '경남')
```

9) '지점명'을 이용해 '시도명'을 구하여 데이터프레임(df\_temp) 새로운 '시도' 컬럼에 저장한다.

```
df_temp['시도']=df_temp['지점명'].apply(lambda x:addr_cities.get(x, x))
```

|       | 일시         | 지점  | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월  | 일  | 시도 |
|-------|------------|-----|-----|------|------|------|------|------|----|----|----|
| 35479 | 2024-12-27 | 296 | 부산  | 1.9  | -2.5 | 6.4  | 0.0  | 2024 | 12 | 27 | 부산 |
| 35480 | 2024-12-28 | 296 | 부산  | 1.3  | -2.0 | 5.4  | 0.0  | 2024 | 12 | 28 | 부산 |
| 35481 | 2024-12-29 | 296 | 부산  | 1.6  | -4.8 | 9.6  | 0.0  | 2024 | 12 | 29 | 부산 |
| 35482 | 2024-12-30 | 296 | 부산  | 6.0  | -1.6 | 12.6 | 0.0  | 2024 | 12 | 30 | 부산 |
| 35483 | 2024-12-31 | 296 | 부산  | 5.2  | -3.3 | 9.9  | 0.0  | 2024 | 12 | 31 | 부산 |

▪ 데이터프레임(df\_temp)를 새로운 '전국기온강수량\_2024.csv' 파일로 저장한다.

```
df_temp.to_csv('data/전국기온강수량_2024_전처리.csv')
```

▪ '시도' 컬럼으로 그룹화한 후 '평균기온', '일강수량'의 평균값을 구한다.

```
group_city=df_temp.groupby('시도')[['평균기온', '일강수량']].mean()
group_city
```

|         | 평균기온      | 일강수량     |
|---------|-----------|----------|
| 시도      |           |          |
| 강원특별자치도 | 13.014425 | 3.548507 |
| 경기      | 14.691829 | 3.694502 |
| 경남      | 15.657240 | 4.598244 |
| 경북      | 14.267115 | 3.369187 |
| 대구      | 15.850000 | 3.159563 |
| 대전      | 14.942350 | 3.717213 |
| 부산      | 16.478005 | 4.862295 |
| 서울      | 15.413843 | 4.657468 |
| 세종특별자치시 | 14.523224 | 3.996721 |

▪ '시도'별 '일강수량'이 많은 순으로 정렬하여 출력한다.

```
group_city.sort_values('일강수량', ascending=False).head()
```

|         | 평균기온      | 일강수량     |
|---------|-----------|----------|
| 시도      |           |          |
| 제주특별자치도 | 18.268033 | 5.252732 |
| 부산      | 16.478005 | 4.862295 |
| 서울      | 15.413843 | 4.657468 |
| 경남      | 15.657240 | 4.598244 |
| 전남      | 15.661202 | 4.504508 |

- '지점명' 칼럼으로 그룹화한 후 '평균기온', '일강수량'의 평균값을 구한다.

```
group_name = df_temp.groupby('지점명', as_index=False)[['평균기온', '일강수량']].mean()
group_name
```

|     | 지점명 | 평균기온      | 일강수량     |
|-----|-----|-----------|----------|
| 0   | 강릉  | 14.795219 | 3.893852 |
| 1   | 강진군 | 15.812568 | 4.981421 |
| 2   | 강화  | 12.922951 | 3.659836 |
| 3   | 거제  | 15.995902 | 6.047541 |
| 4   | 거창  | 13.556011 | 3.349727 |
| ... | ... | ...       | ...      |
| 87  | 함양군 | 14.693169 | 3.337158 |
| 88  | 합천  | 15.414208 | 3.462295 |
| 89  | 해남  | 15.562842 | 4.109290 |
| 90  | 홍성  | 14.393989 | 4.707104 |
| 91  | 홍천  | 12.960656 | 3.400820 |

92 rows × 3 columns

- '월' 칼럼으로 그룹화한 후 '평균기온', '일강수량'의 평균값을 구한다.

```
group_month = df_temp.groupby('월')[['평균기온', '일강수량']].mean()
group_month.head()
```

|   | 평균기온      | 일강수량     |
|---|-----------|----------|
| 월 |           |          |
| 1 | 1.312936  | 1.138577 |
| 2 | 4.425098  | 3.799716 |
| 3 | 7.107951  | 2.199701 |
| 4 | 14.944227 | 2.831134 |
| 5 | 17.724090 | 3.867858 |

- '월'별 '최고기온'이 높은 순으로 정렬하여 출력한다.

```
group_month.sort_values('평균기온', ascending=False).head()
```

|   | 평균기온      | 일강수량      |
|---|-----------|-----------|
| 월 |           |           |
| 8 | 28.002196 | 2.698237  |
| 7 | 26.314105 | 11.362841 |
| 9 | 24.813299 | 8.325395  |
| 6 | 22.676564 | 4.850172  |
| 5 | 17.724090 | 3.867858  |

- 데이터프레임(df\_temp)에서 '일강수량'이 많은 순으로 정렬한다.

```
df_temp.sort_values('일강수량', ascending=False).head(3)
```

|       | 일시         | 지점  | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량  | 년    | 월 | 일  | 시도 |
|-------|------------|-----|-----|------|------|------|-------|------|---|----|----|
| 11242 | 2024-09-21 | 155 | 창원  | 23.8 | 21.5 | 26.3 | 397.7 | 2024 | 9 | 21 | 경남 |
| 1661  | 2024-07-17 | 99  | 파주  | 23.4 | 22.1 | 25.4 | 385.7 | 2024 | 7 | 17 | 경기 |
| 11974 | 2024-09-21 | 159 | 부산  | 22.4 | 20.6 | 26.7 | 378.5 | 2024 | 9 | 21 | 부산 |

- 데이터프레임(df\_temp)에서 '최고기온'이 높은 순으로 정렬한다.

```
df_temp.sort_values('최고기온', ascending=False).head(3)
```

|       | 일시         | 지점  | 지점명 | 평균기온 | 최저기온 | 최고기온 | 일강수량 | 년    | 월 | 일 | 시도 |
|-------|------------|-----|-----|------|------|------|------|------|---|---|----|
| 26185 | 2024-08-03 | 257 | 양산시 | 32.0 | 28.4 | 39.3 | 0.0  | 2024 | 8 | 3 | 경남 |
| 32770 | 2024-08-02 | 283 | 경주시 | 32.2 | 26.0 | 38.9 | 0.0  | 2024 | 8 | 2 | 경북 |
| 32771 | 2024-08-03 | 283 | 경주시 | 31.9 | 27.4 | 38.6 | 0.0  | 2024 | 8 | 3 | 경북 |

- 데이터프레임(df\_temp)에서 '최저기온'이 낮은 순으로 정렬한다.

```
df_temp.sort_values('최저기온').head(3)
```

|      | 일시         | 지점  | 지점명 | 평균기온  | 최저기온  | 최고기온  | 일강수량 | 년    | 월 | 일  | 시도      |
|------|------------|-----|-----|-------|-------|-------|------|------|---|----|---------|
| 1853 | 2024-01-25 | 100 | 대관령 | -12.5 | -21.9 | -4.5  | 0.0  | 2024 | 1 | 25 | 강원특별자치도 |
| 1851 | 2024-01-23 | 100 | 대관령 | -17.5 | -20.2 | -14.3 | 0.0  | 2024 | 1 | 23 | 강원특별자치도 |
| 1852 | 2024-01-24 | 100 | 대관령 | -16.2 | -20.1 | -10.9 | 0.0  | 2024 | 1 | 24 | 강원특별자치도 |

- '평균기온', '최저기온', '최고기온', '일강수량' 컬럼들의 정보를 출력한다.

```
df_temp[['평균기온', '최저기온', '최고기온', '일강수량']].describe()
```

|       | 평균기온         | 최저기온         | 최고기온         | 일강수량         |
|-------|--------------|--------------|--------------|--------------|
| count | 35484.000000 | 35484.000000 | 35484.000000 | 35484.000000 |
| mean  | 14.690325    | 10.218513    | 19.908976    | 3.962234     |
| std   | 9.737962     | 10.196335    | 9.898396     | 14.871520    |
| min   | -17.500000   | -21.900000   | -14.300000   | 0.000000     |
| 25%   | 5.900000     | 1.400000     | 11.100000    | 0.000000     |
| 50%   | 15.500000    | 10.400000    | 21.100000    | 0.000000     |
| 75%   | 23.500000    | 19.400000    | 28.300000    | 0.700000     |
| max   | 33.300000    | 30.400000    | 39.300000    | 397.700000   |

```
max(df_temp['최고기온']), min(df_temp['최저기온'])
```

```
(39.3, -21.9)
```

- 월별 최고기온, 최저기온, 평균기온을 구한다.

1) 데이터프레임(df\_temp)에 월별 '최고기온' 칼럼의 최고기온을 구하여 데이터프레임(group\_max)에 저장한다.

```
group_max = df_temp.groupby('월')[['최고기온']].max()
group_max.head()
```

| 최고기온 |      |
|------|------|
| 월    |      |
| 1    | 18.7 |
| 2    | 22.9 |
| 3    | 26.7 |

2) 데이터프레임(df\_temp)에 월별 '최저기온' 칼럼의 최저기온을 구하여 데이터프레임(group\_min)에 저장한다.

```
group_min = df_temp.groupby('월')[['최저기온']].min()
group_min.head()
```

| 최저기온 |       |
|------|-------|
| 월    |       |
| 1    | -21.9 |
| 2    | -15.9 |
| 3    | -15.6 |

2) 데이터프레임(df\_temp)에 월별 '평균기온' 칼럼의 평균기온을 구하여 데이터프레임(group\_mean)에 저장한다.

```
group_mean = df_temp.groupby('월')[['평균기온']].mean()
group_mean.head()
```

| 평균기온 |          |
|------|----------|
| 월    |          |
| 1    | 1.312936 |
| 2    | 4.425098 |
| 3    | 7.107951 |

3) 세 개의 데이터프레임(group\_min, group\_max, group\_mean)을 index 기준으로 내부 병합하여 데이터프레임(merge)에 저장한다.

```
merge = pd.merge(group_min, group_mean, how='inner', left_index=True, right_index=True)
merge = pd.merge(merge, group_max, how='inner', left_index=True, right_index=True)
merge.head()
```

|   | 최저기온  | 평균기온      | 최고기온 |
|---|-------|-----------|------|
| 월 |       |           |      |
| 1 | -21.9 | 1.312936  | 18.7 |
| 2 | -15.9 | 4.425098  | 22.9 |
| 3 | -15.6 | 7.107951  | 26.7 |
| 4 | -3.8  | 14.944227 | 32.2 |
| 5 | -1.3  | 17.724090 | 33.4 |



▪ 시도별 최고기온, 최저기온, 평균기온을 구한다.

1) 데이터프레임(df\_temp)에 시도별 '최고기온' 칼럼의 최고기온을 구하여 데이터프레임(group\_max)에 저장한다.

```
group_max = df_temp.groupby('시도')[['최고기온']].max()
group_max.head()
```

| 최고기온    |      |
|---------|------|
| 시도      |      |
| 강원특별자치도 | 38.5 |
| 경기      | 37.2 |
| 경남      | 39.3 |

2) 데이터프레임(df\_temp)에 시도별 '최저기온' 칼럼의 최저기온을 구하여 데이터프레임(group\_min)에 저장한다.

```
group_min = df_temp.groupby('시도')[['최저기온']].min()
group_min.head()
```

| 최저기온    |       |
|---------|-------|
| 시도      |       |
| 강원특별자치도 | -21.9 |
| 경기      | -15.7 |
| 경남      | -11.5 |

2) 데이터프레임(df\_temp)에 시도별 '평균기온' 칼럼의 평균기온을 구하여 데이터프레임(group\_mean)에 저장한다.

```
group_mean = df_temp.groupby('시도')[['평균기온']].mean()
group_mean.head()
```

| 평균기온    |       |
|---------|-------|
| 시도      |       |
| 강원특별자치도 | -17.5 |
| 경기      | -12.7 |
| 경남      | -7.6  |

3) 세 개의 데이터프레임(group\_min, group\_max, group\_mean)을 index 기준으로 내부 병합하여 데이터프레임(merge)에 저장한다.

```
merge = pd.merge(group_min, group_mean, how='inner', left_index=True, right_index=True)
merge = pd.merge(merge, group_max, how='inner', left_index=True, right_index=True)
merge.head()
```

|         | 최저기온  | 평균기온  | 최고기온 |
|---------|-------|-------|------|
| 시도      |       |       |      |
| 강원특별자치도 | -21.9 | -17.5 | 38.5 |
| 경기      | -15.7 | -12.7 | 37.2 |
| 경남      | -11.5 | -7.6  | 39.3 |
| 경북      | -14.9 | -10.8 | 38.9 |
| 대구      | -8.8  | -6.2  | 37.2 |

## 20. 시계열 주식

삼성전자, 카카오, NAVER, LG전자, 현대차 주식 코드를 조회하고 이 기업들 특정 기간 사이의 주식동향을 분석한다.

- 새로운 파일 '20시계열주식.ipynb'를 생성한 후 FinanceDataReader 패키지를 설치한다. 이 패키지는 한국 주식 가격, 미국 주식 가격, 지수, 환율, 암호 화폐 가격, 종목 리스트 등을 제공하는 API 패키지이다.

```
pip install -U finance-datareader
```

- FinanceDataReader 패키지를 설치하고 한국거래소(KRX) 종목 리스트를 데이터프레임(df\_krx)에 저장한다.

```
import FinanceDataReader as fdr
df_krx = fdr.StockListing('KRX') #한국거래소(KRX) ,뉴욕거래소(NYSE)
df_krx.head(2)
```

|   | Code   | ISU_CD       | Name   | Market | Dept | Close  | ChangeCode | Changes | ChagesRatio | Open   | High   | Low    | Volume  | Amount       |
|---|--------|--------------|--------|--------|------|--------|------------|---------|-------------|--------|--------|--------|---------|--------------|
| 0 | 005930 | KR7005930003 | 삼성전자   | KOSPI  |      | 60850  | 1          | 1050    | 1.76        | 60400  | 61100  | 60200  | 6206890 | 377020623500 |
| 1 | 000660 | KR7000660001 | SK하이닉스 | KOSPI  |      | 290500 | 2          | -1500   | -0.51       | 291000 | 294500 | 287000 | 1111631 | 324144783750 |

- 한국거래소 종목 리스트에서 삼성전자 종목 정보만 출력한다.

```
filt = df_krx['Name']=='삼성전자'
df_krx[filt]
```

|   | Code   | ISU_CD       | Name | Market | Dept | Close | ChangeCode | Changes | ChagesRatio | Open  | High  | Low   | Volume  | Amount       |
|---|--------|--------------|------|--------|------|-------|------------|---------|-------------|-------|-------|-------|---------|--------------|
| 0 | 005930 | KR7005930003 | 삼성전자 | KOSPI  |      | 60850 | 1          | 1050    | 1.76        | 60400 | 61100 | 60200 | 6206890 | 377020623500 |

- 삼성전자, 카카오, NAVER, LG전자, 현대차 종목 정보를 데이터프레임(df\_krx\_my)에 저장한다.

```
names = ['삼성전자', '카카오', 'NAVER', 'LG전자', '현대차']
filt = df_krx['Name'].isin(names)
df_krx_my = df_krx[filt][['Name', 'Code', 'Open', 'High', 'Low', 'Volume']]
df_krx_my.head()
```

|    | Name  | Code   | Open   | High   | Low    | Volume  |
|----|-------|--------|--------|--------|--------|---------|
| 0  | 삼성전자  | 005930 | 60400  | 61100  | 60200  | 6206890 |
| 6  | 현대차   | 005380 | 204000 | 209000 | 203500 | 227193  |
| 7  | NAVER | 035420 | 264000 | 269000 | 262000 | 742639  |
| 15 | 카카오   | 035720 | 59900  | 63400  | 59300  | 2660458 |
| 46 | LG전자  | 066570 | 74300  | 76800  | 74200  | 351423  |

- 오늘 날짜를 'YYYY-MM-DD' 포맷으로 변경하여 today 변수에 저장한다.

```
from datetime import datetime
today = datetime.now().strftime('%Y-%m-%d')
today
```

```
'2025-07-01'
```

- 특정 종목(삼성전자:005930)의 특정 기간의 주가 데이터를 받아 데이터프레임(df\_stock)에 저장한다.

```
start = '2025-01-01'
end = today
df_stock = fdr.DataReader('005930', start, end)
df_stock.head(2)
```

|            | Open  | High  | Low   | Close | Volume   | Change   |
|------------|-------|-------|-------|-------|----------|----------|
| Date       |       |       |       |       |          |          |
| 2025-01-02 | 52700 | 53600 | 52300 | 53400 | 16630538 | 0.003759 |
| 2025-01-03 | 52800 | 55100 | 52800 | 54400 | 19318046 | 0.018727 |

```
df_stock.tail(2)
```

|            | Open  | High  | Low   | Close | Volume   | Change    |
|------------|-------|-------|-------|-------|----------|-----------|
| Date       |       |       |       |       |          |           |
| 2025-06-30 | 61200 | 61200 | 59800 | 59800 | 17110294 | -0.016447 |
| 2025-07-01 | 60400 | 61100 | 60200 | 60850 | 6406796  | 0.017559  |

- 데이터프레임(df\_krx\_my)에서 종목 코드만 추출하여 list 타입으로 변경 후 my\_codes 변수에 저장한다.

```
my_codes = df_krx_my['Code']
my_codes = list(my_codes)
my_codes

['005930', '005380', '035420', '035720', '066570']
```

- 데이터프레임(df\_krx\_my)에서 종목 이름만 추출하여 list 타입으로 변경 후 my\_names 변수에 저장한다.

```
my_names = df_krx_my['Name'].values
my_names = list(my_names)
my_names

['삼성전자', '현대차', 'NAVER', '카카오', 'LG전자']
```

- 삼성전자, 카카오, NAVER, LG전자, 현대차 특정 기간의 주가 데이터를 데이터프레임(df\_stock\_my)에 저장한다. 각 기업들 Code에 해당하는 종가(Close) 가격이 일자별로 저장된다.

```
df_stock_my = fdr.DataReader(my_codes, start, end)
df_stock_my.head(2)
```

|            | 005930 | 005380 | 035420 | 035720 | 066570 |
|------------|--------|--------|--------|--------|--------|
| Date       |        |        |        |        |        |
| 2025-01-02 | 53400  | 211500 | 193800 | 37450  | 84000  |
| 2025-01-03 | 54400  | 213000 | 200500 | 38050  | 85100  |
| 2025-01-06 | 55900  | 211500 | 208500 | 39150  | 85400  |
| 2025-01-07 | 55400  | 211000 | 208000 | 38650  | 85600  |
| 2025-01-08 | 57300  | 213500 | 207000 | 37400  | 86800  |

- 데이터프레임(df\_stock\_my)의 컬럼 이름을 my\_names로 변경한다.

```
df_stock_my.columns = my_names
df_stock_my.tail(2)
```

|            | 삼성전자  | 현대차    | NAVER  | 카카오   | LG전자  |
|------------|-------|--------|--------|-------|-------|
| Date       |       |        |        |       |       |
| 2025-06-30 | 59800 | 203500 | 262500 | 60000 | 73800 |
| 2025-07-01 | 60850 | 208000 | 264500 | 62000 | 75800 |

- 삼성전자, 카카오, NAVER, LG전자, 현대차 특정 기간의 주가 데이터를 데이터프레임(df\_stock\_all)에 저장한다.

```
import pandas as pd

#5기업의 주식 데이터를 추가할 데이터프레임(df_stock_all)을 정의한다.
df_stock_all = pd.DataFrame()
for idx, code in enumerate(my_codes):
    #주가 데이터를 읽어 데이터프레임(df)에 저장한다.
    df = fdr.DataReader(code, start, end)
    #Code, Name 컬럼을 새로 추가한다.
    df['Code'] = code
    df['Name'] = my_names[idx]
    #데이터프레임(df_stock_all)과 데이터프레임(df)을 행 기준으로 연결한다.
    df_stock_all = pd.concat([df_stock_all, df], axis=0)
    print(df.shape, df_stock_all.shape)
```

```
(119, 8) (119, 8)
(119, 8) (238, 8)
(119, 8) (357, 8)
(119, 8) (476, 8)
(119, 8) (595, 8)
```

```
df_stock_all.head(3)
```

|            | Open  | High  | Low   | Close | Volume   | Change   | Code   | Name |
|------------|-------|-------|-------|-------|----------|----------|--------|------|
| Date       |       |       |       |       |          |          |        |      |
| 2025-01-02 | 52700 | 53600 | 52300 | 53400 | 16630538 | 0.003759 | 005930 | 삼성전자 |
| 2025-01-03 | 52800 | 55100 | 52800 | 54400 | 19318046 | 0.018727 | 005930 | 삼성전자 |
| 2025-01-06 | 54400 | 56200 | 54300 | 55900 | 19034284 | 0.027574 | 005930 | 삼성전자 |

```
df_stock_all.tail(3)
```

|            | Open  | High  | Low   | Close | Volume | Change    | Code   | Name |
|------------|-------|-------|-------|-------|--------|-----------|--------|------|
| Date       |       |       |       |       |        |           |        |      |
| 2025-06-27 | 74800 | 75200 | 73700 | 74100 | 434975 | -0.008032 | 066570 | LG전자 |
| 2025-06-30 | 74300 | 74800 | 73700 | 73800 | 422420 | -0.004049 | 066570 | LG전자 |
| 2025-07-01 | 74300 | 76800 | 74200 | 75500 | 580141 | 0.023035  | 066570 | LG전자 |

- 'Code', 'Name' 칼럼을 앞으로 출력되도록 칼럼의 위치를 변경하여 데이터프레임(df\_stock)에 저장한다.

```
cols = list(df_stock_all.columns)
cols
```

```
['Open', 'High', 'Low', 'Close', 'Volume', 'Change', 'Code', 'Name']
```

```
df_stock = df_stock_all[cols[-2:] + cols[0:-2]]
df_stock.head(3)
```

|            | Code   | Name | Open  | High  | Low   | Close | Volume   | Change   |
|------------|--------|------|-------|-------|-------|-------|----------|----------|
| Date       |        |      |       |       |       |       |          |          |
| 2025-01-02 | 005930 | 삼성전자 | 52700 | 53600 | 52300 | 53400 | 16630538 | 0.003759 |
| 2025-01-03 | 005930 | 삼성전자 | 52800 | 55100 | 52800 | 54400 | 19318046 | 0.018727 |
| 2025-01-06 | 005930 | 삼성전자 | 54400 | 56200 | 54300 | 55900 | 19034284 | 0.027574 |

- 데이터프레임(df\_stock)에 Year, Month, Day 칼럼을 추가한다.

```
df_stock.loc[:, 'Year'] = df_stock.index.year
df_stock.loc[:, 'Month'] = df_stock.index.month
df_stock.loc[:, 'Day'] = df_stock.index.day
df_stock.head(3)
```

|            | Code   | Name | Open  | High  | Low   | Close | Volume   | Change   | Year | Month | Day |
|------------|--------|------|-------|-------|-------|-------|----------|----------|------|-------|-----|
| Date       |        |      |       |       |       |       |          |          |      |       |     |
| 2025-01-02 | 005930 | 삼성전자 | 52700 | 53600 | 52300 | 53400 | 16630538 | 0.003759 | 2025 | 1     | 2   |
| 2025-01-03 | 005930 | 삼성전자 | 52800 | 55100 | 52800 | 54400 | 19318046 | 0.018727 | 2025 | 1     | 3   |
| 2025-01-06 | 005930 | 삼성전자 | 54400 | 56200 | 54300 | 55900 | 19034284 | 0.027574 | 2025 | 1     | 6   |

- 전처리 작업을 마친 데이터프레임(df\_stock)을 '전처리주가데이터\_2025.csv' 파일로 저장한다.

```
df_stock.to_csv('data/전처리주가데이터_2025.csv')
```

- 데이터프레임(df\_stock) 안에 있는 숫자 데이터에 대한 간단한 요약을 출력한다.

```
df_stock[df_stock.columns[2:8]].describe()
```

|       | Open          | High          | Low           | Close         | Volume       | Change     |
|-------|---------------|---------------|---------------|---------------|--------------|------------|
| count | 595.000000    | 595.000000    | 595.000000    | 595.000000    | 5.950000e+02 | 595.000000 |
| mean  | 116505.462185 | 118190.294118 | 114914.495798 | 116479.579832 | 4.806514e+06 | 0.001566   |
| std   | 72825.193588  | 73947.206101  | 71724.504329  | 72859.832042  | 7.569804e+06 | 0.025942   |
| min   | 35900.000000  | 36350.000000  | 35700.000000  | 35750.000000  | 1.728870e+05 | -0.090778  |
| 25%   | 55000.000000  | 55550.000000  | 54450.000000  | 54950.000000  | 5.366625e+05 | -0.011604  |
| 50%   | 76900.000000  | 78400.000000  | 75800.000000  | 77200.000000  | 9.126600e+05 | 0.000000   |
| 75%   | 196400.000000 | 198500.000000 | 194000.000000 | 196850.000000 | 4.919504e+06 | 0.011884   |
| max   | 288500.000000 | 295000.000000 | 277500.000000 | 290500.000000 | 4.218628e+07 | 0.179177   |

- 데이터프레임(df\_stock)을 'Code'로 그룹화 하여 'Close', 'Volume' 칼럼의 평균을 구한다.

```
group=df_stock.groupby('Code')[['Close', 'Volume']].mean()
group
```

|        | Close         | Volume       |
|--------|---------------|--------------|
| Code   |               |              |
| 005380 | 198882.352941 | 7.528896e+05 |
| 005930 | 56341.176471  | 1.804420e+07 |
| 035420 | 207773.949580 | 9.511712e+05 |
| 035720 | 42655.882353  | 3.768271e+06 |
| 066570 | 76744.537815  | 5.160391e+05 |

- 데이터프레임(df\_stock)을 'Code', 'Name'으로 그룹화 하여 'Close', 'High' 평균을 구한다. as\_index=False 옵션은 index를 일반 칼럼으로 변경한다.

```
group=df_stock.groupby(['Code', 'Name'], as_index=False)[['Close', 'High']].mean()
group
```

|   | Code   | Name  | Close         | High          |
|---|--------|-------|---------------|---------------|
| 0 | 005380 | 현대차   | 198882.352941 | 201579.411765 |
| 1 | 005930 | 삼성전자  | 56341.176471  | 57015.966387  |
| 2 | 035420 | NAVER | 207773.949580 | 211014.285714 |
| 3 | 035720 | 카카오   | 42655.882353  | 43542.647059  |
| 4 | 066570 | LG전자  | 76744.537815  | 77799.159664  |

- 데이터프레임(df\_stock)을 'Month', 'Code', 'Name'으로 그룹화 하여 'Close', 'Volume' 평균을 구한다.

```
group=df_stock.groupby(['Month', 'Code', 'Name'])[['Close', 'Volume']].mean()
group
```

|       |        |       | Close         | Volume       |
|-------|--------|-------|---------------|--------------|
| Month | Code   | Name  |               |              |
| 1     | 005380 | 현대차   | 213111.111111 | 7.834582e+05 |
|       | 005930 | 삼성전자  | 54361.111111  | 1.904949e+07 |
|       | 035420 | NAVER | 204933.333333 | 6.590726e+05 |
|       | 035720 | 카카오   | 37063.888889  | 1.848812e+06 |
|       | 066570 | LG전자  | 84933.333333  | 4.847957e+05 |
| ...   |        |       |               |              |
| 7     | 005380 | 현대차   | 210000.000000 | 1.016089e+06 |
|       | 005930 | 삼성전자  | 60300.000000  | 1.237644e+07 |
|       | 035420 | NAVER | 260500.000000 | 1.721586e+06 |
|       | 035720 | 카카오   | 60000.000000  | 4.739281e+06 |
|       | 066570 | LG전자  | 75400.000000  | 6.257610e+05 |

- 위 예제 그룹화한 데이터프레임(group)의 7월 데이터만 출력한다.

```
group.loc[7]
```

|        |       | Close    | Volume     |
|--------|-------|----------|------------|
| Code   | Name  |          |            |
| 005380 | 현대차   | 210000.0 | 1016089.0  |
| 005930 | 삼성전자  | 60300.0  | 12376435.0 |
| 035420 | NAVER | 260500.0 | 1721586.0  |
| 035720 | 카카오   | 60000.0  | 4739281.0  |
| 066570 | LG전자  | 75400.0  | 625761.0   |

- 위 예제 그룹화한 데이터프레임(group)의 7월 '현대차' 데이터만 출력한다.

```
group.loc[7, '005380']
```

|      | Close    | Volume    |
|------|----------|-----------|
| Name |          |           |
| 현대차  | 210000.0 | 1016089.0 |

- 위 예제 그룹화한 데이터프레임(group)의 1월 데이터를 'Close'순으로 내림차순 정렬한다.

```
group_1 = group.loc[1]
```

```
group_1.sort_values('Close', ascending=False)
```

|        |       | Close         | Volume       |
|--------|-------|---------------|--------------|
| Code   | Name  |               |              |
| 005380 | 현대차   | 213111.111111 | 7.834582e+05 |
| 035420 | NAVER | 204933.333333 | 6.590726e+05 |
| 066570 | LG전자  | 84933.333333  | 4.847957e+05 |
| 005930 | 삼성전자  | 54361.111111  | 1.904949e+07 |
| 035720 | 카카오   | 37063.888889  | 1.848812e+06 |

- 위 예제 그룹화한 데이터프레임(group)의 1월 데이터를 'Name'순으로 내림차순 정렬한다.

```
group_1.reset_index(inplace=True)
```

```
group_1.set_index('Name', inplace=True)
```

```
group_1.sort_index(ascending=False)
```

|       | Code   | Close         | Volume       |
|-------|--------|---------------|--------------|
| Name  |        |               |              |
| 현대차   | 005380 | 213111.111111 | 7.834582e+05 |
| 카카오   | 035720 | 37063.888889  | 1.848812e+06 |
| 삼성전자  | 005930 | 54361.111111  | 1.904949e+07 |
| NAVER | 035420 | 204933.333333 | 6.590726e+05 |
| LG전자  | 066570 | 84933.333333  | 4.847957e+05 |

- 그룹화한 1월 데이터프레임의 'Volume' 칼럼을 정수로 변환하여 새로운 칼럼 'Volume1'에 저장한다.

```
group_1['Volume1'] = group_1['Volume'].astype(int)
sum=group_1['Volume1'].sum()
print(f'{sum:,}원')
```

22,825,629원

- 데이터프레임(df\_stock)을 'Code', 'Name'으로 그룹화하여 'High'의 최댓값을 구하여 group\_max에 저장한다.

```
group_max = df_stock.groupby(['Code', 'Name'])[['High']].max()
group_max
```

|        |       | High   |
|--------|-------|--------|
| Code   | Name  |        |
| 005380 | 현대차   | 229000 |
| 005930 | 삼성전자  | 62000  |
| 035420 | NAVER | 295000 |
| 035720 | 카카오   | 71600  |
| 066570 | LG전자  | 89300  |

- 데이터프레임(df\_stock)을 'Code', 'Name'으로 그룹화하여 'High'의 최솟값을 구하여 group\_min에 저장한다.

```
group_min = df_stock.groupby(['Code', 'Name'])[['Low']].max()
group_min
```

|        |       | Low    |
|--------|-------|--------|
| Code   | Name  |        |
| 005380 | 현대차   | 220500 |
| 005930 | 삼성전자  | 60800  |
| 035420 | NAVER | 277500 |
| 035720 | 카카오   | 67200  |
| 066570 | LG전자  | 85600  |

- group\_max, group\_min 데이터프레임을 'Code', 'Name' index로 inner 조인 후 칼럼명을 변경한다.

```
group_merge = pd.merge(group_max, group_min, how='inner', on=['Code', 'Name'])
group_merge.columns=['High-Max', 'Low-Min']
group_merge
```

|        |       | High-Max | Low-Min |
|--------|-------|----------|---------|
| Code   | Name  |          |         |
| 005380 | 현대차   | 229000   | 220500  |
| 005930 | 삼성전자  | 62000    | 60800   |
| 035420 | NAVER | 295000   | 277500  |
| 035720 | 카카오   | 71600    | 67200   |
| 066570 | LG전자  | 89300    | 85600   |



## • 학사관리 데이터분석 예제

- 아래 4개의 csv 파일(교수, 학생, 강좌, 수강신청)을 생성한다.

### [data] 교수.csv

```
교수번호,교수이름,교수학과,임용일,교수직급,급여
221,이병렬,전산,75/04/03,정교수,3000000
228,이재광,전산,91/09/19,부교수,2500000
311,강승일,전자,94/06/09,부교수,2300000
509,오문환,건축,92/10/14,조교수,2000000
```

### [data] 학생.csv

```
학생번호,학생이름,학생학과,학년,생년월일,지도교수
92414029,서연우,전산,3,73/10/06,228
92414033,김창덕,전산,4,73/10/26,221
92514009,이지행,전자,4,73/11/16,311
92514023,김형명,전자,4,73/08/29,311
92454018,이원구,건축,3,74/09/30,509
95454003,이재영,건축,4,76/02/06,509
95414058,박혜경,전산,4,76/03/12,221
96414404,김수정,전산,3,77/12/22,228
```

### [data] 강좌.csv

```
강좌번호,강좌이름,강의시수,강의실,담당교수,최대인원
C301,파일처리론,3,506,221,100
C401,데이터베이스,3,414,221,80
C421,알고리즘,3,510,228,80
C312,자료구조,2,510,228,100
E221,논리회로,3,304,311,100
A109,한국의건축문화,2,101,509,120
```

### [data] 수강신청.csv

```
강좌번호,학생번호,신청일,점수
C401,92414033,98/03/02,85
C301,92414033,98/03/02,80
C421,92414033,98/03/02,
C401,95414058,98/03/03,90
C301,95414058,98/03/03,80
C312,95414058,98/03/03,80
C401,92514023,98/03/03,
C301,92514023,98/03/03,70
C301,92414029,98/03/03,90
C421,92414029,98/03/03,
C312,92414029,98/03/03,70
E221,92414029,98/03/03,75
A109,92414029,98/03/03,90
C301,92514009,98/03/03,70
E221,92514009,98/03/03,85
C301,96414404,98/03/04,75
C401,96414404,98/03/04,75
C421,96414404,98/03/04,75
C312,92454018,98/03/04,90
E221,92454018,98/03/04,90
A109,95454003,98/03/05,85
C301,95454003,98/03/05,83
E221,95454003,98/03/05,75
```

## 01. 데이터선택

- 새로운 파일 '**01.데이터선택.ipynb**'를 생성한다.
- pandas 라이브러리를 import 한다.

```
import pandas as pd
```

- 교수 파일을 csv 파일로 읽어 데이터프레임 변수에 저장한다.

```
df_pro = pd.read_csv('data/교수.csv')  
df_pro
```

- 전체 교수들의 교수이름 Column을 선택한다.

```
df_pro['교수이름']
```

- 전체 교수들의 교수이름과 교수직급 Column값을 선택한다.

```
df_pro[['교수이름', '교수직급']]
```

- 처음부터 Index3 Row 이전(Index2 Row)까지의 모든 Column값을 선택한다.

```
df_pro[:3]
```

```
df_pro.head(3)
```

- 마지막 Index 부터 2개 Row의 모든 Column값을 선택한다.

```
df_pro.tail(2)
```

- Index1부터 Index3이전(Index2)까지 Row의 모든 Column값을 선택한다.

```
df_pro[1:3]
```

- Index2의 교수이름 Column값을 선택한다.

```
df_pro['교수이름'][2]
```

- Index0부터 5개의 Row의 교수이름, 교수학과 Column값을 선택한다.

```
df_pro[['교수이름', '교수학과']].head()
```

- Index1부터 Index3이전(Index2)까지 Row의 교수이름, 임용일 Column값만 선택한다.

```
df_pro[['교수이름', '임용일']][1:3]
```

- Index1부터 Index3까지 Row의 모든 Column값을 선택한다. (df\_pro[1:3]과 차이 확인)

```
df_pro.loc[1:3]
```

- Index1부터 Index3까지 Row의 교수이름과 임용일 Column값을 선택한다.

```
df_pro.loc[1:2, ['교수이름', '임용일']]
```

- Index0부터 Index2까지 Row들의 교수이름 부터 급여까지 Column값을 선택한다.

```
df_pro.loc[0:2, '교수이름':'급여']
```

- 교수 파일의 Column 이름 목록을 출력한다.

```
df_pro.columns
```

- 교수번호 Column을 Index로 지정한 후 새로운 데이터프레임(df\_index\_pcode)에 저장한다.

```
df_index_pcode = df_pro.set_index('교수번호')  
df_index_pcode
```

- Index221번 Row의 교수이름 Column값을 선택한다.

```
df_index_pcode.loc[221, '교수이름']
```

```
df_index_pcode['교수이름'][221]
```

- Index221, 228번 Row의 교수이름 Column값을 선택한다.

```
df_index_pcode.loc[200:300, '교수이름']
```

```
df_index_pcode['교수이름'][[221, 228]]
```

- 모든 Index행의 교수이름, 교수학과 Column값을 선택한다.

```
df_index_pcode.loc[:, ['교수이름', '교수학과']]
```

- Index300부터 마지막까지 Row의 교수이름부터 교수직급까지 Column값을 선택한다.

```
df_index_pcode.loc[300:, '교수이름':'교수직급']
```

- Index300이전의 Row의 교수이름부터 임용일까지 Column값을 선택한다.

```
df_index_pcode.loc[:300, '교수이름':'임용일']
```

- 데이터프레임(df\_pro)에서 Index3 Row의 모든 Column값을 선택한다.

```
df_pro.iloc[3]
```

- 데이터프레임(df\_pro)에서 Index0 Row의 Index0 값을 선택한다.

```
df_pro.iloc[0, 0]
```

- 데이터프레임(df\_pro)에서 Index0 ~ Index1 Row의 Index0 ~ Index2 Column값을 선택한다.

```
df_pro.iloc[0:2, 0:3]
```

- 데이터프레임(df\_pro)에서 Index3 Row의 모든 Index Column값을 선택한다.

```
df_pro.iloc[3, :]
```

- 데이터프레임(df\_pro)에서 모든 Index해의 Index3 Column값을 선택한다.

```
df_pro.iloc[:, 1]
```

- 데이터프레임(df\_pro)에서 Index1, Index3 Row의 Index0, Index1 Column값을 선택한다.

```
df_pro.iloc[[1,3], [0,1]]
```

- 데이터프레임(df\_pro)에서 Index0, Index3... Row의 모든 Column값을 선택한다.

```
df_pro.iloc[::3, :]
```

- 데이터프레임(df\_pro)에서 모든 Row의 Index0, Index2... Column값을 선택한다.

```
df_pro.iloc[:, ::2]
```

- 데이터프레임(df\_pro)에서 모든 Row의 마지막 Column값을 선택한다.

```
df_pro.iloc[:, -1]
```

```
df_pro.loc[:, '급여']
```

- 데이터프레임(df\_pro)에서 마지막 Row의 모든 Column값을 선택한다.

```
df_pro.iloc[-1, :]
```

```
df_pro.loc[3, :]
```

- 데이터프레임(df\_pro)에서 급여가 2500000원 이상인 Row의 모든 Column값을 선택한다.

```
filt=df_pro['급여'] >= 2500000
df_pro[filt]
```

```
df_pro[df_pro['급여'] >= 2500000]
```

- 데이터프레임(df\_pro)에서 급여가 2500000원 미만인 Row의 모든 Column값을 선택한다.

```
df_pro[~filt]
```

- 데이터프레임(df\_pro)에서 급여가 2500000원 이상인 Row의 교수이름, 급여 Column값을 선택한다.

```
df_pro.loc[filt, ['교수이름', '급여']]
```

- 데이터프레임(df\_pro)에서 교수학과가 전산이면서 교수직급이 부교수인 Row의 모든 Column값을 선택한다.

```
filt=(df_pro['교수학과']=='전산') & (df_pro['교수직급']=='부교수')
df_pro[filt]
```

- 데이터프레임(df\_pro)에서 교수학과가 전산이거나 전자과인 Row의 모든 Column값을 선택한다.

```
filt=(df_pro['교수학과'] == '전산') | (df_pro['교수학과'] == '전자')
df_pro[filt]
```

- 데이터프레임(df\_pro)에서 위 조건에 만족하는 Row의 교수번호, 교수학과 Column값을 선택한다.

```
df_pro.loc[filt, ['교수번호', '교수학과']]
```

```
df_pro[['교수번호', '교수학과']][filt]
```

- 데이터프레임(df\_pro)에서 성이 '이'인 Row의 모든 Column값을 선택한다.

```
filt=df_pro['교수이름'].str.startswith('이')
df_pro[filt]
```

- 데이터프레임(df\_pro)에서 교수이름에 '승'을 포함하는 Row의 모든 Column값을 선택한다.

```
filt = df_pro['교수이름'].str.contains('승')
df_pro[filt]
```

- 데이터프레임(df\_pro)에서 교수직급이 '조교수'이거나 '부교수'인 Row의 모든 Column값을 선택한다.

```
title=['조교수', '부교수']
filt = df_pro['교수직급'].isin(title)
df_pro[filt]
```

## 02. 데이터 정렬

- 새로운 파일 '02.데이터정렬.ipynb'를 생성한다.
- pandas 라이브러리를 import한 후 '학생.csv' 파일을 읽어 데이터프레임(df\_stu)에 저장한다.

```
import pandas as pd
df_stu = pd.read_csv('data/학생.csv')
df_stu
```

- '학생번호' Column을 Index로 지정한 후 데이터프레임에 바로 적용한다.

```
df_stu.set_index('학생번호', inplace=True)
```

```
df_stu
```

- Index '학생번호' 순으로 오름차순 정렬하여 출력한다.

```
df_stu.sort_index()
```

- Index '학생번호' 순으로 내림차순 정렬하여 출력한다.

```
df_stu.sort_index(ascending=False)
```

- Index '학생번호'를 원래대로 Reset한 후 데이터프레임에 바로 적용한다.

```
df_stu.reset_index(inplace=True)
```

- '학생이름' Column값으로 오름차순 정렬하여 출력한다.

```
df_stu.sort_values('학생이름')
```

- '학생이름' Column값으로 내림차순 정렬하여 출력한다.

```
df_stu.sort_values('학생이름', ascending=False)
```

- '학생학과' Column값으로 오름차순 정렬한 후 학과가 같으면 '학년' Column값으로 오름차순 정렬하여 출력한다.

```
df_stu.sort_values(['학생학과', '학년'])
```

- '학생학과' Column값으로 오름차순 정렬한 후 학과가 같으면 '학년' Column값으로 내림차순 정렬하여 출력한다.

```
df_stu.sort_values(['학생학과', '학년'], ascending=[True, False])
```

### 03. 데이터수정

- 새로운 파일 '03.데이터수정.ipynb'를 생성한다.
- pandas 라이브러리를 import한 후 '강좌.csv' 파일을 읽어 데이터프레임(df\_cou)에 저장한다.

```
import pandas as pd
df_cou = pd.read_csv('data/강좌.csv')
df_cou
```

- 강좌이름 Column의 '데이터베이스'를 'Database'로 '알고리즘'을 'Algorithm'으로 변경 후 바로 적용한다.

```
df_cou.replace({'강좌이름':{'데이터베이스':'Database', '알고리즘':'Algorithm'}}, inplace=True)
```

- 강좌이름 Column의 값을 모두 소문자로 변경 후 저장한다.

```
df_cou['강좌이름'] = df_cou['강좌이름'].str.lower()
```

- 강좌이름 Column의 값을 모두 대문자로 변경 후 저장한다.

```
df_cou['강좌이름'] = df_cou['강좌이름'].str.upper()
```

- 강좌이름 Column의 값을 앞 한글자만 대문자로 변경 후 저장한다.

```
df_cou['강좌이름'] = df_cou['강좌이름'].str.capitalize()
```

- 강의실 Column 값에 '호'를 연결해 저장한다. (강의실이 정수이므로 문자로 변경한 후 연결한다.)

```
df_cou['강의실'] = df_cou['강의실'].astype(str) + '호'
```

- 최대인원의 80%를 새로운 수강인원 Column에 저장한 후 수강인원 값을 정수로 변환한다.

```
df_cou['수강인원'] = df_cou['최대인원'] * 0.8
df_cou['수강인원'] = df_cou['수강인원'].astype(int)
```

- 구분 Column를 새로 생성한 후 numpy 라이브러리의 null값을 저장한다.

```
import numpy as np
df_cou['구분'] = np.nan
```

- Index4의 강의실 Column의 값을 '304호'로 수정한다.

```
df_cou.loc[4, '강의실'] = '304호'
```

- Index1의 강좌이름, 강의시수를 '데이터베이스', 4로 수정하여 저장한다.

```
df_cou.loc[1, ['강좌이름', '강의시수']] = ['데이터베이스', 4]
```

- 강의시수 Column이 3시간 이상이면 구분 Column에 '전공' 아니면 '교양'으로 저장한다.

```
filt = df_cou['강의시수'] >= 3  
df_cou.loc[filt, '구분'] = '전공'  
df_cou.loc[~filt, '구분'] = '교양'
```

- 새로운 Index6, 7, 9에 아래 데이터를 추가 저장한다.

```
df_cou.loc[6] = ['C001', '데이터분석', 3, '401호', 221, 50, 30, '전공']  
df_cou.loc[7] = ['C002', '데이터시각화', 3, '401호', 221, 50, 30, '전공']  
df_cou.loc[9] = ['C002', '머신러닝', 3, '401호', 221, 50, 30, '전공']
```

- 데이터프레임(df\_cou)의 구분 Column를 사라지게 한다.

```
df_cou.drop(columns='구분')
```

- 데이터프레임(df\_cou)의 최대인원, 수강인원 Column를 사라지게 한다.

```
df_cou.drop(columns=['최대인원', '수강인원'])
```

- 데이터프레임(df\_cou)에서 Index0, Index4를 사라지게 한다.

```
df_cou.drop(index=[0, 4])
```

- 데이터프레임(df\_cou)에서 구분 Column의 값이 '전공'인 Row의 index 값을 구한다.

```
filt = df_cou['구분'] == '전공'  
df_cou[filt].index
```

- 위조건(구분==전공)을 만족하는 Index값들을 구하여 해당 Row를 사라지게 한다.

```
df_cou.drop(index = df_cou[filt].index)
```

- Index6 Row를 데이터프레임(df\_cou)에서 삭제한다.

```
df_cou.drop(index=6, inplace=True)
```

- Index 7, 9를 데이터프레임(df\_cou)에서 삭제한다.

```
df_cou.drop(index=[7,9], inplace=True)
```



- 데이터프레임(df\_cou)에서 Column 이름들을 list 타입으로 변경하여 저장 후 출력한다.

```
cols = df_cou.columns
cols = list(cols)
```

- 데이터프레임(df\_cou)에서 마지막 Column에 해당하는 값을 Serial로 선택한다.

```
df_cou[cols[-1]]
```

- 데이터프레임(df\_cou)에서 처음 Column에서 마지막 전 Column에 해당하는 값을 데이터프레임으로 선택한다.

```
df_cou[cols[0:-1]]
```

- 마지막 Column(Serial)을 앞으로 가져오고, 나머지 Column(DataFrame)들과 합쳐서 순서를 변경한다.

```
df_cou = df_cou[[cols[-1]] + cols[0:-1]]
```

- 데이터프레임(df\_cou)에서 강좌번호, 강좌이름, 강의실 Column을 이용해 새로운 데이터프레임(df)에 저장한다.

```
df = df_cou[['강좌번호', '강좌이름', '강의실']]
```

- 데이터프레임(df\_cou)의 Column 이름들을 목록으로 출력한다.

```
df.columns
```

- 데이터프레임(df\_cou)의 Column 이름을 'Code', 'Name', 'Room'으로 변경한다.

```
df.columns=['Code', 'Name', 'Room']
```

- 데이터프레임(df\_cou)에서 수강인원 65명 이상인 강좌들을 새로운 데이터프레임(df)에 저장한다.

```
filt = df_cou['수강인원'] >= 65
df = df_cou[filt]
```

- 새로운 데이터프레임(df)를 강좌65.csv 파일에 저장한다. (Index 제거)

```
df.to_csv('data/강좌65.csv', encoding='utf-8-sig', index=False)
```

- 새로운 데이터프레임(df)를 강좌65.txt 파일에 저장한다. (Column을 탭으로 구분)

```
df.to_csv('data/강좌65.txt', sep='\t')
```

- 새로운 데이터프레임(df)를 강좌65.xlsx 파일에 저장한다. (Index Column의 이름을 번호로 수정)

```
df.index.name="번호"
df.to_excel('data/강좌65.xlsx')
```

## 04. 함수적용

- 새로운 파일 '04.함수적용.ipynb'를 생성한다.
- pandas 라이브러리로 수강.csv 파일을 읽어 데이터프레임(df\_enrol)에 저장한다.

```
import pandas as pd
df_enrol = pd.read_csv('data/수강.csv')
```

- 데이터프레임(df\_enrol)의 점수 Column이 null이면 0으로 대체하여 데이터프레임(df\_enrol)에 적용한다.

```
df_enrol.fillna({'점수':0}, inplace=True)
```

- 데이터프레임(df\_enrol)에서 강좌번호 Column이 'E221'인 점수의 개수와 합계를 구한다.

```
filt = df_enrol['강좌번호'] == 'E221'
df = df_enrol[filt]
count = df['점수'].count()
sum = df['점수'].sum()
print(f'합계:{int(sum)}점\n수강인원:{count}명')
```

- 강좌번호를 입력 받아 수강 신청한 학생들의 점수 합계를 구하여 Return하는 함수(sum1)를 정의한다.

```
def sum1(code):
    filt = df_enrol['강좌번호'] == code
    df = df_enrol[filt]
    sum = df['점수'].sum()
    return int(sum)
```

```
print(f'합계:{sum('E221')}')
```

- 강좌.csv 파일을 읽어 데이터프레임(df\_cou)에 저장한다.

```
df_cou=pd.read_csv('data/강좌.csv')
```

- 데이터프레임(df\_cou)의 '강좌번호'값을 매개변수로 sum1()함수를 적용한 후 결과를 '합계'에 저장한다.

```
df_cou['합계'] = df_cou['강좌번호'].apply(sum1)
```

- 강좌번호를 입력 받아 수강 신청한 학생 수를 구하여 Return하는 함수(count1)를 정의한다.

```
def count1(code):
    filt = df_enrol['강좌번호'] == code
    df = df_enrol[filt]
    count = df['점수'].count()
    return int(count)
```

- 강좌번호 'E221'을 수강 신청한 학생 수를 구하여 출력한다.

```
print(f'합계:{count1('E221')}')
```

- 데이터프레임(df\_cou)의 '강좌번호'값을 매개변수로 count()함수를 적용한 후 결과를 '수강인원'에 저장한다.

```
df_cou['수강인원'] = df_cou['강좌번호'].apply(count1)
```

- 데이터프레임(df\_cou)의 '평균' Column에 '합계/수강인원' 계산 값을 저장한다.

```
df_cou['평균'] = df_cou['합계'] / df_cou['수강인원']
```

- 학생번호를 입력 받아 수강 신청한 과목들의 점수 합계를 구하여 Return하는 함수(sum2)를 정의한다.

```
def sum2(code):
    filt = df_enrol['학생번호'] == code
    df = df_enrol[filt]
    sum = df['점수'].sum()
    return int(sum)
```

- 학생.csv 파일을 읽어 데이터프레임(df\_stu)에 저장한다.

```
df_stu = pd.read_csv('data/학생.csv')
df_stu
```

- 데이터프레임(df\_stu)의 '학생번호'값을 매개변수로 sum2함수를 적용한 후 결과를 '합계'에 저장한다.

```
df_stu['합계'] = df_stu['학생번호'].apply(sum2)
```

- '학생번호'를 입력 받아 수강 신청한 강좌 수를 구하여 Return하는 함수(count2)를 정의한다.

```
def count2(code):
    filt = df_enrol['학생번호'] == code
    df = df_enrol[filt]
    count = df['점수'].count()
    return int(count)
```

- 데이터프레임(df\_stu)의 '학생번호'값을 매개변수로 count2()함수를 적용한 후 결과를 '수강과목수'에 저장한다.

```
df_stu['수강과목수'] = df_stu['학생번호'].apply(count2)
```

- 데이터프레임(df\_cou)의 '평균' Column에 '합계/수강과목수' 계산 값을 저장한다.

```
df_stu['평균'] = df_stu['합계'] / df_stu['수강과목수']
```

## 05. 데이터병합

- 새로운 파일 '05.데이터병합.ipynb'를 생성한다.

- pandas 라이브러리를 import 한다.

```
import pandas as pd
```

- '수강.csv' 파일을 읽어 각각 데이터프레임 변수(df\_enrol)에 저장한다.

```
df_enrol = pd.read_csv('data/수강.csv')
```

- '학생.csv' 파일을 읽어 각각 데이터프레임 변수(df\_stu)에 저장한다.

```
df_stu = pd.read_csv('data/학생.csv')
```

- '강좌.csv' 파일을 읽어 각각 데이터프레임 변수(df\_cou)에 저장한다.

```
df_cou = pd.read_csv('data/강좌.csv')
```

- '교수.csv' 파일을 읽어 각각 데이터프레임 변수(df\_pro)에 저장한다.

```
df_pro = pd.read_csv('data/교수.csv')
```

- 데이터프레임(df\_pro)에서 '교수번호', '교수이름' Column을 새로운 데이터프레임(df\_pcode\_pname)을 생성한다.

```
df_pcode_pname = df_pro[['교수번호', '교수이름']]
```

- 데이터프레임(df\_stu)와 데이터프레임(df\_pcode\_pname)을 병합하여 지도교수 정보도 출력한다.

```
df_merge = pd.merge(df_stu, df_pcode_pname, how='inner', left_on='지도교수', right_on='교수번호')
df_merge1.drop(columns=['교수번호'], inplace=True) #교수번호 삭제
```

- 데이터프레임(df\_cou)와 데이터프레임(df\_pcode\_pname)을 병합하여 담당교수 정보도 출력한다.

```
df_merge = pd.merge(df_cou, df_pcode_pname, how='inner', left_on='담당교수', right_on='교수번호')
df_merge
```

- '수강' 파일에서 '강좌번호' 'E221'을 수강신청 한 학생들만 필터링한 후 데이터프레임(df)에 저장한다.

```
df = df_enrol[df_enrol['강좌번호'] == 'E221']
```

- 데이터프레임(df\_stu)에서 '학생번호', '학생이름' Column을 새로운 데이터프레임(df\_scode\_sname)을 생성한다.

```
df_scode_sname = df_stu[['학생번호', '학생이름']]
```

- 위에서 필터링한 데이터프레임과 df\_scode\_sname 데이터프레임을 '학생번호'를 키로 inner merge 한다.

```
df_merge = pd.merge(df, df_scode_sname, how='inner', on='학생번호')
df_merge
```

- 위에서 한 작업을 함수로 만든 후 결과 데이터프레임(df\_merge)을 Return한다.

```
def merge_enrol_stu(code):
    filt = df_enrol['강좌번호'] == code
    df = df_enrol[filt]
    df_scode_sname = df_stu[['학생번호', '학생이름']]
    df_merge = pd.merge(df, df_scode_sname, how='inner', on='학생번호')
    return df_merge
```

- 강좌번호 'E221'에 대하여 mege\_enrol\_stu() 함수를 실행한 후 결과를 출력한다.

```
df_merge = merge_enrol_stu('E221')
df_merge
```

- 수강' 파일에서 '학생번호' 92414033(정수)가 수강신청 강좌들만 필터링한 후 데이터프레임(df)에 저장한다.

```
filt = df_enrol['학생번호'] == 92414033
df = df_enrol[filt]
```

- 데이터프레임(df\_cou)에서 '강좌번호', '강좌이름' Column을 새로운 데이터프레임(df\_lcode\_lname)을 생성한다.

```
df_lcode_lname = df_cou[['강좌번호', '강좌이름']]
```

- 위에서 필터링한 데이터프레임과 df\_lcode\_lname 데이터프레임을 '강좌번호'를 키로 inner merge 한다.

```
df_merge = pd.merge(df, df_lcode_lname, how='inner', on='강좌번호')
df_merge
```

- 위에서 한 작업을 함수로 만든 후 결과 데이터프레임(df\_merge)을 Return한다.

```
def merge_enrol_cou(code):
    filt = df_enrol['학생번호'] == code
    df = df_enrol[filt]
    df_lcode_lname = df_cou[['강좌번호', '강좌이름']]
    df_merge = pd.merge(df, df_lcode_lname, how='inner', on='강좌번호')
    return df_merge
```

- 학생번호 92414033에 대하여 mege\_enrol\_stu() 함수를 실행한 후 결과를 출력한다.

```
df_merge = merge_enrol_cou(92414033)
df_merge
```

## 06. 데이터그룹화

- 새로운 파일 '06.데이터그룹화.ipynb'를 생성한다.
- '수강.csv', '학생.csv', '강좌.csv' 파일을 읽어 각각의 데이터프레임 변수에 저장한다.

```
import pandas as pd
df_enrol = pd.read_csv('data/수강.csv')
df_stu = pd.read_csv('data/학생.csv')
df_cou = pd.read_csv('data/강좌.csv')
```

- 데이터프레임(df\_stu)에서 '학생학과'별로 그룹화를 한 후 '전산'과 학생들을 출력한다.

```
df_stu.groupby('학생학과').get_group('전산')
```

- 데이터프레임(df\_stu)에서 '학생학과'별로 그룹화를 한 후 '전자'과 학생들을 출력한다.

```
df_stu.groupby('학생학과').get_group('전자')
```

- '학생.csv' 파일에 아래 데이터를 추가한 후 파일을 다시 읽어 데이터프레임(df\_stu)에 저장한다.

```
96414405,,전산,3,77/12/22,228
```

```
df_stu = pd.read_csv('data/학생.csv')
```

- 데이터프레임(df\_stu)에서 '학생학과'별로 그룹화를 한 후 학과별 학생 수를 출력한다. (null값도 count에 포함)

```
df_stu.groupby('학생학과')['학생이름'].size()
```

- 데이터프레임(df\_stu)에서 '학생학과'별로 그룹화를 한 후 학과별 학생 수를 출력한다. (null값은 count에 제외)

```
df_stu.groupby('학생학과')['학생이름'].count()
```

- 데이터프레임(df\_stu)에서 unique한 학생학과 목록을 출력 후 학생학과, 학생 수를 출력한다.

```
df_stu['학생학과'].unique()
```

```
df_stu['학생학과'].value_counts()
```

- 데이터프레임(df\_stu)에서 학생학과가 전산과 학생 수를 출력한다.

```
df_stu['학생학과'].value_counts().loc['전산']
```

```
df_stu.groupby('학생학과')['학생이름'].count().loc['건축']
```

- 데이터프레임(df\_stu)에서 '학생학과', '학년'별로 그룹화한 후 학생 수를 출력한다. (null값도 count에 포함)

```
group_stu=df_stu.groupby(['학생학과', '학년'])['학생이름'].size()
df_group = group_stu.to_frame()
df_group
```

- 데이터프레임(df\_stu)에서 '학생학과', '학년'별로 그룹화한 후 학생 수를 출력한다. (null값은 count에 제외)

```
group_stu=df_stu.groupby(['학생학과', '학년'])['학생이름'].count()
df_group = group_stu.to_frame()
df_group
```

- 데이터프레임(df\_stu)에서 '학생학과', '학년'별로 그룹화한 후 학생 수를 출력한다. (null값은 count에 제외)

```
group_stu=df_stu.groupby('학생학과')['학년'].value_counts()
df_group = group_stu.to_frame()
df_group
```

- 위 그룹핑 한 결과(df\_group)에서 '전산'과의 학년별 학생 수를 출력한다.

```
df_group.loc['전산']
```

- 위 그룹핑 한 결과(df\_group)에서 '전산'과의 '3'학년 학생 수를 출력한다.

```
df_group.loc['전산', 3]
```

- 데이터프레임(df\_stu)에서 '학생학과', '학년'별로 그룹화한 후 학생 수를 비율로 출력한다.

```
group_stu = df_stu.groupby('학생학과')['학년'].value_counts(normalize=True)
df_group = group_stu.to_frame('비율')
df_group
```

- 데이터프레임(df\_enrol)에서 '학생번호'별로 그룹화를 한 후 '점수'의 평균을 출력한다.

```
df_enrol.groupby('학생번호')['점수'].mean()
```

- 데이터프레임(df\_enrol)에서 '강좌번호'별로 그룹화를 한 후 '점수'의 평균을 출력한다.

```
df_enrol.groupby('강좌번호')['점수'].mean()
```

- 데이터프레임(df\_enrol)에서 '강좌번호'별로 그룹화를 한 후 'A109'과목의 평균을 출력한다.

```
group_enrol = df_enrol.groupby('강좌번호')['점수'].mean()
group_enrol.loc['A109']
```

- df\_stu, df\_cou, df\_enrol 세 개의 데이터프레임 중 필요한 Column만을 이용해 병합한다.

1) 데이터프레임(df\_stu)에서 '학생번호', '학생이름', '학생학과', '학년' Column들만 추출한다.

```
df_stu = df_stu[['학생번호', '학생이름', '학생학과', '학년']]
```

2) 데이터프레임(df\_cou)에서 '강좌번호', '강좌이름' Column들만 추출한다.

```
df_cou = df_cou[['강좌번호', '강좌이름']]
```

3) df\_enrol, df\_stu 두개의 데이터프레임을 '학생번호'를 키로 inner로 병합하여 새로운 데이터프레임(df\_merge)에 저장한다.

```
df_merge = pd.merge(df_enrol, df_stu, how='inner', on='학생번호')
```

4) df\_merge, df\_cou 두개의 데이터프레임을 '강좌번호'를 키로 inner로 병합하여 데이터프레임(df\_merge)에 저장한다.

```
df_merge = pd.merge(df_merge, df_cou, how='inner', on='강좌번호')
```

4) 데이터프레임(df\_merge)에서 아래 Column들만 추출하여 새로운 데이터프레임(df)에 저장한다.

```
df = df_merge[['학생번호', '학생이름', '학생학과', '학년', '강좌번호', '강좌이름', '점수', '신청일']]
```

- 데이터프레임(df)에서 '학년'별 그룹화한 후 '학년'별 '점수'의 평균값을 출력한다.

```
df.groupby('학년')[['점수']].mean()
```

- 데이터프레임(df)에서 '학년'별 그룹화한 후 '4'학년 학생들의 평균값을 출력한다.

```
df.groupby('학년')[['점수']].mean().loc[4]
```

- 데이터프레임(df)에서 '학생학과', '학년'별로 그룹화한 후 '점수'에 대한 평균을 출력한다.

```
df.groupby(['학생학과', '학년'])[['점수']].mean()
```

- 데이터프레임(df)에서 '학생학과', '학년'별로 그룹화한 후 '전산'과 4학년의 평균을 출력한다.

```
df.groupby(['학생학과', '학년'])[['점수']].mean().loc['전산', 4]
```

- 데이터프레임(df)에서 '학생학과', '학년'별로 그룹화한 후 '점수' 내림차순으로 정렬하여 출력한다.

```
df.groupby(['학생학과', '학년'])[['점수']].mean().sort_values('점수', ascending=False)
```

- 데이터프레임(df)에서 '학생학과', '학년'별로 그룹화한 후 Index(학생학과, 학년)으로 내림차순으로 정렬하여 출력한다.

```
df.groupby(['학생학과', '학년'])[['점수']].mean().sort_index(ascending=False)
```



- 데이터프레임(df)에서 '학생번호'별로 그룹화한 후 '수강과목수'를 데이터프레임(group\_count)으로 변경한다.

```
group_count = df.groupby(['학생번호', '학생이름', '학생학과', '학년'])['강좌번호'].count()
group_count.to_frame('수강과목수')
```

- 데이터프레임(df)에서 '학생번호'별로 그룹화한 후 '평균'을 데이터프레임(group\_mean)으로 변경한다.

```
group_mean = df.groupby(['학생번호', '학생이름', '학생학과', '학년'])['점수'].mean()
group_mean.to_frame('평균')
```

- 데이터프레임(df)에서 '강좌번호'별로 그룹화한 후 '수강인원수'를 데이터프레임(group\_count)으로 변경한다.

```
group_count = df.groupby(['강좌번호', '강좌이름'])['학생번호'].count()
group_count.to_frame('수강인원수')
```

- 데이터프레임(df)에서 '강좌번호'별로 그룹화한 후 '평균'을 데이터프레임(group\_mean)으로 변경한다.

```
group_mean = df.groupby(['강좌번호', '강좌이름'])['점수'].mean()
group_mean.to_frame('평균')
```

- '학생번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '수강과목수'를 Return하는 함수를 작성한다.

```
def count_cou(code):
    filt = df['학생번호']==code
    if df[filt].empty:
        return 0
    else:
        return df[filt]['강좌번호'].count()
```

- '학생번호'값을 함수(count\_cou)에 적용하여 결과를 데이터프레임(df\_stu) '수강과목수'에 저장한다.

```
df_stu['수강과목수'] = df_stu['학생번호'].apply(count_cou)
```

- '학생번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '평균'을 Return하는 함수를 작성한다.

```
def mean(code):
    filt = df['학생번호']==code
    if df[filt].empty:
        return 0
    else:
        return df[filt]['점수'].mean()
```

- '학생번호'값을 함수(mean)에 적용하여 결과를 데이터프레임(df\_stu) '평균'에 저장한다.

```
df_stu['평균'] = df_stu['학생번호'].apply(mean)
```

- '학생번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '합계'을 Return하는 함수를 작성한다.

```
def sum(code):
    filt = df['학생번호'] == code
    if df[filt].empty: return 0
    else: return df[filt]['점수'].sum()
```

- '학생번호'값을 함수(sum)에 적용하여 결과를 데이터프레임(df\_stu) '합계'에 저장한다.

```
df_stu['합계'] = df_stu['학생번호'].apply(sum)
```

- '강좌번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '수강인원수'를 Return하는 함수를 작성한다.

```
def count_cou(code):
    filt = df['강좌번호'] == code
    if df[filt].empty: return 0
    else: return df[filt]['학생번호'].count()
```

- '강좌번호'값을 함수(count\_cou)에 적용하여 결과를 데이터프레임(df\_cou) '수강인원수'에 저장한다.

```
df_cou['수강인원수'] = df_cou['강좌번호'].apply(count_cou)
```

- '강좌번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '평균'을 Return하는 함수를 작성한다.

```
def mean_cou(code):
    filt = df['강좌번호'] == code
    if df[filt].empty: return 0
    else: return df[filt]['점수'].mean()
```

- '강좌번호'값을 함수(count\_cou)에 적용하여 결과를 데이터프레임(df\_cou) '평균'에 저장한다.

```
df_cou['평균'] = df_cou['강좌번호'].apply(mean_cou)
```

- '강좌번호'를 입력 받아 데이터프레임(df)에 없으면 0을 Return, 아니면 '합계'를 Return하는 함수를 작성한다.

```
def sum_cou(code):
    filt = df['강좌번호'] == code
    if df[filt].empty: return 0
    else: return df[filt]['점수'].sum()
```

- '강좌번호'값을 함수(sum\_cou)에 적용하여 결과를 데이터프레임(df\_cou) '합계'에 저장한다.

```
df_cou['합계'] = df_cou['강좌번호'].apply(sum_cou)
```

## 07. 날짜포맷(Date Format)

- 새로운 파일 '07.날짜포맷.ipynb'를 생성하고 list 데이터를 읽어 데이터프레임(df) 변수에 저장한다.

```
import pandas as pd
df = pd.DataFrame({'Birth' : ['75-02-03', '69-05-08', '02-12-17', '70-10-04', '10-04-00']})
```

- 'Birth'열 타입을 확인하면 'object' String 타입이다.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Birth    5 non-null      object
dtypes: object(1)
memory usage: 172.0+ bytes
```

df

|   | Birth    |
|---|----------|
| 0 | 75-02-03 |
| 1 | 69-05-08 |
| 2 | 02-12-17 |
| 3 | 70-10-04 |
| 4 | 10-04-00 |

- 'Birth'열을 'yy-mm-dd' 포맷의 datetime 타입으로 변환하여 'Birthday'열에 저장한다.  
errors='coerce': 변환 시 에러가 나면 null값으로 채운다.

```
df['Birthday'] = pd.to_datetime(df['Birth'], format='%y-%m-%d', errors='coerce')
df
```

|   | Birth    | Birthday   |
|---|----------|------------|
| 0 | 75-02-03 | 1975-02-03 |
| 1 | 69-05-08 | 1969-05-08 |
| 2 | 02-12-17 | 2002-12-17 |
| 3 | 70-10-04 | 1970-10-04 |
| 4 | 10-04-00 | NaT        |

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Birth        5 non-null      object
1   Birthday     4 non-null      datetime64[ns]
dtypes: datetime64[ns](1), object(1)
memory usage: 212.0+ bytes
```

- 'Birthday'열값이 null인 경우 오늘 날짜시간으로 수정한다.

```
import datetime
df.fillna({'Birthday':datetime.datetime.now()}, inplace=True)
df
```

|   | Birth    | Birthday                   |
|---|----------|----------------------------|
| 0 | 75-02-03 | 1975-02-03 00:00:00.000000 |
| 1 | 69-05-08 | 1969-05-08 00:00:00.000000 |
| 2 | 02-12-17 | 2002-12-17 00:00:00.000000 |
| 3 | 70-10-04 | 1970-10-04 00:00:00.000000 |
| 4 | 10-04-00 | 2025-01-21 12:20:09.562749 |

- 'Birthday'열을 'yyyy년mm월dd일 hh시 mm분 ss초' 포맷의 datetime 타입으로 변경해 '생년월일'열에 저장한다.

```
df['생년월일'] = df['Birthday'].dt.strftime('%Y년%m월%d일 %H시 %M분 %S초')
df
```

|   | Birth    | Birthday                   | 생년월일                    |
|---|----------|----------------------------|-------------------------|
| 0 | 75-02-03 | 1975-02-03 00:00:00.000000 | 1975년02월03일 00시 00분 00초 |
| 1 | 69-05-08 | 1969-05-08 00:00:00.000000 | 1969년05월08일 00시 00분 00초 |
| 2 | 02-12-17 | 2002-12-17 00:00:00.000000 | 2002년12월17일 00시 00분 00초 |
| 3 | 70-10-04 | 1970-10-04 00:00:00.000000 | 1970년10월04일 00시 00분 00초 |
| 4 | 10-04-00 | 2025-01-21 12:20:09.562749 | 2025년01월21일 12시 20분 09초 |

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Birth       5 non-null      object
1   Birthday    5 non-null      datetime64[ns]
2   생년월일    5 non-null      object
dtypes: datetime64[ns](1), object(2)
memory usage: 252.0+ bytes
```

- [연습] : 아래 Birth list 데이터를 읽어 'yyyy-mm-dd HH:MM:SS' 포맷으로 변경하는 코드를 작성하시오.

```
df = pd.DataFrame({'Birth' : ['2019-01-01 09:10:00',
                              '2019-01-08 09:20:30',
                              '2019-02-01 10:20:00',
                              '2019-02-02 11:40:50',
                              '07-20-2019 10:00:00']})
```

|   | Birth               |
|---|---------------------|
| 0 | 2019-01-01 09:10:00 |
| 1 | 2019-01-08 09:20:30 |
| 2 | 2019-02-01 10:20:00 |
| 3 | 2019-02-02 11:40:50 |
| 4 | 07-20-2019 10:00:00 |

- pandas 패키지를 import하고 '교수.csv', '학생.csv' 파일을 읽어 데이터프레임 변수로 저장한다.

```
import pandas as pd
df_pro = pd.read_csv('data/교수.csv')
df_stu = pd.read_csv('data/학생.csv')
```

df\_pro

|   | 교수번호 | 교수이름 | 교수학과 | 임용일      | 교수직급 | 급여      |
|---|------|------|------|----------|------|---------|
| 0 | 221  | 이병렬  | 전산   | 75/04/03 | 정교수  | 3000000 |
| 1 | 228  | 이재광  | 전산   | 91/09/19 | 부교수  | 2500000 |
| 2 | 311  | 강승일  | 전자   | 94/06/09 | 부교수  | 2300000 |
| 3 | 509  | 오문환  | 건축   | 92/10/14 | 조교수  | 2000000 |

- index4 행에 새로운 데이터를 입력하다. (00년은 2000년을 의미한다.)

```
df_pro.loc[4]=['601', '홍길동', '전자', '00/12/01', '정교수', 3000000]
```

df\_pro

|   | 교수번호 | 교수이름 | 교수학과 | 임용일      | 교수직급 | 급여      |
|---|------|------|------|----------|------|---------|
| 0 | 221  | 이병렬  | 전산   | 75/04/03 | 정교수  | 3000000 |
| 1 | 228  | 이재광  | 전산   | 91/09/19 | 부교수  | 2500000 |
| 2 | 311  | 강승일  | 전자   | 94/06/09 | 부교수  | 2300000 |
| 3 | 509  | 오문환  | 건축   | 92/10/14 | 조교수  | 2000000 |
| 4 | 601  | 홍길동  | 전자   | 00/12/01 | 정교수  | 3000000 |

- 데이터프레임(df\_pro)의 열 정보를 출력한다.

```
df_pro.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 6 columns):
#   Column   Non-Null Count  Dtype
---  -
0   교수번호  4 non-null      int64
1   교수이름  4 non-null      object
2   교수학과  4 non-null      object
3   임용일    4 non-null      object
4   교수직급  4 non-null      object
5   급여      4 non-null      int64
dtypes: int64(2), object(4)
memory usage: 324.0+ bytes
```

- String 타입의 '임용일'열을 dateType(yy/mm/dd) 포맷으로 변경해 'HireDate'열에 저장한다.  
errors='coerce': 변환 시 에러가 나면 null값으로 채운다.

```
df_pro['HireDate'] = pd.to_datetime(df_pro['임용일'], format='%y/%m/%d', errors='coerce')
```

- 데이터프레임(df\_pro)의 열 정보를 출력한다.

df\_pro.info()

```
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   교수번호    5 non-null     object
1   교수이름    5 non-null     object
2   교수학과    5 non-null     object
3   임용일      5 non-null     object
4   교수직급    5 non-null     object
5   급여        5 non-null     int64
6   HireDate    5 non-null     datetime64[ns]
dtypes: datetime64[ns](1), int64(1), object(5)
memory usage: 320.0+ bytes
```

df\_pro

|   | 교수번호 | 교수이름 | 교수학과 | 임용일      | 교수직급 | 급여      | HireDate   |
|---|------|------|------|----------|------|---------|------------|
| 0 | 221  | 이병렬  | 전산   | 75/04/03 | 정교수  | 3000000 | 1975-04-03 |
| 1 | 228  | 이재광  | 전산   | 91/09/19 | 부교수  | 2500000 | 1991-09-19 |
| 2 | 311  | 강승일  | 전자   | 94/06/09 | 부교수  | 2300000 | 1994-06-09 |
| 3 | 509  | 오문환  | 건축   | 92/10/14 | 조교수  | 2000000 | 1992-10-14 |
| 4 | 601  | 홍길동  | 전자   | 00/12/01 | 정교수  | 3000000 | 2000-12-01 |

- 'HireDate'열을 '%Y년%m월%d' 포맷의 String으로 변환하여 'FmtDate'열에 저장한다.

df\_pro['FmtDate'] = df\_pro['HireDate'].dt.strftime('%Y년%m월%d일')

df\_pro

|   | 교수번호 | 교수이름 | 교수학과 | 임용일      | 교수직급 | 급여      | HireDate   | fmtDate   | FmtDate     |
|---|------|------|------|----------|------|---------|------------|-----------|-------------|
| 0 | 221  | 이병렬  | 전산   | 75/04/03 | 정교수  | 3000000 | 1975-04-03 | 75년04월03일 | 1975년04월03일 |
| 1 | 228  | 이재광  | 전산   | 91/09/19 | 부교수  | 2500000 | 1991-09-19 | 91년09월19일 | 1991년09월19일 |
| 2 | 311  | 강승일  | 전자   | 94/06/09 | 부교수  | 2300000 | 1994-06-09 | 94년06월09일 | 1994년06월09일 |
| 3 | 509  | 오문환  | 건축   | 92/10/14 | 조교수  | 2000000 | 1992-10-14 | 92년10월14일 | 1992년10월14일 |
| 4 | 601  | 홍길동  | 전자   | 00/12/01 | 정교수  | 3000000 | 2000-12-01 | 00년12월01일 | 2000년12월01일 |

- 데이터프레임(df\_pro)의 열 정보를 출력한다.

df\_pro.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   교수번호    4 non-null     int64
1   교수이름    4 non-null     object
2   교수학과    4 non-null     object
3   임용일      4 non-null     object
4   교수직급    4 non-null     object
5   급여        4 non-null     int64
6   Birthday    4 non-null     datetime64[ns]
7   fmtDate     4 non-null     object
dtypes: datetime64[ns](1), int64(2), object(5)
memory usage: 388.0+ bytes
```